

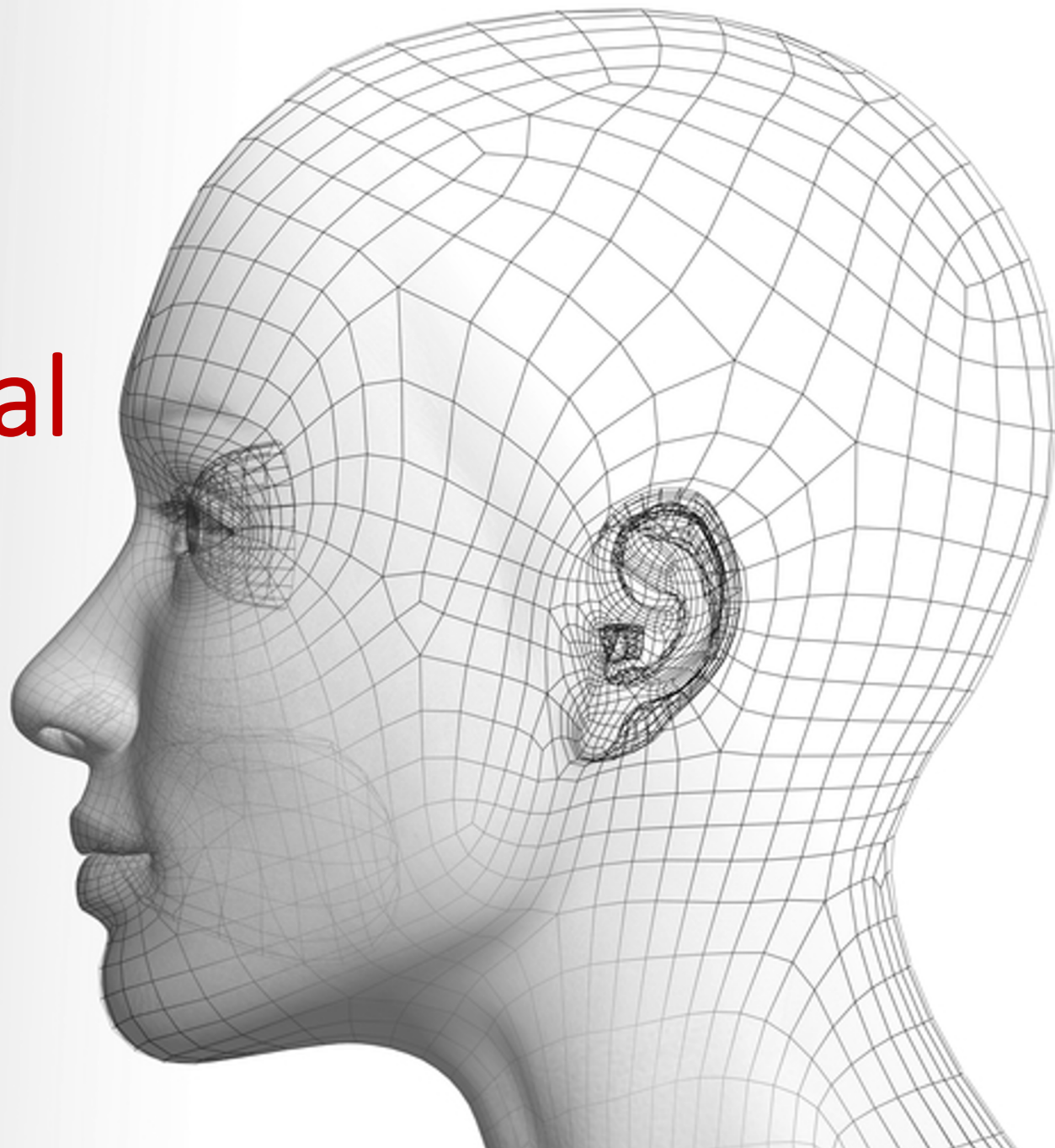
Convolutional Neural Networks

Chen-Change Loy

吕健勤

<http://www.ntu.edu.sg/home/ccloy/>

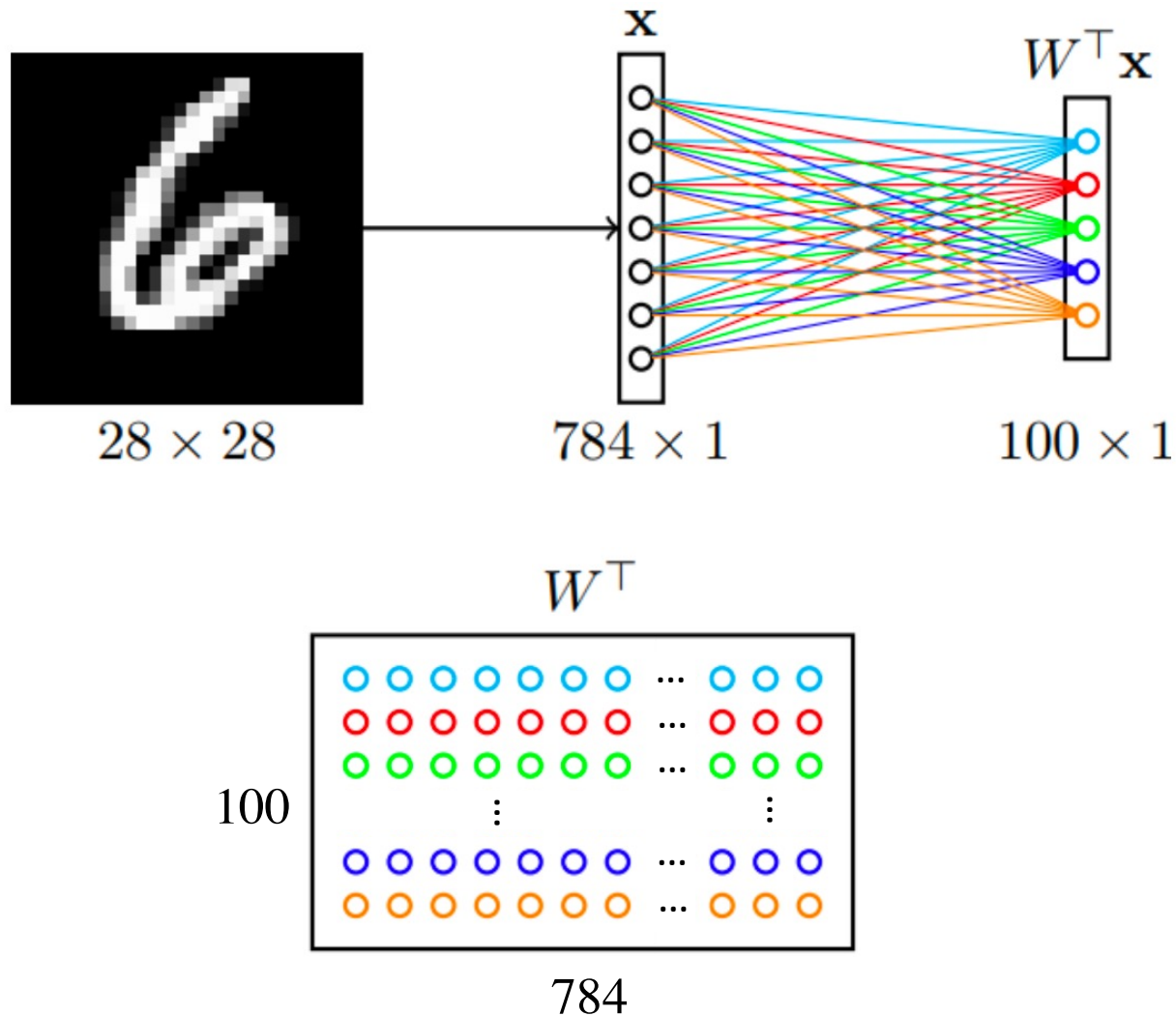
<https://twitter.com/ccloy>



Outline

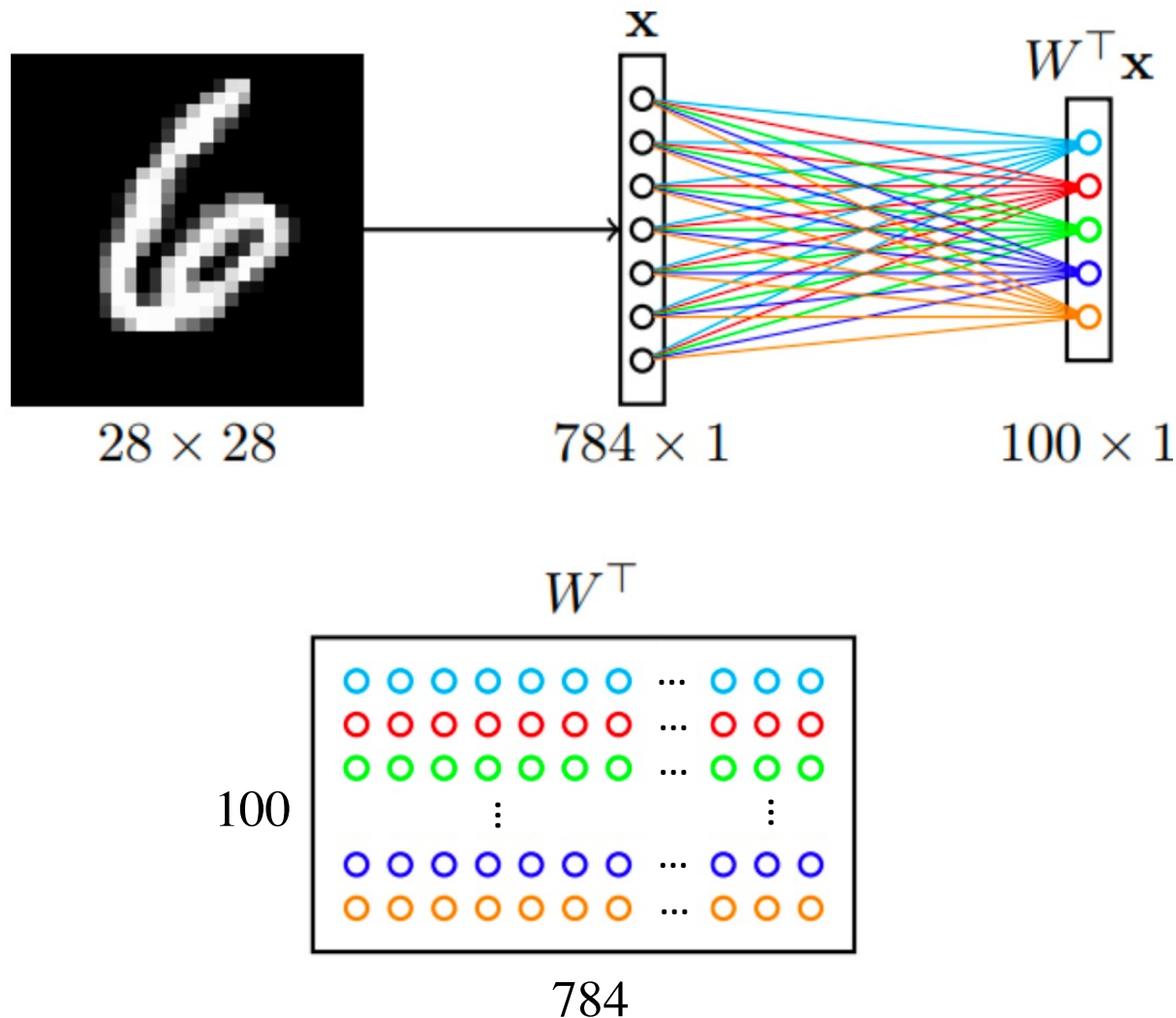
- Basic components in CNN
- CNN architectures
- Short introduction to Colab

Motivation



Let us consider the first layer of a MLP taking images as input. What are the problems with this architecture?

Motivation



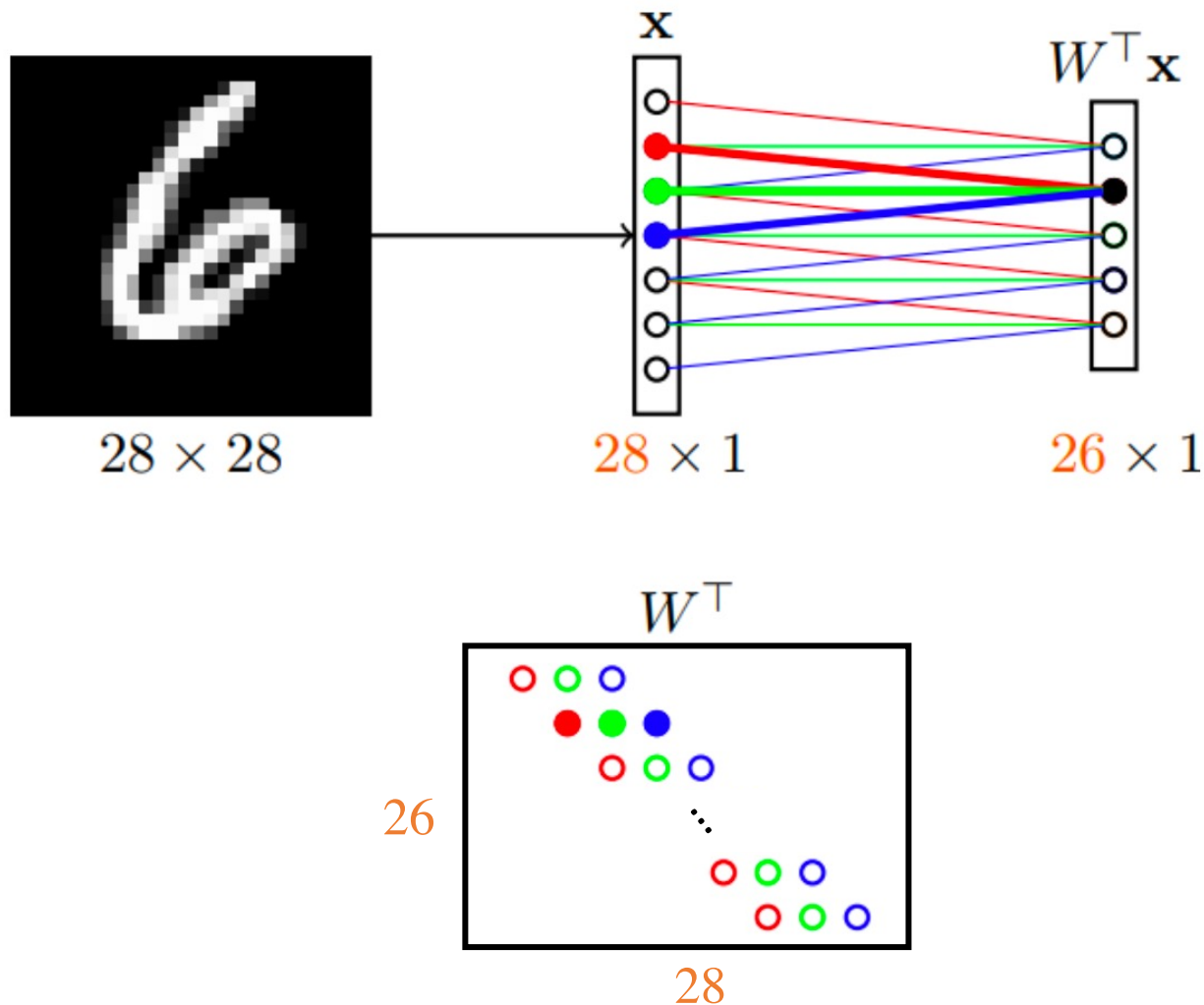
Issues

- Too many parameters: $100 \times 784 + 100$.
 - What if images are $640 \times 480 \times 3$?
 - What if the first layer counts 1000 units?

Fully connected networks where neurons at one level are connected to the neurons in other layers are not feasible for signals of large resolutions and are **computationally expensive** in feedforward and backpropagation computations.

- Spatial organization of the input is destroyed.
 - The network is not invariant to transformations (e.g., translation).

Locally connected networks



Instead, let us only keep a **sparse** set of connections, where all weights having the same color are **shared**.

- The resulting operation can be seen as **shifting** the same weight triplet (**kernel**).
- The set of inputs seen by each unit is its receptive field.

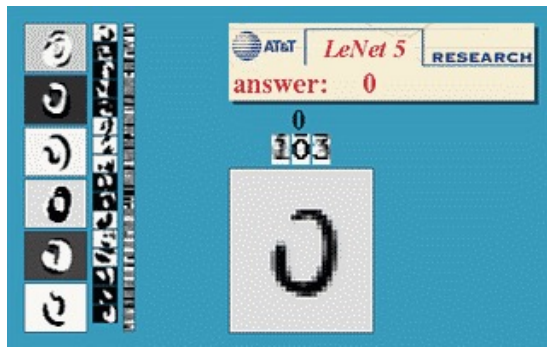
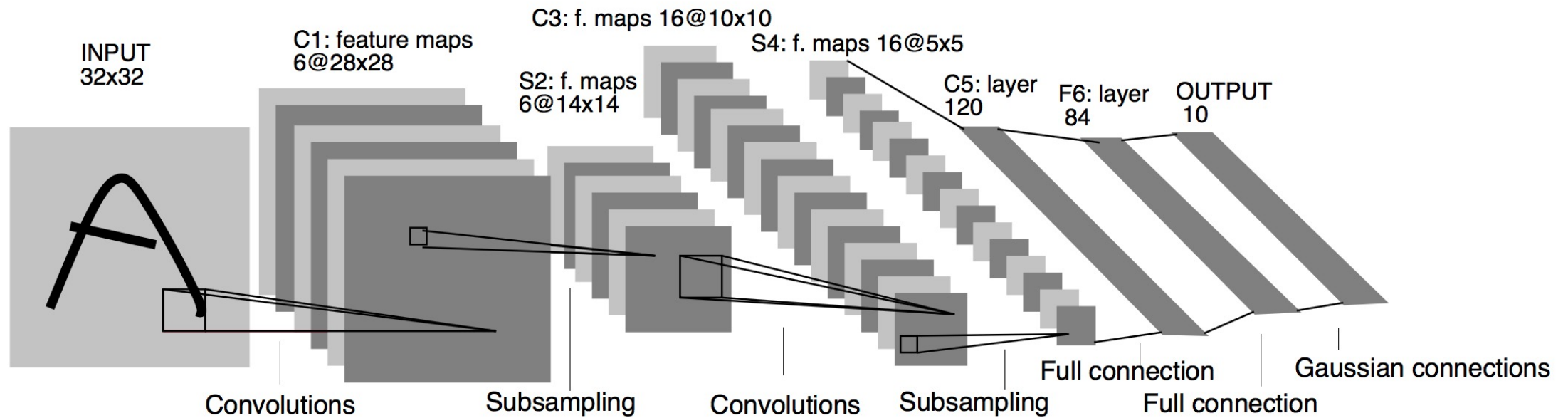
This is a 1D **convolution**, which can be generalized to more dimensions.

Basic Components in CNN

Credits

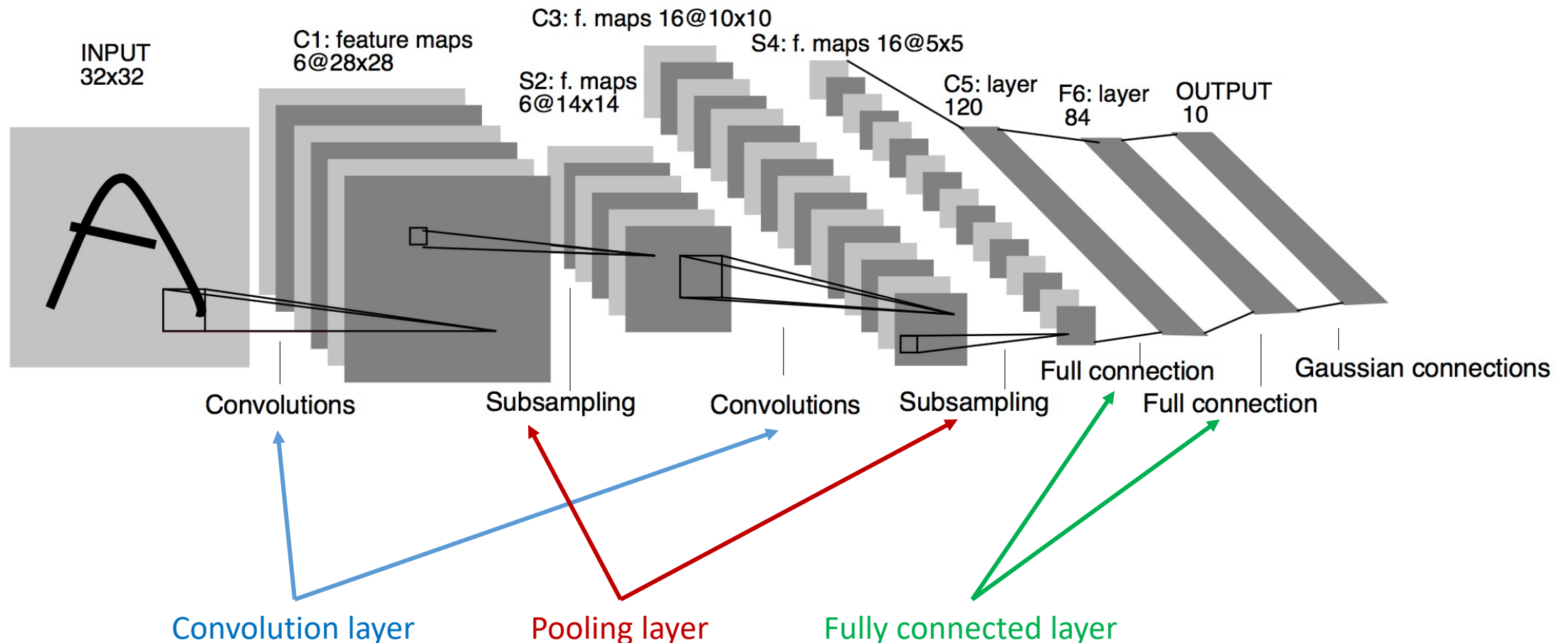
- CS 230: Deep Learning
 - <https://stanford.edu/~shervine/teaching/cs-230.html>
- CS231n: Convolutional Neural Networks for Visual Recognition
 - <http://cs231n.stanford.edu/syllabus.html>

An example of convolutional network: LeNet 5



LeCun et al., 1998

An example of convolutional network: LeNet 5

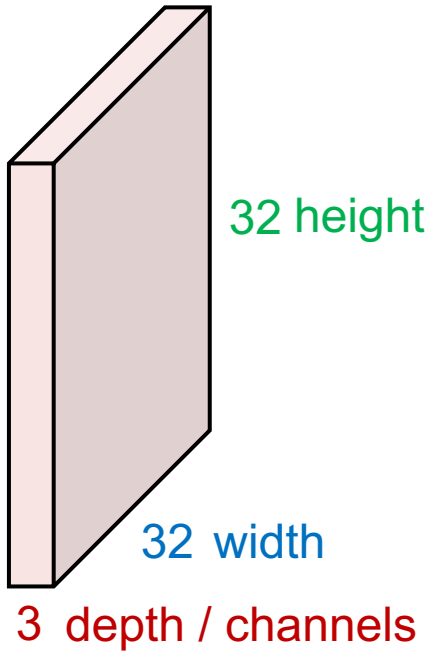


As we go deeper (left to right) the height and width tend to go down and the number of channels increased.

Common layer arrangement: Conv $\rightarrow$ pool $\rightarrow$ Conv $\rightarrow$ pool $\rightarrow$ fully connected $\rightarrow$ fully connected $\rightarrow$ output

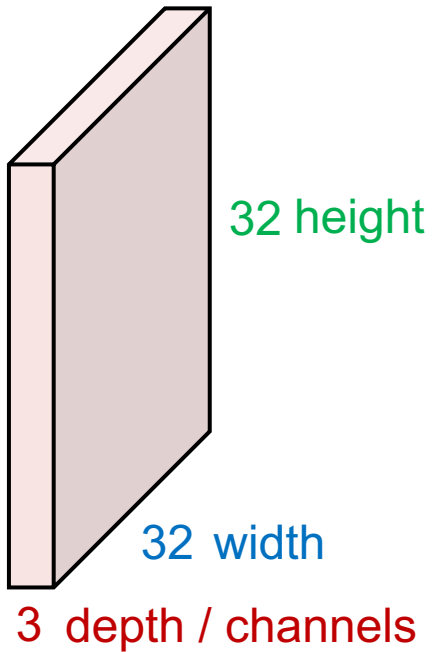
Convolution layer

3x32x32 image



Convolution layer

3x32x32 image



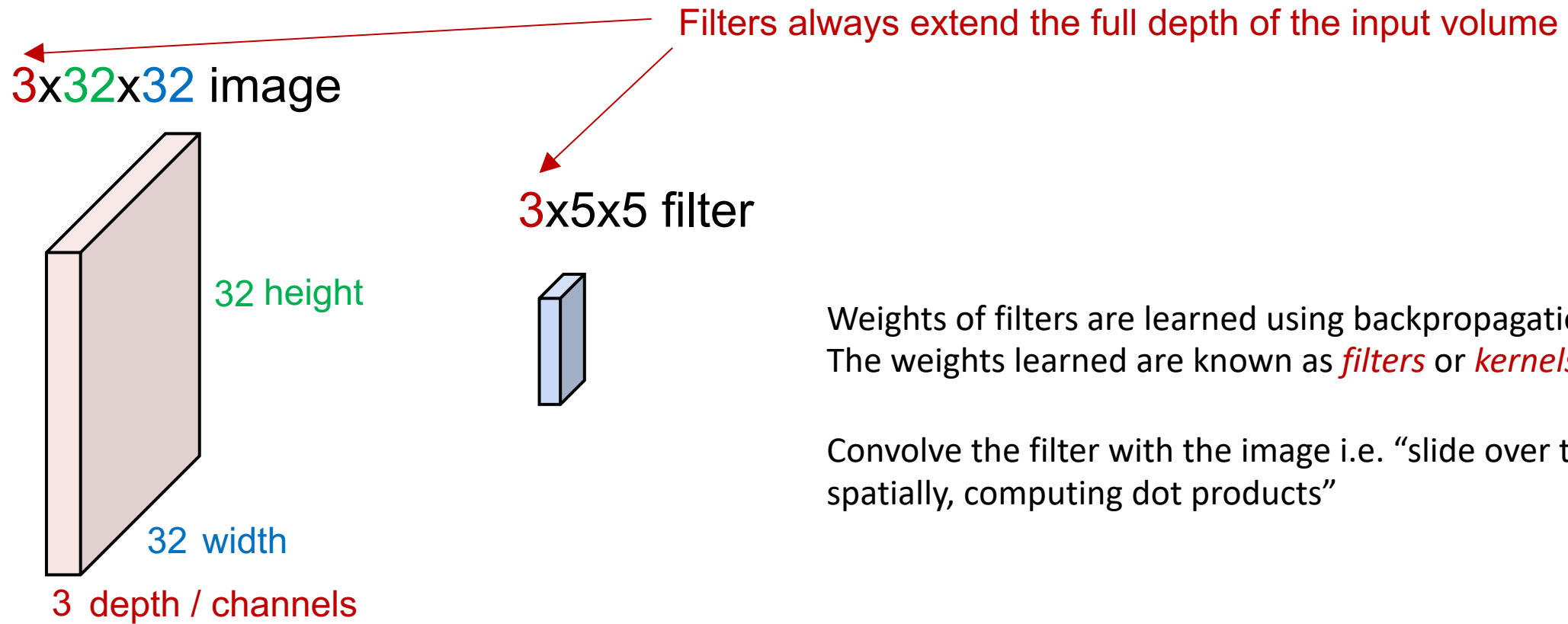
3x5x5 filter



Weights of filters are learned using backpropagation algorithm.
The weights learned are known as *filters* or *kernels*

Convolve the filter with the image i.e. “slide over the image spatially, computing dot products”

Convolution layer

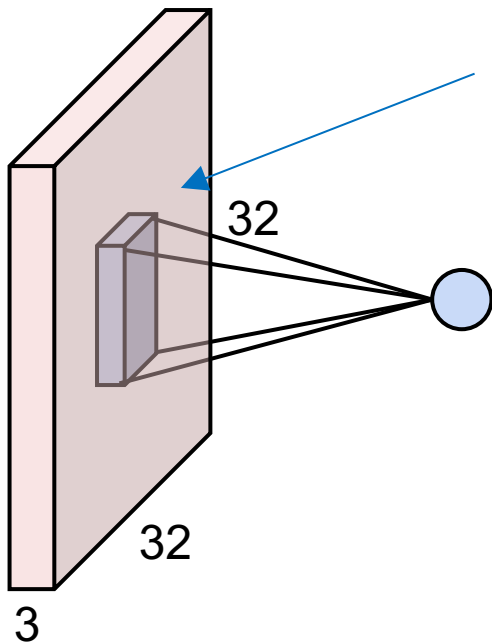


Weights of filters are learned using backpropagation algorithm.
The weights learned are known as *filters* or *kernels*

Convolve the filter with the image i.e. “slide over the image spatially, computing dot products”

Convolution layer

3x32x32 image



3x5x5 filter

1 number:

the result of taking a dot product between the filter and a small 3x5x5 chunk of the image

(i.e. $3 \times 5 \times 5 = 75$ -dimensional dot product + bias)

An example of convolving
a 1x5x5 image with a
1x3x3 filter

1	0	1
0	1	1
0	0	1

Filter (weights or kernel)

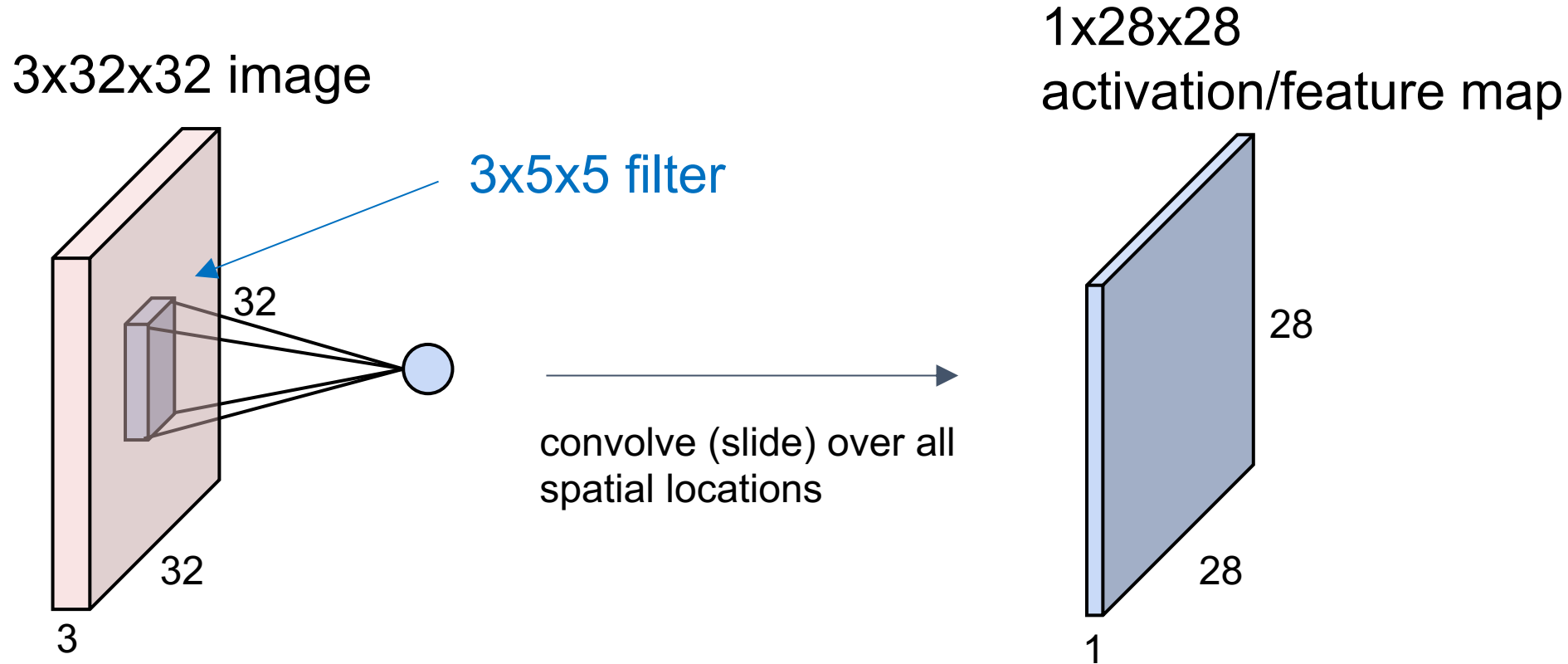
1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

Convolution layer

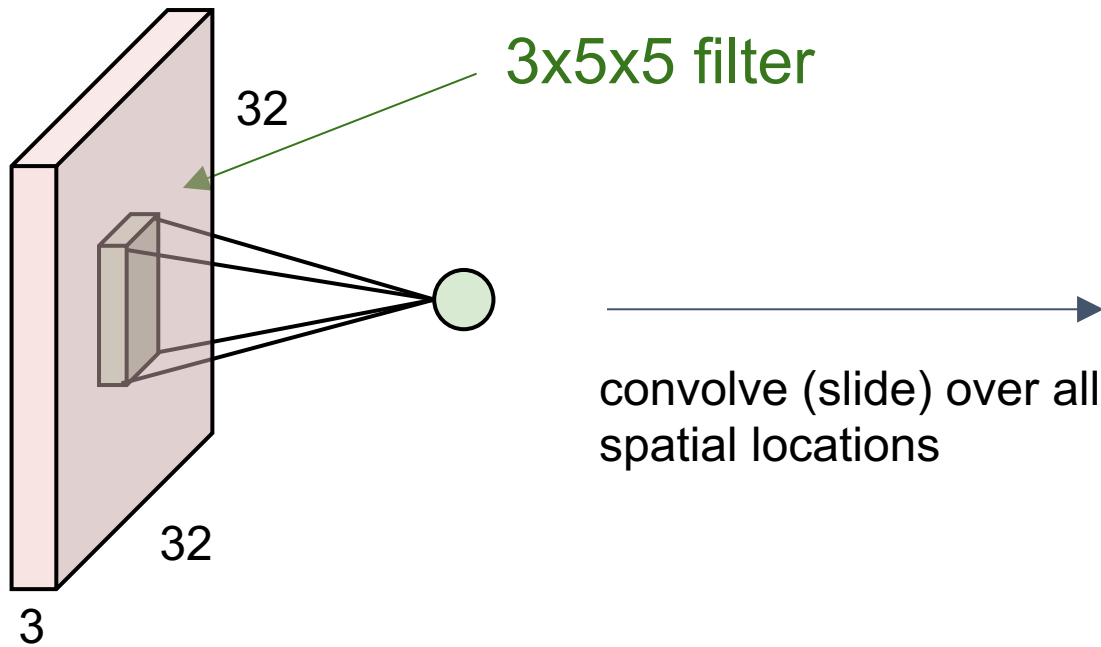


The activations are obtained by convolving the filters (weights) with the input activations. The output activation produced by a particular filter is known as a *activation map / feature map*

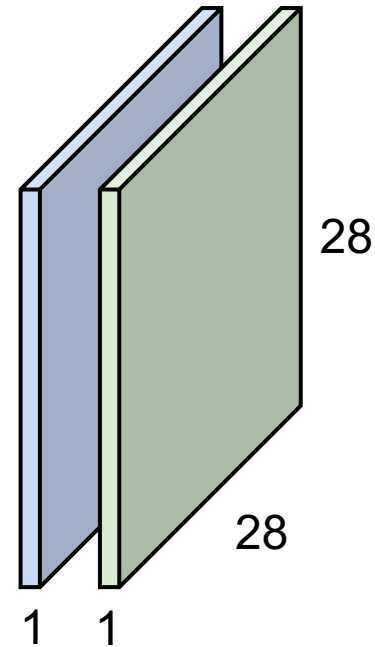
Convolution layer

Consider a second, **green** filter

3x32x32 image

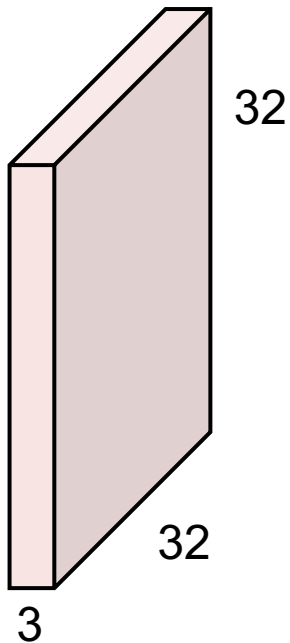


Two 1x28x28
activation/feature maps



Convolution layer

3x32x32 image



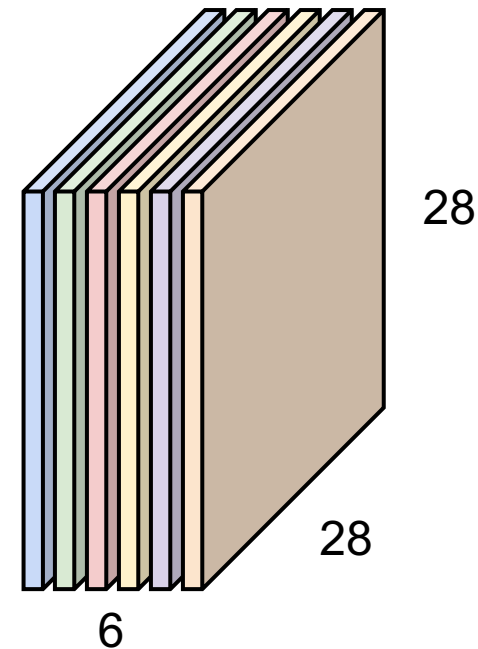
If we had **six** 3x5x5 filters,
we'll get **six** separate
activation maps:

convolution layer

6x3x5x5
filters



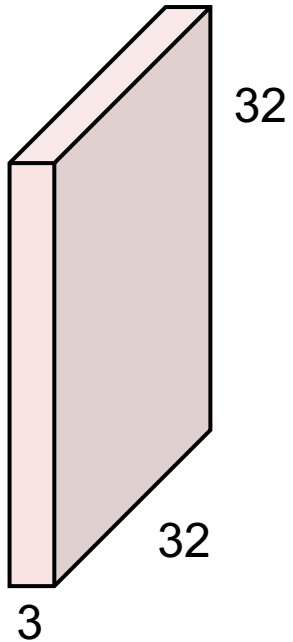
Six 1x28x28
activation/feature maps



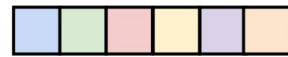
We stack these up to
get a “new image” of
size 6x28x28

Convolution layer

3x32x32 image



Also the 6-dim bias vector



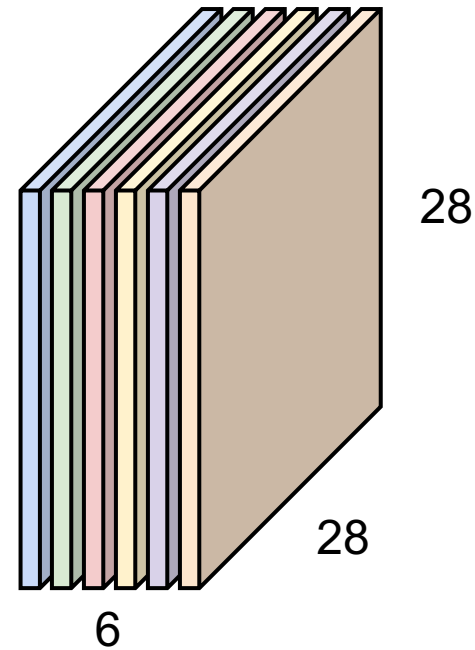
convolution layer

6x3x5x5
filters

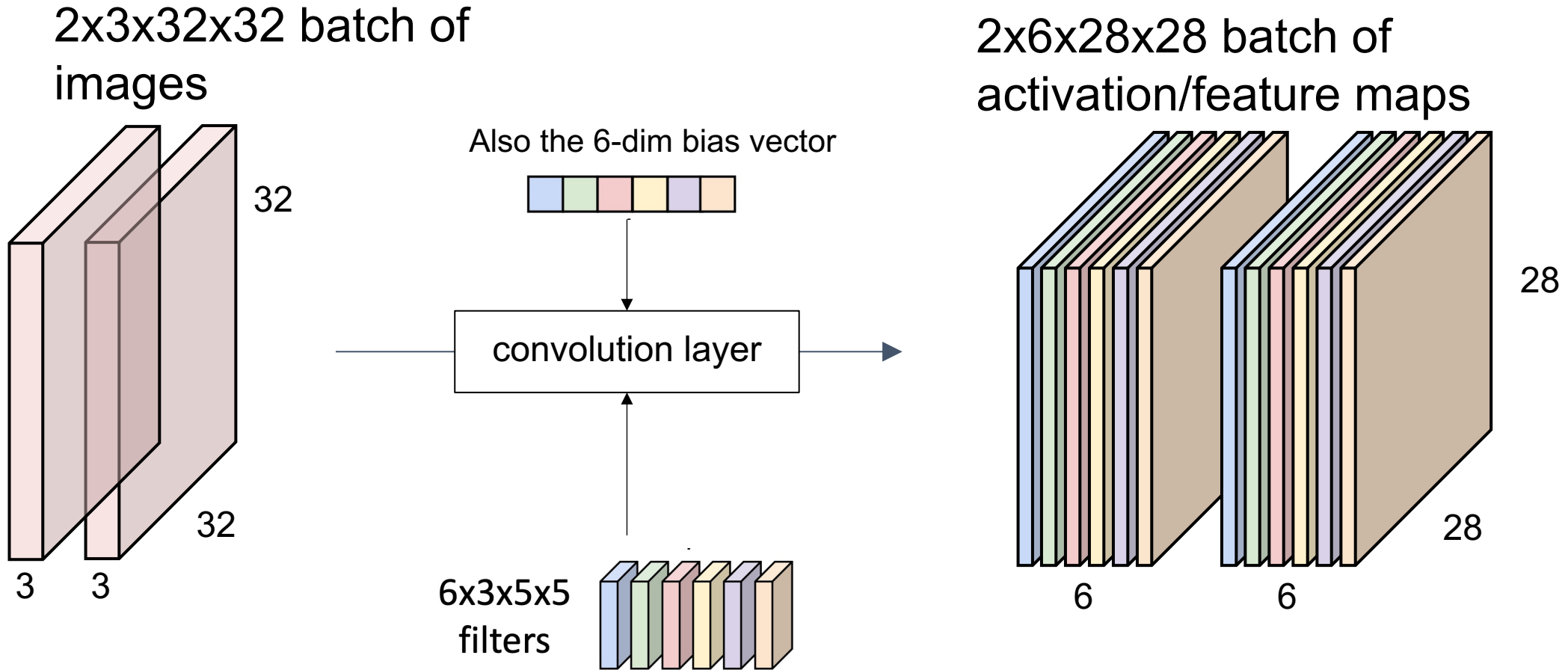


6x28x28

activation/feature maps

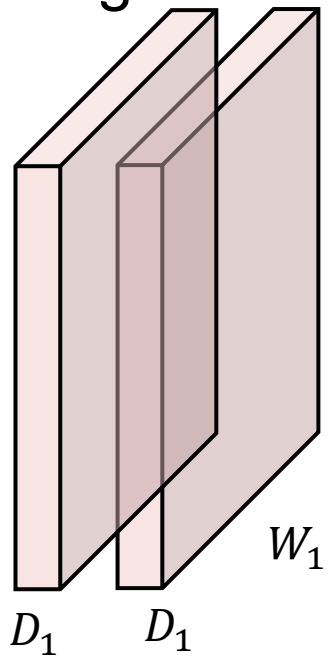


Convolution layer

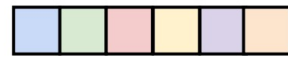


Convolution layer

$N \times D_1 \times H_1 \times W_1$ batch of images



Also the D_2 -dim bias vector

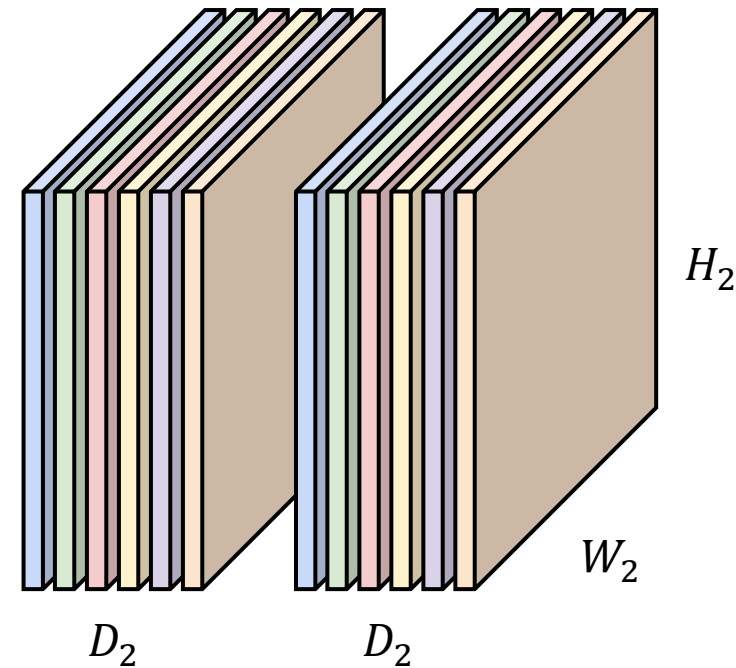


convolution layer

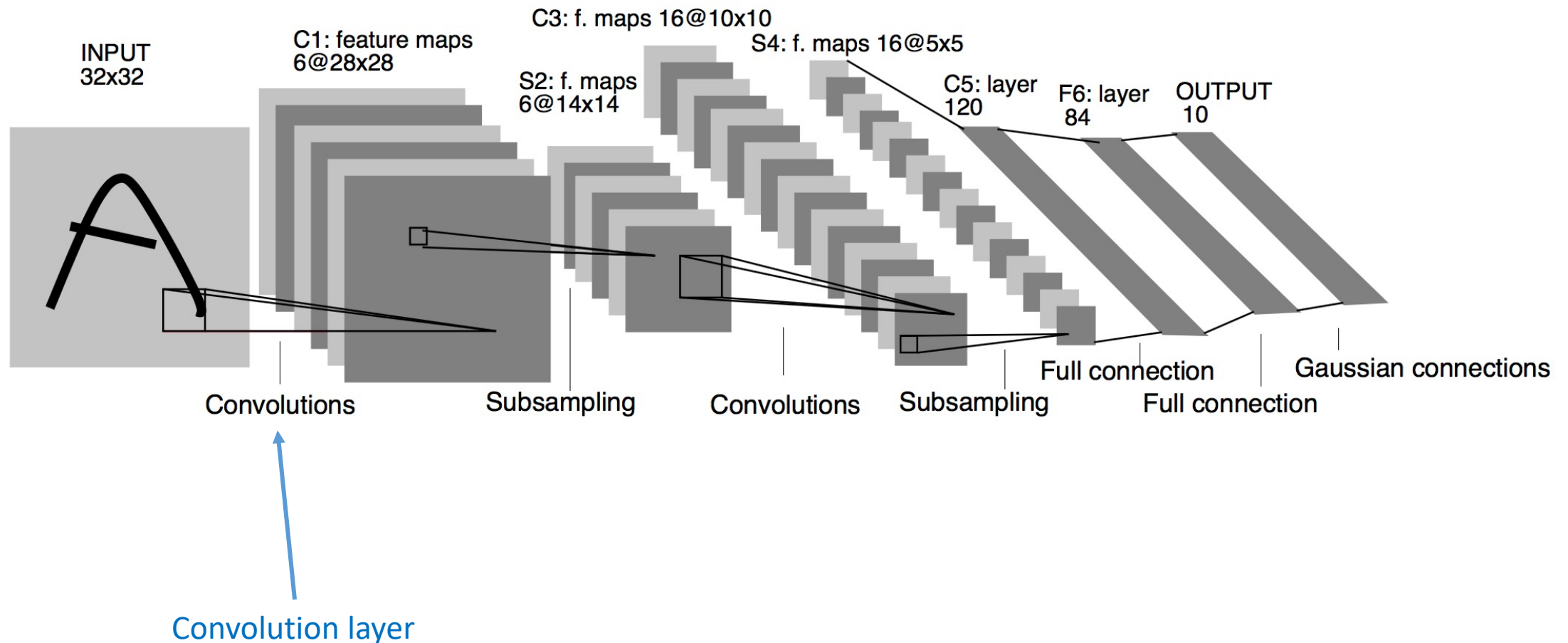
$D_2 \times D_1 \times F \times F$
filters



$N \times D_2 \times H_2 \times W_2$ batch of activation/feature maps



An example of convolutional network: LeNet 5

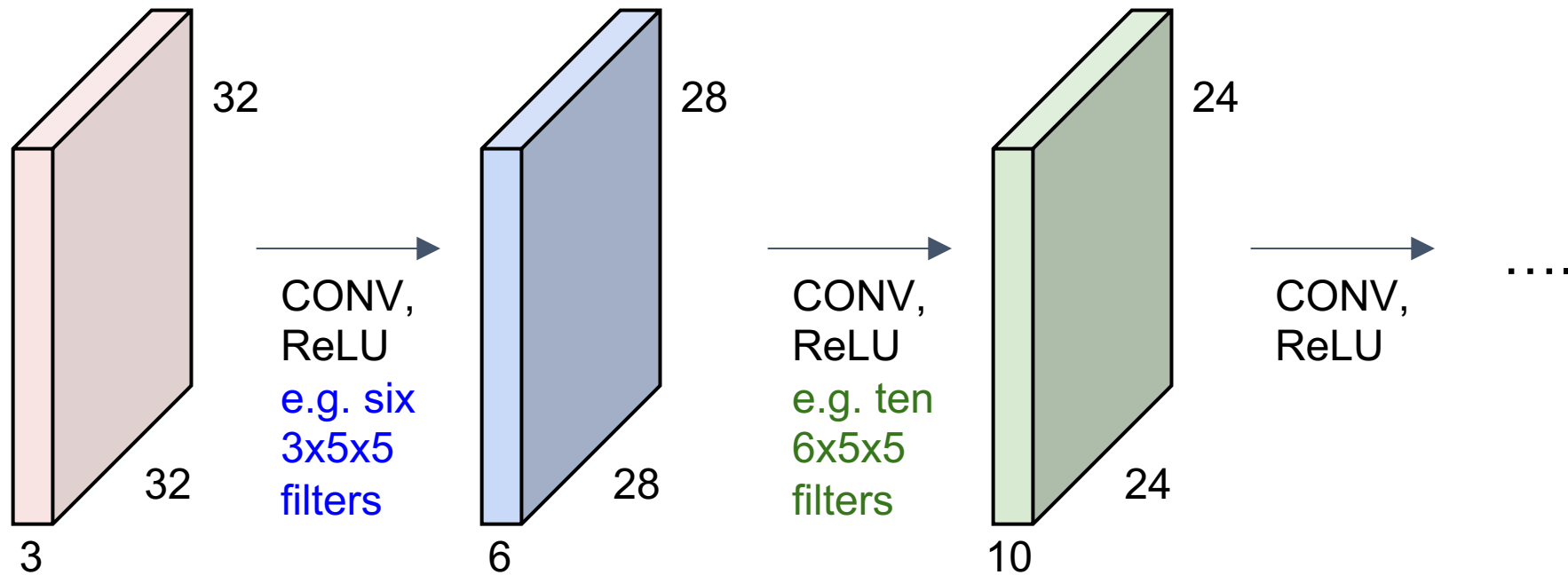


Back to this example, 6@28x28 means that we have 6 feature maps of size 28*28

You can imagine that the first convolutional layer uses 6 filters and each filter is of size 5 x 5 (how do we know that? We will discuss that later)

Convolution layer

CNN is a sequence of convolutional layers, interspersed with activation functions



Convolution layer

Consider a kernel $\mathbf{w} = \{w(l, m)\}$, which has a size of $L \times M, L = 2a + 1, M = 2b + 1$

Synaptic input at location $p = (i, j)$ of the first hidden layer due to a kernel is given by

$$u(i, j) = \sum_{l=-a}^a \sum_{m=-b}^b x(i + l, j + m)w(l, m) + b$$

For instance, given $L = 3, M = 3$

$$u(i, j) = x(i - 1, j - 1)w(-1, -1) + x(i - 1, j)w(-1, 0) + \dots \\ + x(i, j)w(0, 0) + x(i + 1, j + 1)w(1, 1) + b$$

Convolution layer

The output of the neuron at (i, j) of the convolution layer

$$y(i, j) = f(u(i, j))$$

where f is an activation function. For deep CNN, we typically use ReLU, $f(x) = \max(0, x)$.

Note that one weight tensor $\mathbf{w}_k = \{w_k(l, m)\}$ or kernel (filter) creates one feature map:

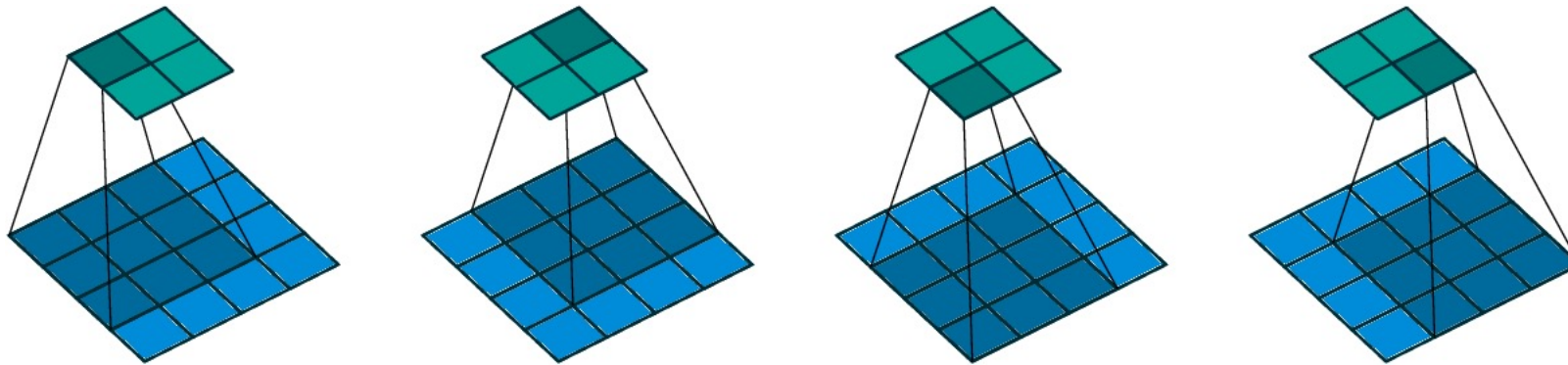
$$\mathbf{y}_k = \{y_k(i, j)\}$$

If there are K weight vectors $(\mathbf{w}_k)_{k=1}^K$, the convolutional layer is formed by K feature maps

$$\mathbf{y} = (\mathbf{y}_k)_{k=1}^K$$

Convolution layer

Convolution by doing a sliding window



As a guiding example, let us consider the convolution of single-channel tensors $\mathbf{x} \in \mathbb{R}^{4 \times 4}$ and $\mathbf{w} \in \mathbb{R}^{3 \times 3}$:

$$\mathbf{w} \star \mathbf{x} = \begin{pmatrix} 1 & 4 & 1 \\ 1 & 4 & 3 \\ 3 & 3 & 1 \end{pmatrix} \begin{pmatrix} 4 & 5 & 8 & 7 \\ 1 & 8 & 8 & 8 \\ 3 & 6 & 6 & 4 \\ 6 & 5 & 7 & 8 \end{pmatrix} = \begin{pmatrix} 122 & 148 \\ 126 & 134 \end{pmatrix}$$

Convolution layer

$$\mathbf{w} \star \mathbf{x} = \begin{pmatrix} 1 & 4 & 1 \\ 1 & 4 & 3 \\ 3 & 3 & 1 \end{pmatrix} \begin{pmatrix} 4 & 5 & 8 & 7 \\ 1 & 8 & 8 & 8 \\ 3 & 6 & 6 & 4 \\ 6 & 5 & 7 & 8 \end{pmatrix} = \begin{pmatrix} 122 & 148 \\ 126 & 134 \end{pmatrix}$$

$$u(i, j) = \sum_{l=-a}^a \sum_{m=-b}^b x(i+l, j+m)w(l, m) + b$$

$$u(1,1) = 4 \times 1 + 5 \times 4 + 8 \times 1 + 1 \times 1 + 8 \times 4 + 8 \times 3 + 3 \times 3 + 6 \times 3 + 6 \times 1 = 122$$

Convolution layer

$$\mathbf{w} \star \mathbf{x} = \begin{pmatrix} 1 & 4 & 1 \\ 1 & 4 & 3 \\ 3 & 3 & 1 \end{pmatrix} \begin{pmatrix} 4 & 5 & 8 & 7 \\ 1 & 8 & 8 & 8 \\ 3 & 6 & 6 & 4 \\ 6 & 5 & 7 & 8 \end{pmatrix} = \begin{pmatrix} 122 & 148 \\ 126 & 134 \end{pmatrix}$$

$$u(i, j) = \sum_{l=-a}^a \sum_{m=-b}^b x(i+l, j+m)w(l, m) + b$$

$$u(1,1) = 4 \times 1 + 5 \times 4 + 8 \times 1 + 1 \times 1 + 8 \times 4 + 8 \times 3 + 3 \times 3 + 6 \times 3 + 6 \times 1 = 122$$

$$u(1,2) = 5 \times 1 + 8 \times 4 + 7 \times 1 + 8 \times 1 + 8 \times 4 + 8 \times 3 + 6 \times 3 + 6 \times 3 + 4 \times 1 = 148$$

Convolution layer

$$\mathbf{w} \star \mathbf{x} = \begin{pmatrix} 1 & 4 & 1 \\ 1 & 4 & 3 \\ 3 & 3 & 1 \end{pmatrix} \begin{pmatrix} 4 & 5 & 8 & 7 \\ \color{red}1 & \color{red}8 & \color{red}8 & 8 \\ \color{red}3 & \color{red}6 & \color{red}6 & 4 \\ \color{red}6 & \color{red}5 & \color{red}7 & 8 \end{pmatrix} = \begin{pmatrix} 122 & 148 \\ 126 & 134 \end{pmatrix}$$

$$u(i, j) = \sum_{l=-a}^a \sum_{m=-b}^b x(i+l, j+m)w(l, m) + b$$

$$u(1,1) = 4 \times 1 + 5 \times 4 + 8 \times 1 + 1 \times 1 + 8 \times 4 + 8 \times 3 + 3 \times 3 + 6 \times 3 + 6 \times 1 = 122$$

$$u(1,2) = 5 \times 1 + 8 \times 4 + 7 \times 1 + 8 \times 1 + 8 \times 4 + 8 \times 3 + 6 \times 3 + 6 \times 3 + 4 \times 1 = 148$$

$$u(2,1) = \color{red}1 \times 1 + \color{red}8 \times 4 + \color{red}8 \times 1 + \color{red}3 \times 1 + \color{red}6 \times 4 + \color{red}6 \times 3 + \color{red}6 \times 3 + \color{red}5 \times 3 + \color{red}7 \times 1 = 126$$

Convolution layer

$$\mathbf{w} \star \mathbf{x} = \begin{pmatrix} 1 & 4 & 1 \\ 1 & 4 & 3 \\ 3 & 3 & 1 \end{pmatrix} \begin{pmatrix} 4 & 5 & 8 & 7 \\ 1 & 8 & 8 & 8 \\ 3 & 6 & 6 & 4 \\ 6 & 5 & 7 & 8 \end{pmatrix} = \begin{pmatrix} 122 & 148 \\ 126 & 134 \end{pmatrix}$$

$$u(i, j) = \sum_{l=-a}^a \sum_{m=-b}^b x(i+l, j+m)w(l, m) + b$$

$$u(1,1) = 4 \times 1 + 5 \times 4 + 8 \times 1 + 1 \times 1 + 8 \times 4 + 8 \times 3 + 3 \times 3 + 6 \times 3 + 6 \times 1 = 122$$

$$u(1,2) = 5 \times 1 + 8 \times 4 + 7 \times 1 + 8 \times 1 + 8 \times 4 + 8 \times 3 + 6 \times 3 + 6 \times 3 + 4 \times 1 = 148$$

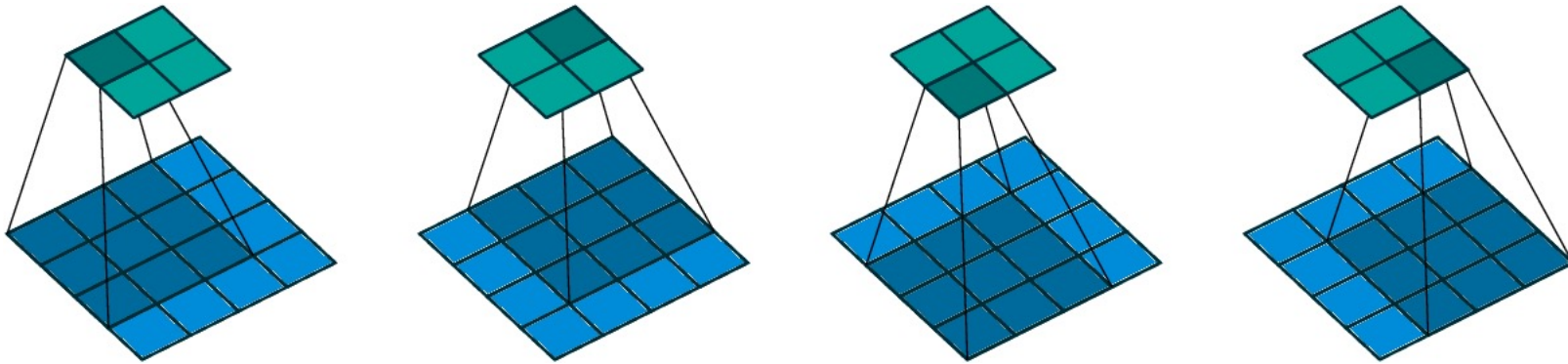
$$u(2,1) = 1 \times 1 + 8 \times 4 + 8 \times 1 + 3 \times 1 + 6 \times 4 + 6 \times 3 + 6 \times 3 + 5 \times 3 + 7 \times 1 = 126$$

$$u(2,2) = 8 \times 1 + 8 \times 4 + 8 \times 1 + 6 \times 1 + 6 \times 4 + 4 \times 3 + 5 \times 3 + 7 \times 3 + 8 \times 1 = 134$$

Convolution layer

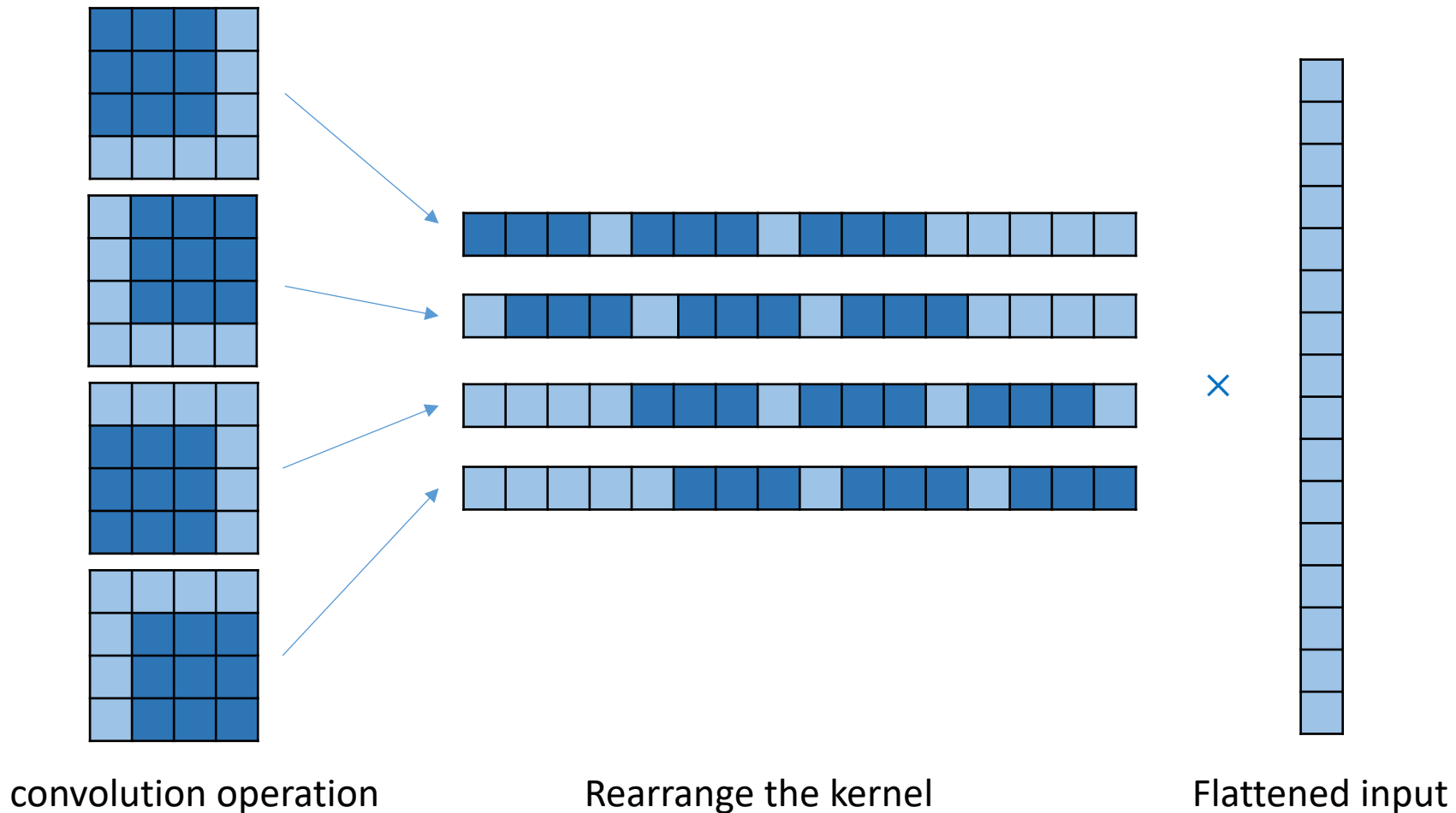
Convolution as matrix multiplication

- Convolution operation essentially performs dot products between the filters and local regions of the input (see the image below).
- Common practice is to take advantage of this fact and formulate the forward pass of a convolutional layer as one big matrix multiply



Convolution layer

The convolution operation can be equivalently re-expressed as a single matrix multiplication:



Convolution layer

Convolution as matrix multiplication

Using the same guiding example, let us consider the convolution of single-channel tensors $\mathbf{x} \in \mathbb{R}^{4 \times 4}$ and $\mathbf{w} \in \mathbb{R}^{3 \times 3}$:

$$\mathbf{w} \star \mathbf{x} = \begin{pmatrix} 1 & 4 & 1 \\ 1 & 4 & 3 \\ 3 & 3 & 1 \end{pmatrix} \begin{pmatrix} 4 & 5 & 8 & 7 \\ 1 & 8 & 8 & 8 \\ 3 & 6 & 6 & 4 \\ 6 & 5 & 7 & 8 \end{pmatrix} = \begin{pmatrix} 122 & 148 \\ 126 & 134 \end{pmatrix}$$

Convolution layer

- The convolutional kernel $\mathbf{w}$ is rearranged as a **sparse Toeplitz circulant matrix**, called the convolution matrix:

$$\mathbf{W} = \begin{pmatrix} 1 & 4 & 1 & 0 & 1 & 4 & 3 & 0 & 3 & 3 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 4 & 1 & 0 & 1 & 4 & 3 & 0 & 3 & 3 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 4 & 1 & 0 & 1 & 4 & 3 & 0 & 3 & 3 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 4 & 1 & 0 & 1 & 4 & 3 & 0 & 3 & 3 & 1 \end{pmatrix}$$

- The input $\mathbf{x}$ is flattened row by row, from top to bottom:

$$v(\mathbf{x}) = (4 \quad 5 \quad 8 \quad 7 \quad 1 \quad 8 \quad 8 \quad 8 \quad 3 \quad 6 \quad 6 \quad 4 \quad 6 \quad 5 \quad 7 \quad 8)^T$$

Then,

$$\mathbf{W}v(\mathbf{x}) = (122 \quad 148 \quad 126 \quad 134)^T$$

which we can reshape to a 2×2 matrix to obtain $\mathbf{w} \star \mathbf{x}$.

Convolution layer – visualization

3 channels



one filter =>
one activation map

example 3x5x5 filters
(32 filters in total)

Activations:

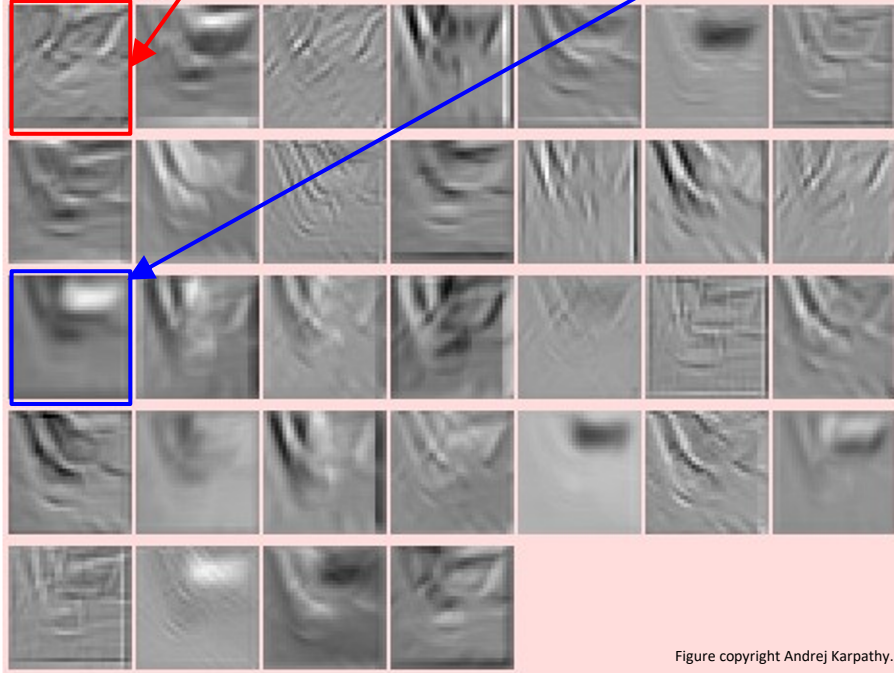
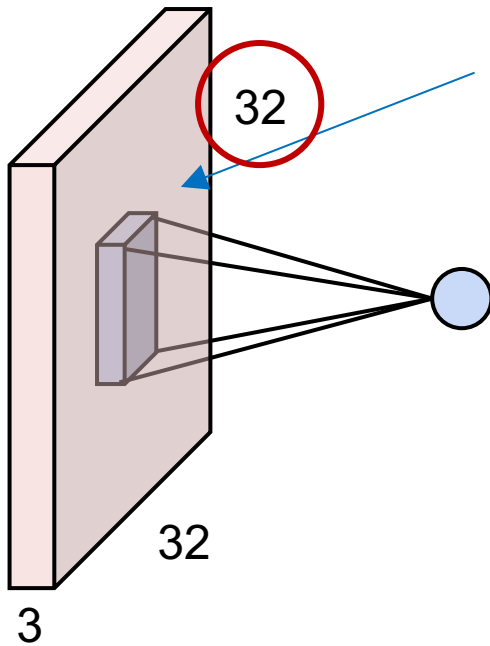


Figure copyright Andrej Karpathy.

Convolution layer – spatial dimensions

3x32x32 image

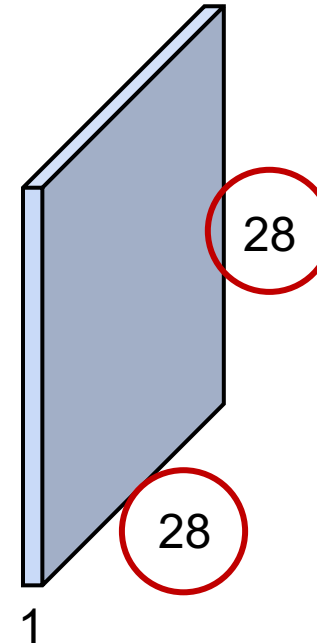


3x5x5 filter

convolve (slide) over all spatial locations

1x28x28

activation/feature map

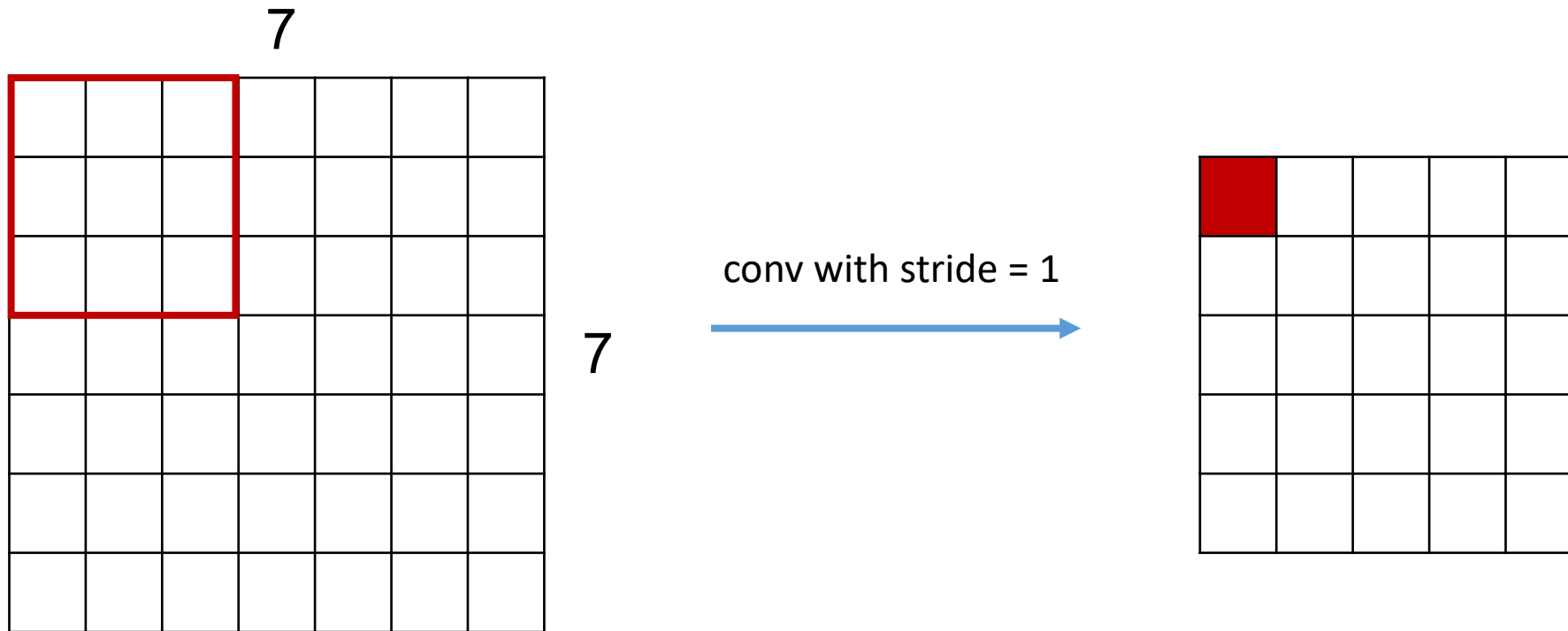


Why does the feature map has a size of 28x28?

Convolution layer – spatial dimensions

7x7 input (spatially), assume 3x3 filter

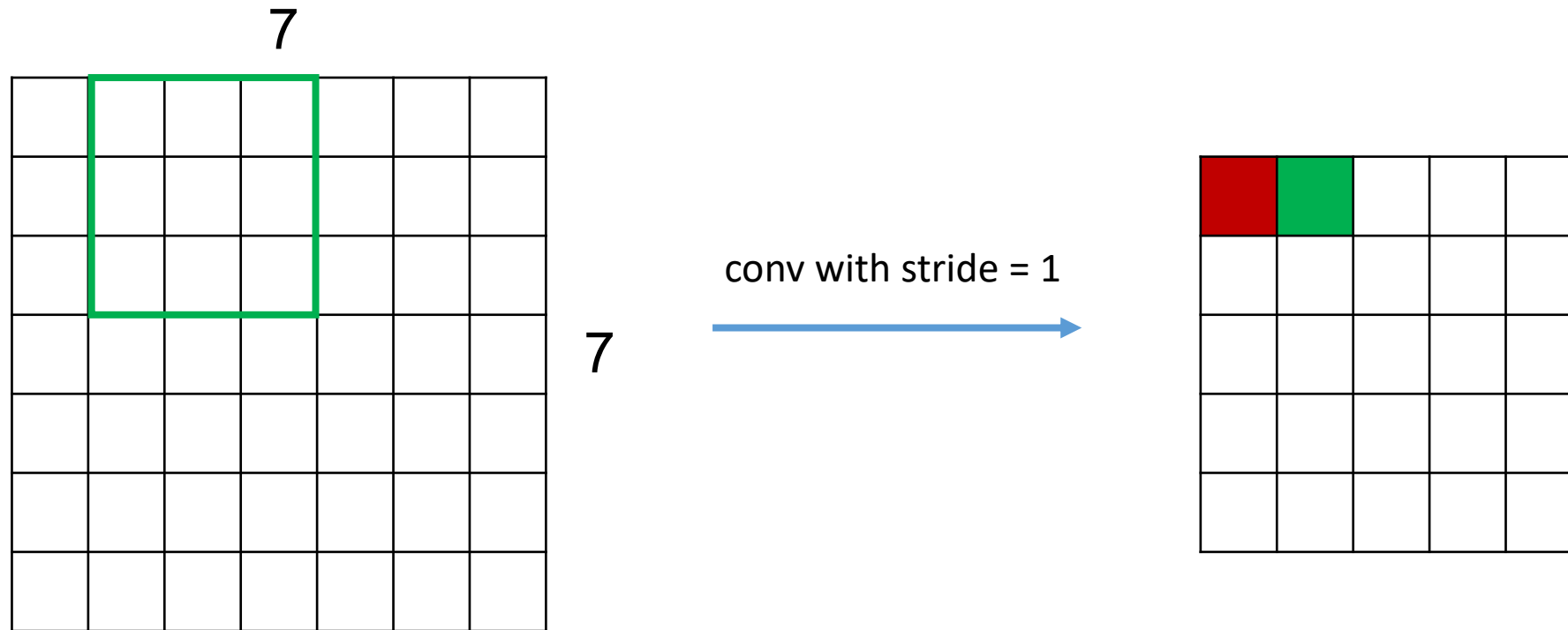
Stride is the factor with which the output is subsampled. That is, *distance between adjacent centers of the kernel*.



Convolution layer – spatial dimensions

7x7 input (spatially), assume 3x3 filter

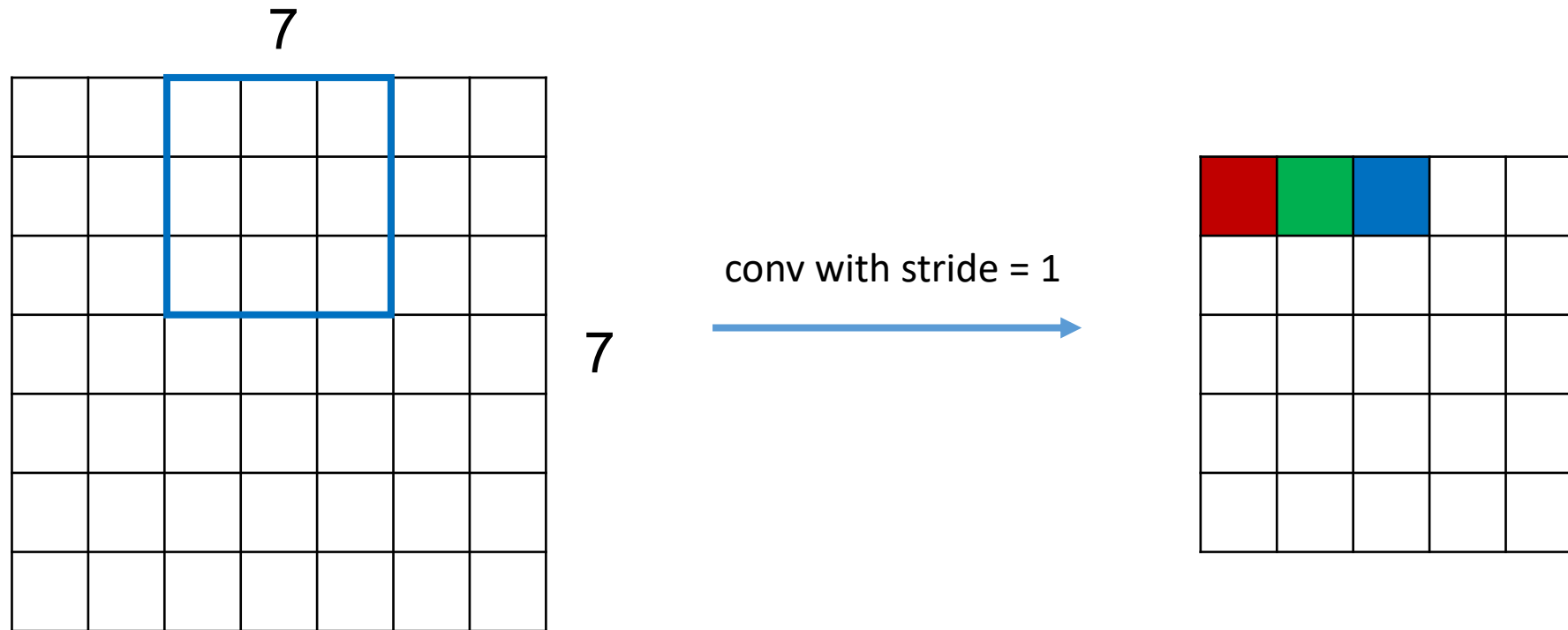
Stride is the factor with which the output is subsampled. That is, *distance between adjacent centers of the kernel*.



Convolution layer – spatial dimensions

7x7 input (spatially), assume 3x3 filter

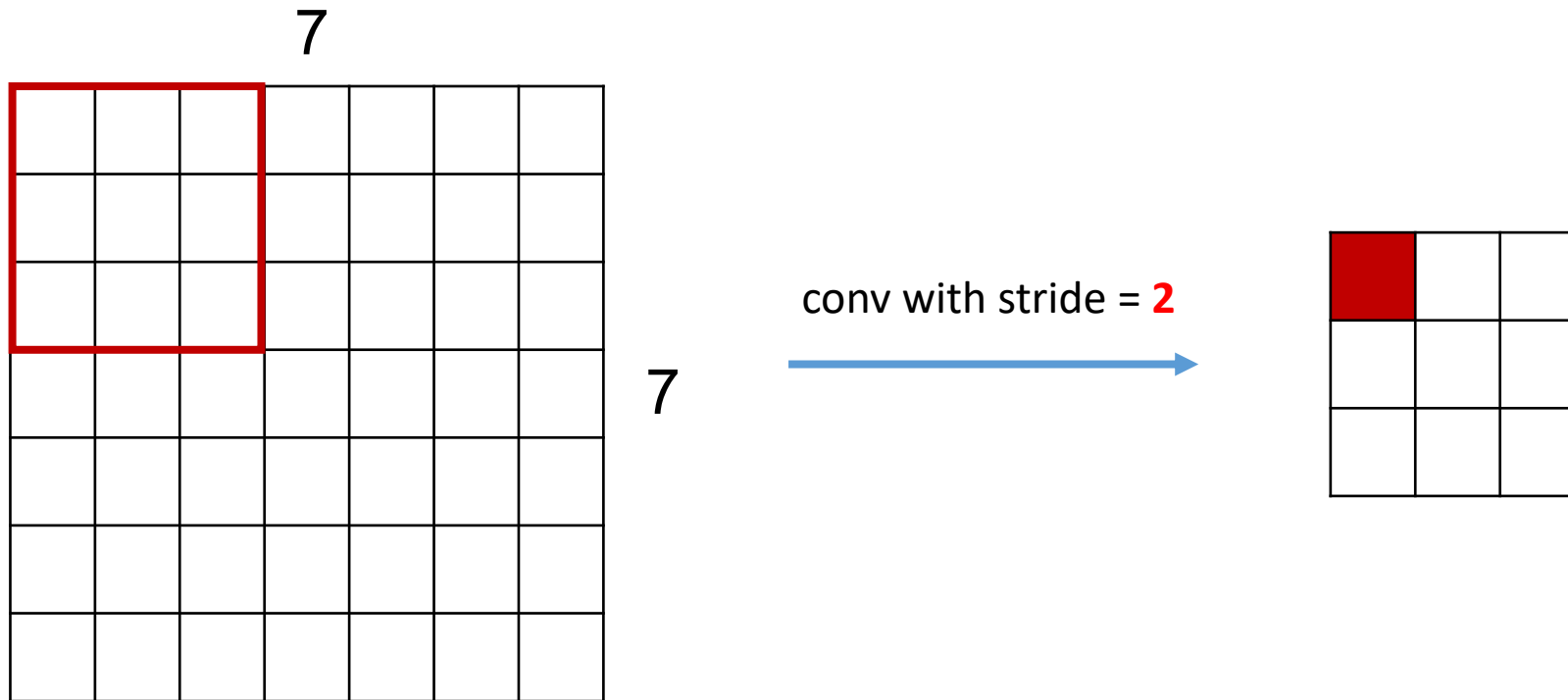
Stride is the factor with which the output is subsampled. That is, *distance between adjacent centers of the kernel*.



Convolution layer – spatial dimensions

7x7 input (spatially), assume 3x3 filter

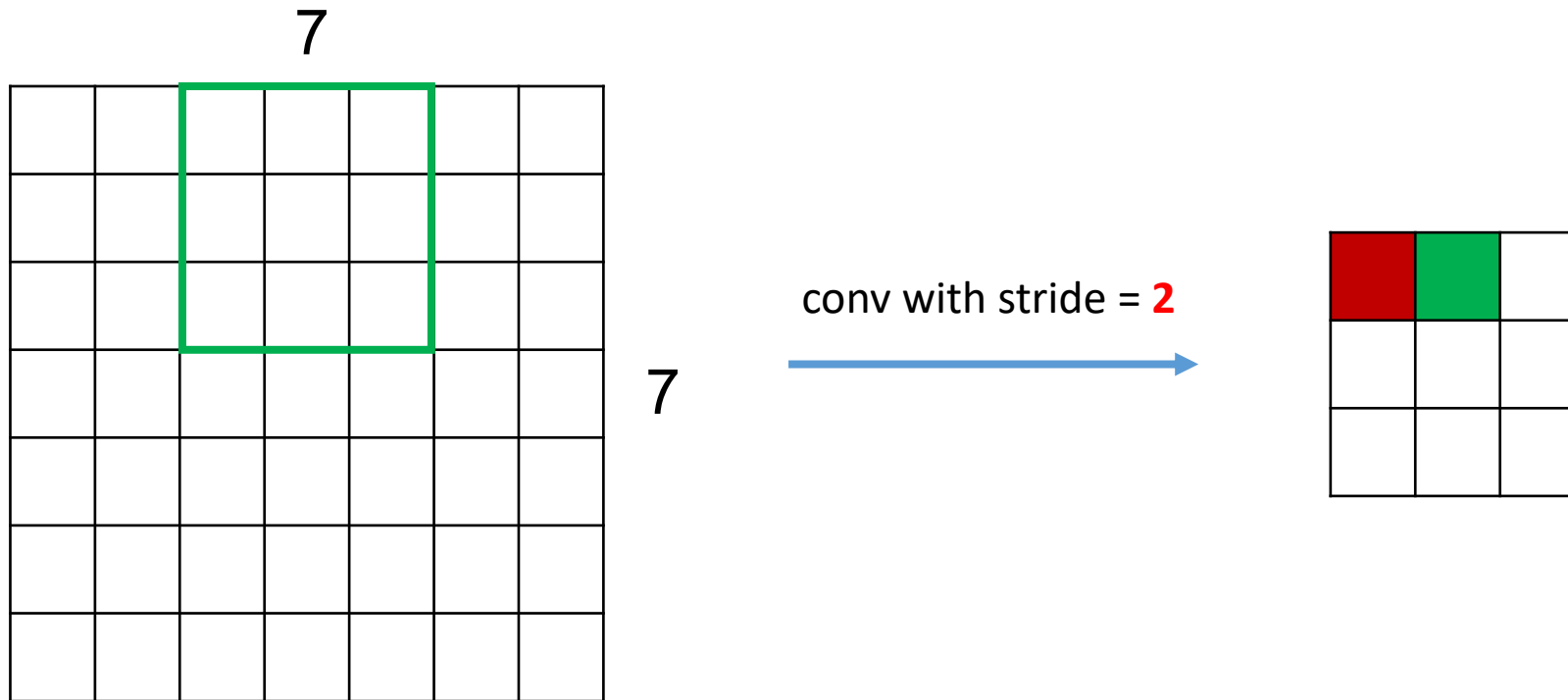
Stride is the factor with which the output is subsampled. That is, *distance between adjacent centers of the kernel*.



Convolution layer – spatial dimensions

7x7 input (spatially), assume 3x3 filter

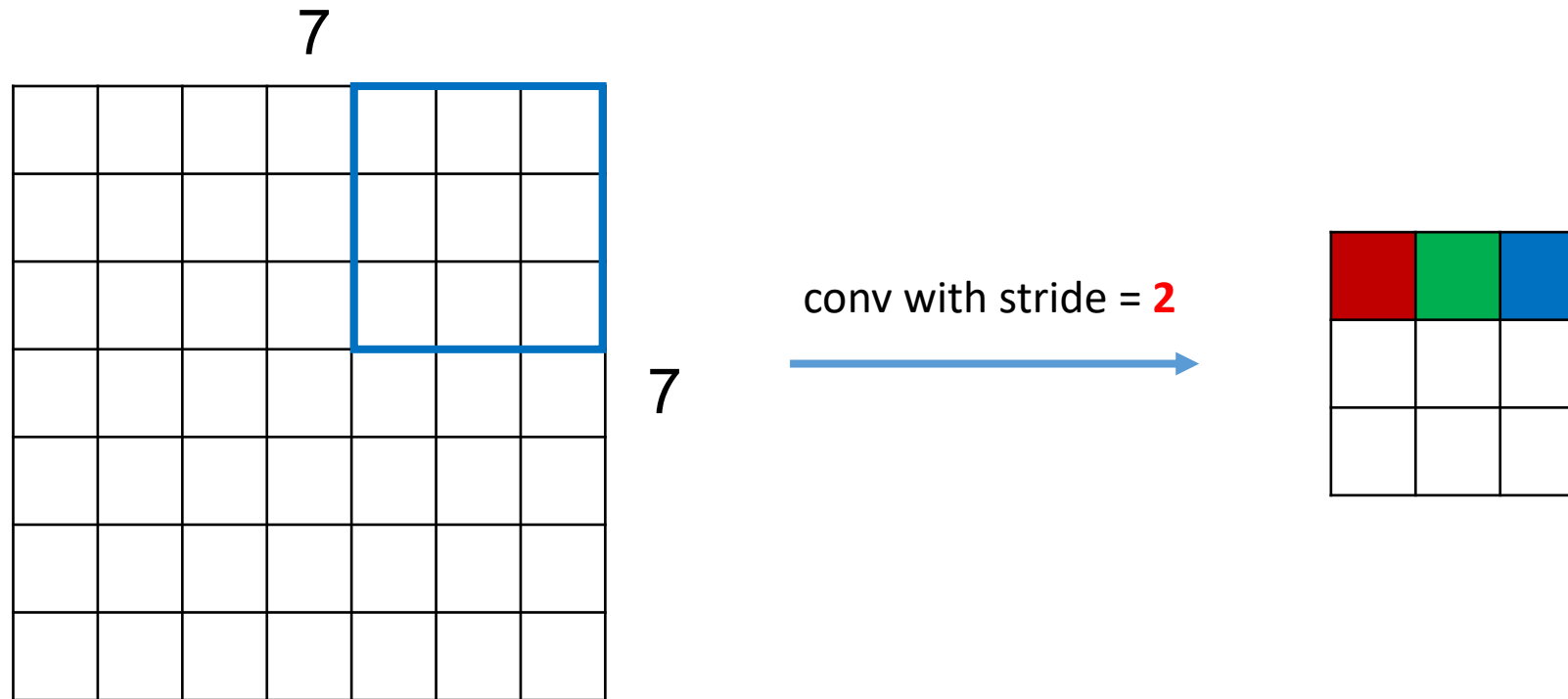
Stride is the factor with which the output is subsampled. That is, *distance between adjacent centers of the kernel*.



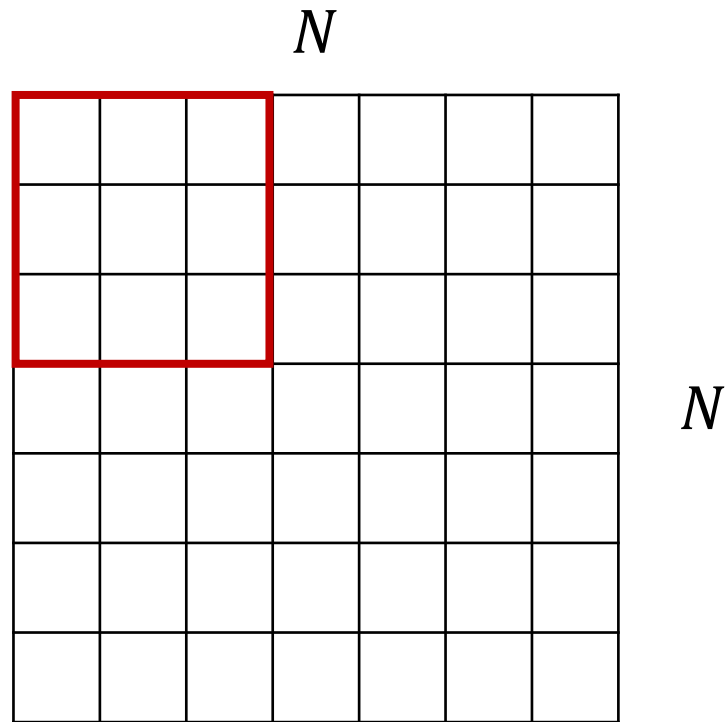
Convolution layer – spatial dimensions

7x7 input (spatially), assume 3x3 filter

Stride is the factor with which the output is subsampled. That is, *distance between adjacent centers of the kernel*.



Convolution layer – spatial dimensions



$N \times N$ input (spatially), assume $F \times F$ filter, and S stride

$$\text{Output size} = \frac{N-F}{S} + 1$$

e.g.

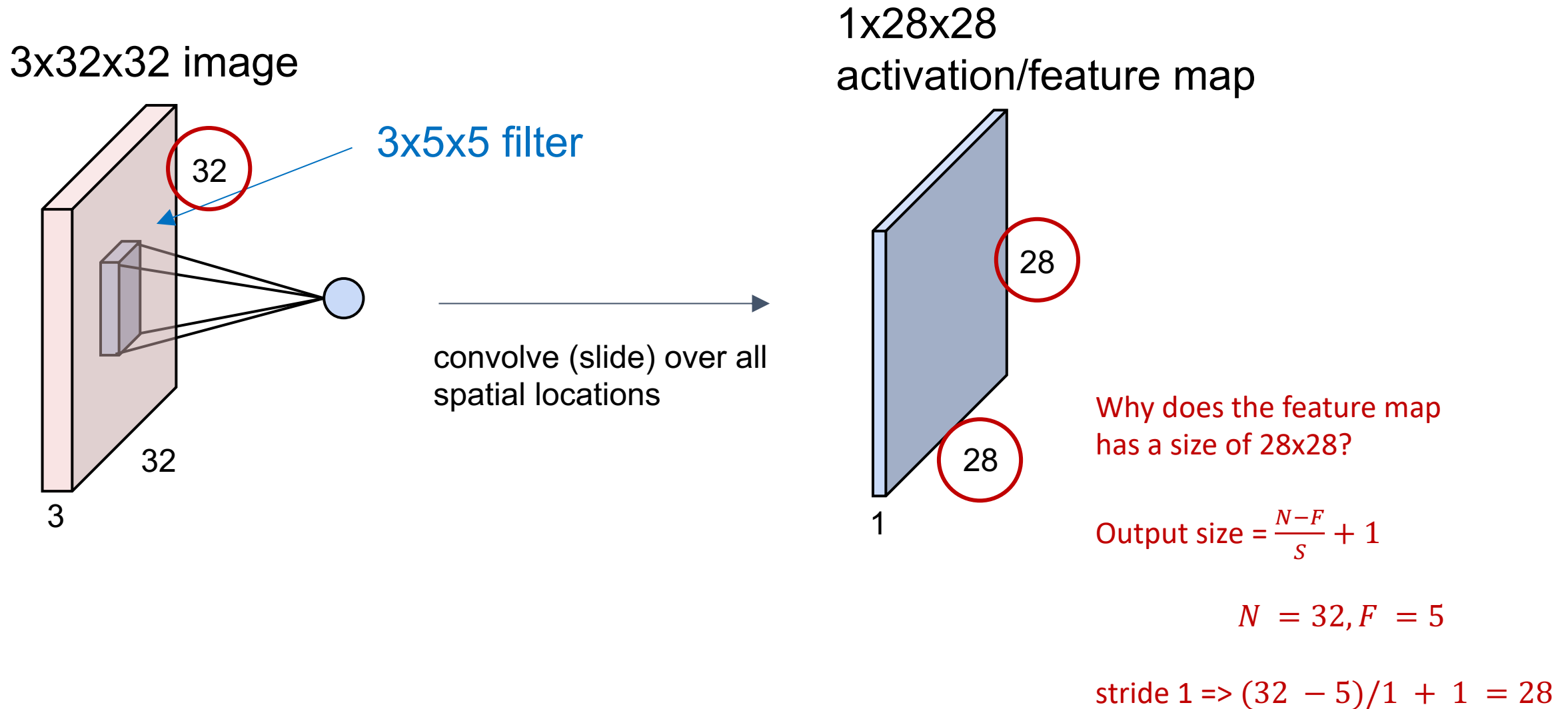
$$N = 7, F = 3$$

$$\text{stride } 1 \Rightarrow (7 - 3)/1 + 1 = 5$$

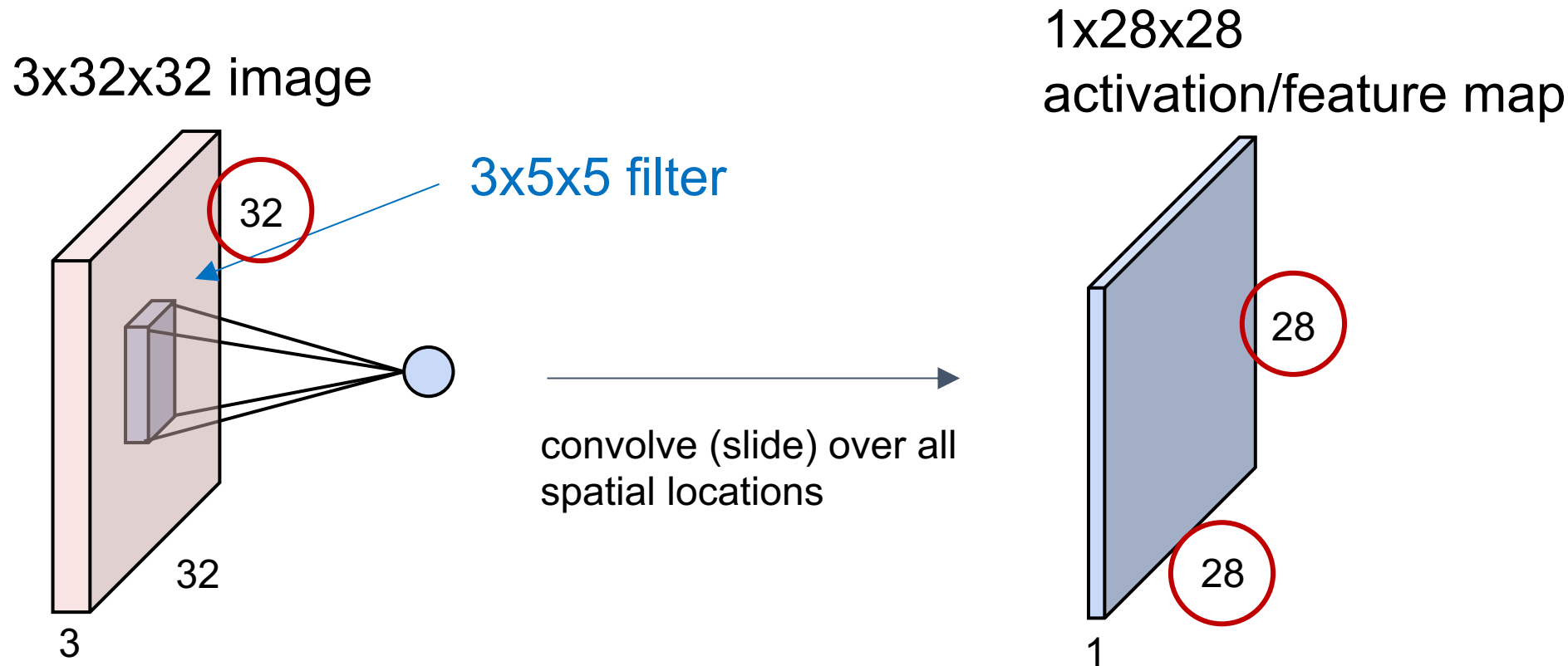
$$\text{stride } 2 \Rightarrow (7 - 3)/2 + 1 = 3$$

$$\text{stride } 3 \Rightarrow (7 - 3)/3 + 1 = 2.33 \quad :\backslash$$

Convolution layer – spatial dimensions



Convolution layer – spatial dimensions



The valid feature map is smaller than the input after convolution.

Without zero-padding, the width of the representation shrinks by the $F - 1$ at each layer

To avoid shrinking the spatial extent of the network rapidly, small filters have to be used

Convolution layer – zero padding

By zero-padding in each layer, we prevent the representation from shrinking with depth. In practice, we zero pad the border. In **PyTorch**, by default, we pad top, bottom, left, right with zeros (this can be customized).

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output shape?

Convolution layer – zero padding

By zero-padding in each layer, we prevent the representation from shrinking with depth. In practice, we zero pad the border. In **PyTorch**, by default, we pad top, bottom, left, right with zeros (this can be customized).

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output shape?

Recall that without padding, output size = $\frac{N-F}{S} + 1$

With padding, output size = $\frac{N-F+2P}{S} + 1$

e.g.

$$N = 7, F = 3$$

$$\text{stride } 1 \Rightarrow (7 - 3 + 2(1))/1 + 1 = 7$$

Convolution layer – zero padding

By zero-padding in each layer, we prevent the representation from shrinking with depth. In practice, we zero pad the border. In **PyTorch**, by default, we pad top, bottom, left, right with zeros (this can be customized).

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output shape?

7x7 output!

In general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F - 1)/2$. (**will preserve size spatially**)

e.g. $F = 3$ => zero pad with 1

$F = 5$ => zero pad with 2

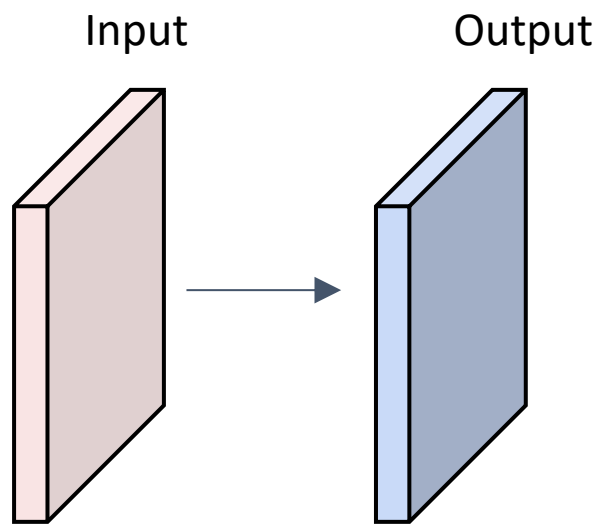
$F = 7$ => zero pad with 3

Example

Input volume: $3 \times 32 \times 32$

Ten $3 \times 5 \times 5$ filters with stride 1, pad 2

Output volume size: ?

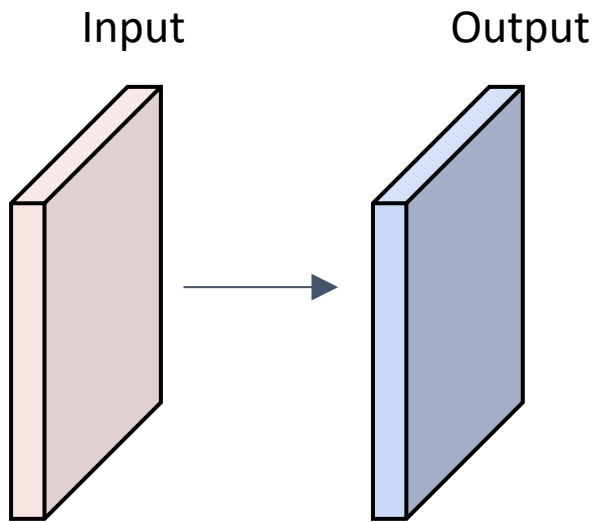


Example

Input volume: $3 \times 32 \times 32$

Ten $3 \times 5 \times 5$ filters with stride 1, pad 2

Output volume size: ?



Output size =

$$\frac{N - F + 2P}{S} + 1$$

$$\frac{32 - 5 + 2(2)}{1} + 1 = 32 \text{ spatially}$$

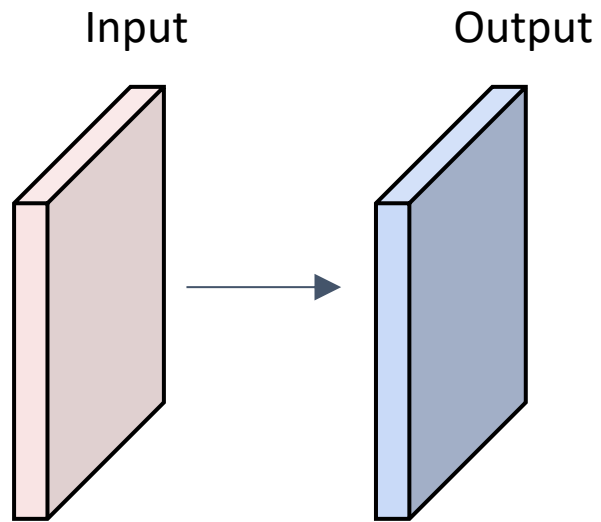
The output volume size is $10 \times 32 \times 32$

Example

Input volume: $3 \times 32 \times 32$

Ten $3 \times 5 \times 5$ filters with stride 1, pad 2

Number of parameters in this layer: ?

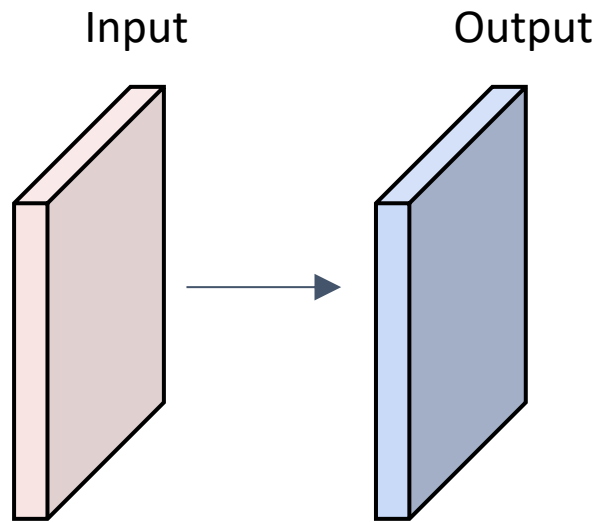


Example

Input volume: $3 \times 32 \times 32$

Ten $3 \times 5 \times 5$ filters with stride 1, pad 2

Number of parameters in this layer: ?



Each filter has $5 \times 5 \times 3 + 1 = 76$ params (+1 for bias)
 $\Rightarrow 76 \times 10 = 760$

Convolution layer – a numerical example

Input volume (+pad 1)
 $C \times I \times I = 3 \times 7 \times 7$

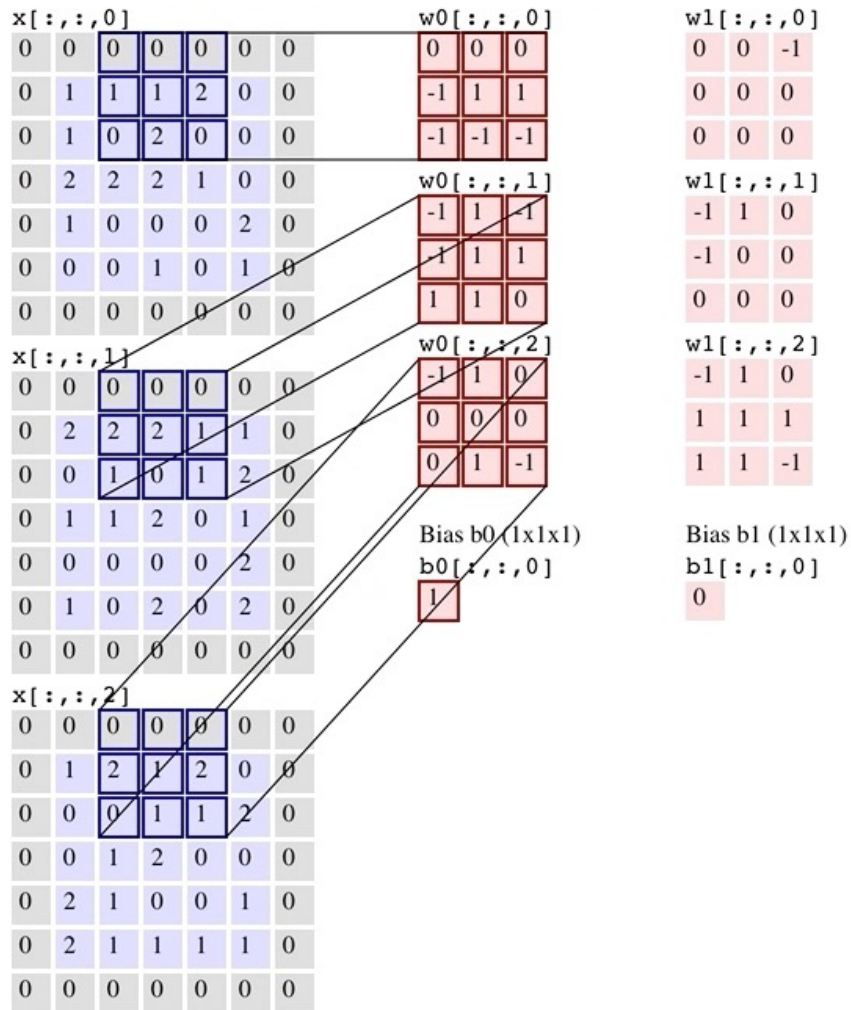
Filters W_0, W_1 volume
 $C \times F \times F = 3 \times 3 \times 3$

Output volume
 $C \times O \times O = 2 \times 3 \times 3$

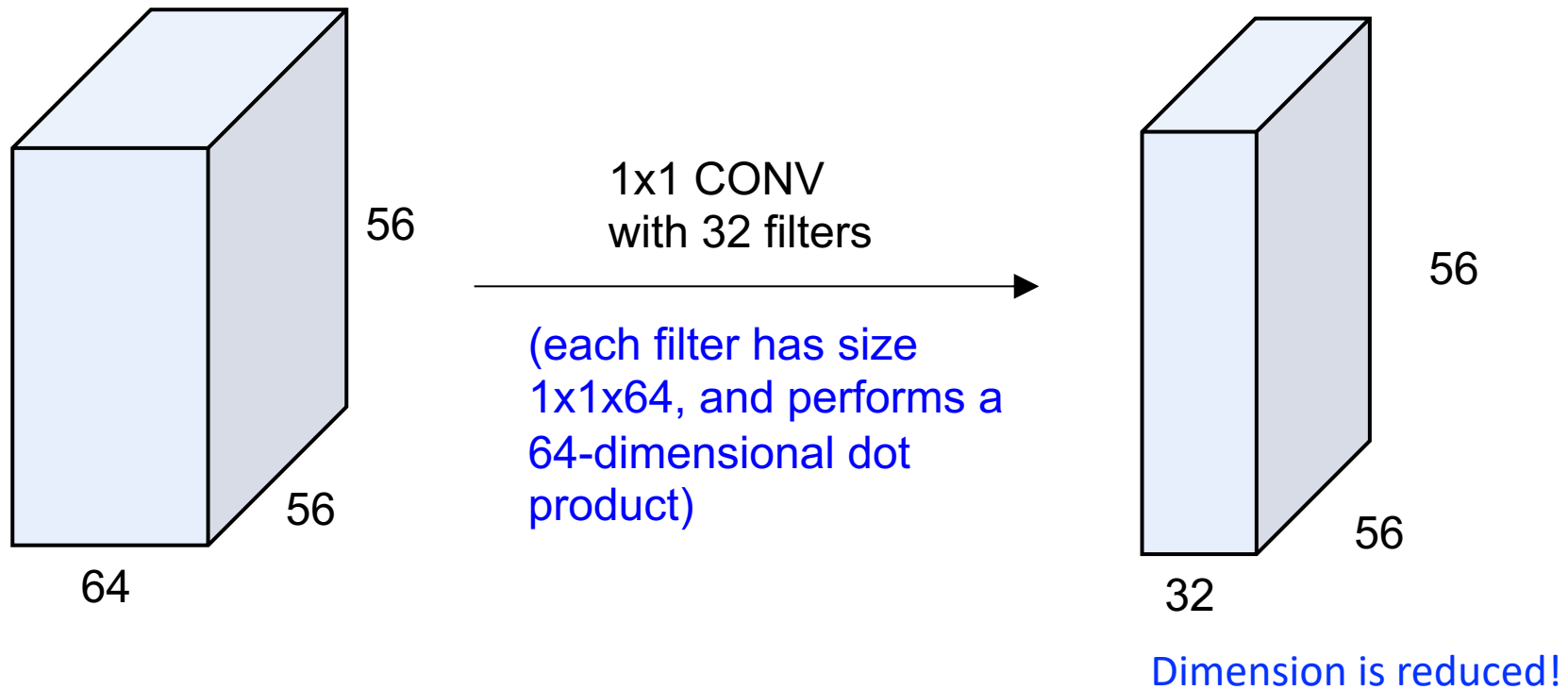
- $C = 3$ channel inputs

- Two filters of size $F \times F = 3 \times 3$, and they are applied with a stride of 2.

- Each element of the output activations (**green**) is computed by elementwise multiplying the highlighted input (**blue**) with the filter (**red**), summing it up, and then offsetting the result by the bias.



Can we have 1x1 filter?

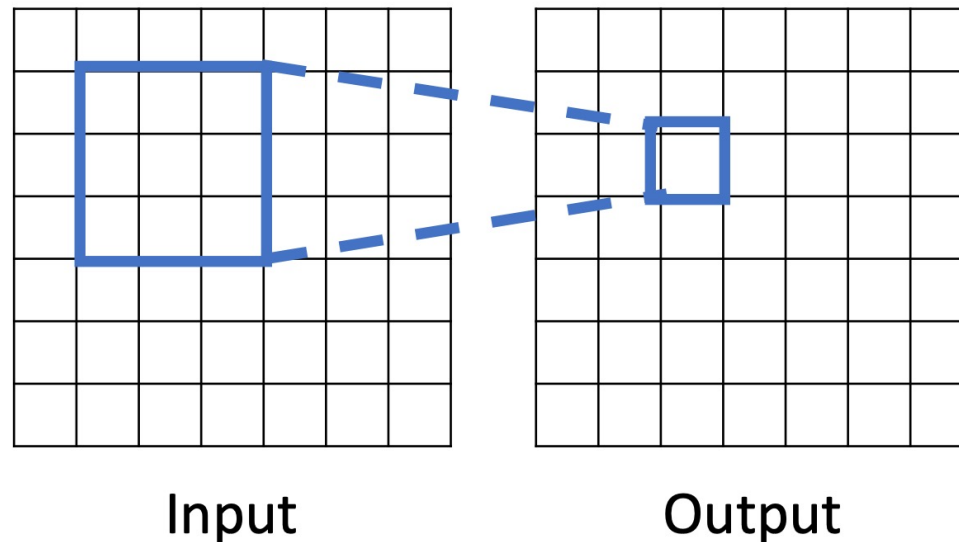


Receptive field

Sparse local connectivity - CNNs exploit spatially local correlations by enforcing **local connectivity** between neurons of adjacent layers.

Note that the **receptive fields of the neurons are limited because of local connectivity**.

For convolution with kernel size K , each element in the output depends on a $K \times K$ receptive field in the input



Receptive field

- Receptive field can be briefly defined as the region in the input space that a particular CNN's feature is looking at
- The receptive field of the units in the deeper layers of a convolutional network is **larger** than the receptive field of the units in the shallow layers
- This effect **increases** if the network include strided convolution or pooling
- Even though direct connections in a CNN are very sparse, units in the deeper layers can be indirectly connected to all or most of the input image

Receptive field

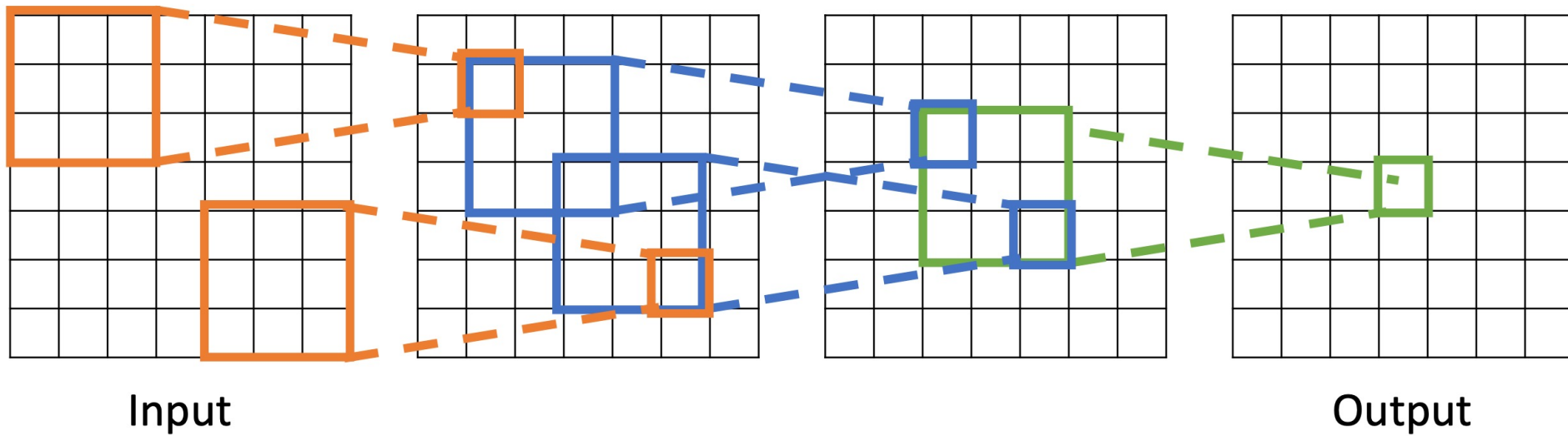
The receptive field at layer k is the area denoted $R_k \times R_k$ of the input that each pixel of the k -th activation map can 'see'.

By calling F_j the filter size of layer j and S_i the stride value of layer i and with the convention $S_0 = 1$, the receptive field at layer k can be computed with the formula:

$$R_k = 1 + \sum_{j=1}^k (F_j - 1) \prod_{i=0}^{j-1} S_i$$

Receptive field

Example



$$R_k = 1 + \sum_{j=1}^k (F_j - 1) \prod_{i=0}^{j-1} S_i$$

Filter size $F_1 = F_2 = F_3 = 3$

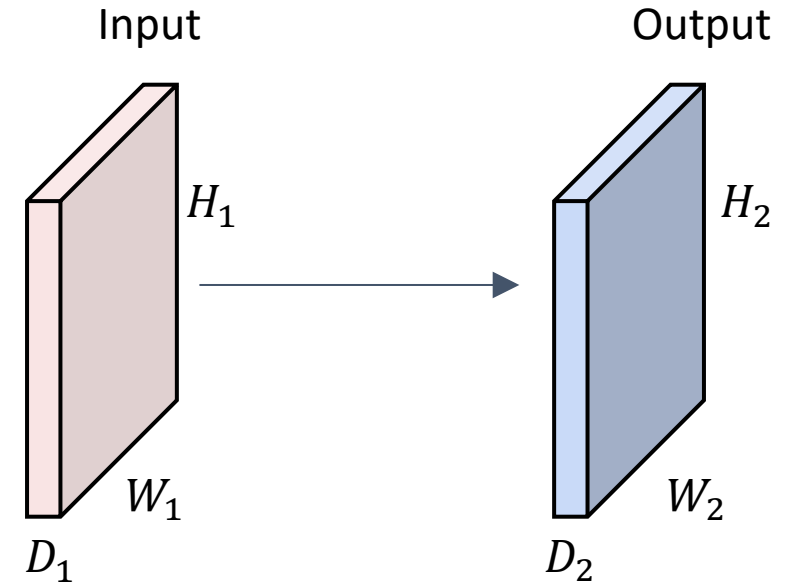
Stride $S_1 = S_2 = S_3 = 1$

$$R_3 = 1 + \sum_{j=1}^3 (F_j - 1) \prod_{i=0}^{j-1} S_i = 1 + (2 \times 1) + (2 \times 1) + (2 \times 1) = 7$$

Convolution layer - summary

A convolution layer

- Accepts a volume of size $D_1 \times H_1 \times W_1$
- Requires four hyperparameters
 - Number of filters K
 - Their spatial extent F
 - The stride S
 - The amount of zero padding P
- Produces a volume of size $D_2 \times H_2 \times W_2$, where
 - $W_2 = (W_1 - F + 2P)/S + 1$
 - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e., width and height are computed equally by symmetry)
 - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases
- In the output volume, the d -th depth slice (of size $H_2 \times W_2$) is the result of a valid convolution of the d -th filter over the input volume with a stride of S , and then offset by d -th bias



Convolutional layer in PyTorch

Conv2d

```
CLASS torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros')
```

[SOURCE]

Applies a 2D convolution over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C_{in}, H, W) and output $(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ can be precisely described as:

$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \star \text{input}(N_i, k)$$

where $\star$ is the valid 2D **cross-correlation** operator, N is a batch size, C denotes a number of channels, H is a height of input planes in pixels, and W is width in pixels.

- `stride` controls the stride for the cross-correlation, a single number or a tuple.
- `padding` controls the amount of implicit zero-paddings on both sides for `padding` number of points for each dimension.
- `dilation` controls the spacing between the kernel points; also known as the à trous algorithm. It is harder to describe, but this [link](#) has a nice visualization of what `dilation` does.
- `groups` controls the connections between inputs and outputs. `in_channels` and `out_channels` must both be divisible by `groups`. For example,
 - At `groups=1`, all inputs are convolved to all outputs.
 - At `groups=2`, the operation becomes equivalent to having two conv layers side by side, each seeing half the input channels, and producing half the output channels, and both subsequently concatenated.
 - At `groups= in_channels`, each input channel is convolved with its own set of filters, of size: $\left\lfloor \frac{\text{out_channels}}{\text{in_channels}} \right\rfloor$.

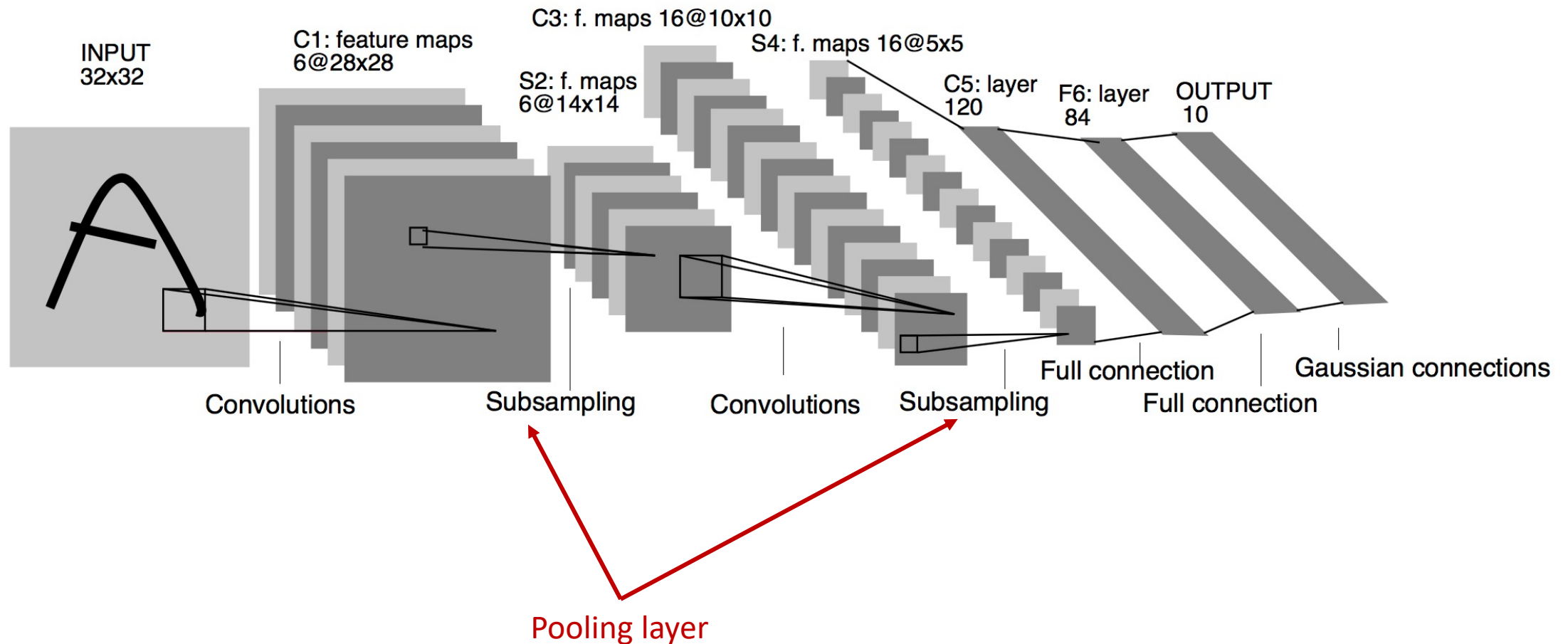
The parameters `kernel_size`, `stride`, `padding`, `dilation` can either be:

- a single `int` – in which case the same value is used for the height and width dimension
- a `tuple` of two ints – in which case, the first `int` is used for the height dimension, and the second `int` for the width dimension

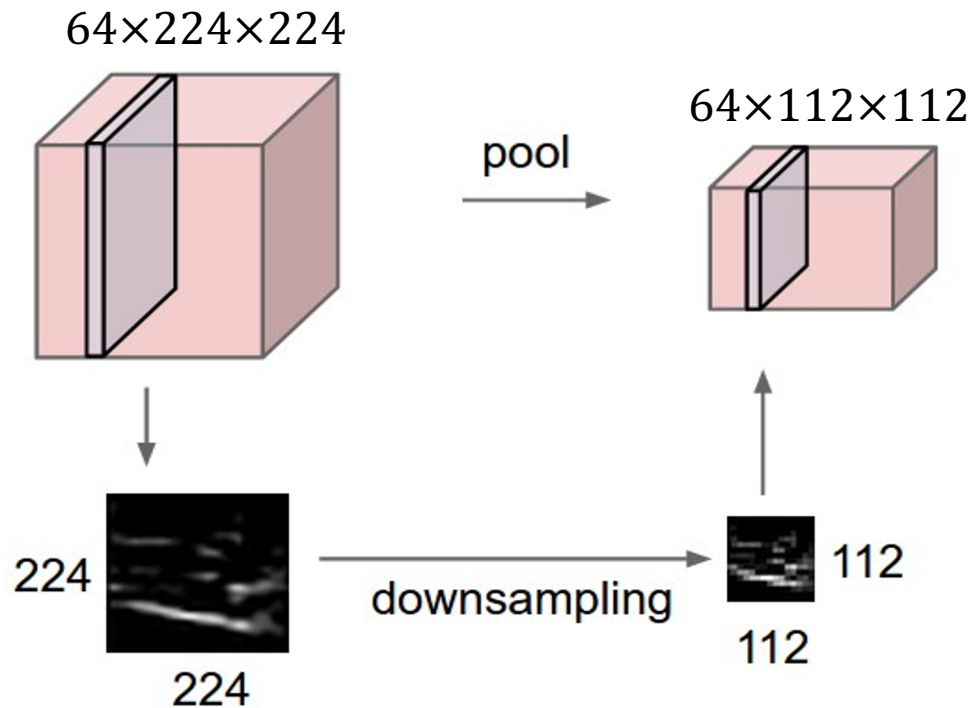
Examples:

```
>>> # With square kernels and equal stride
>>> m = nn.Conv2d(16, 33, 3, stride=2)
>>> # non-square kernels and unequal stride and with padding
>>> m = nn.Conv2d(16, 33, (3, 5), stride=(2, 1), padding=(4, 2))
>>> # non-square kernels and unequal stride and with padding and dilation
>>> m = nn.Conv2d(16, 33, (3, 5), stride=(2, 1), padding=(4, 2), dilation=(3, 1))
>>> input = torch.randn(20, 16, 50, 100)
>>> output = m(input)
```


An example of convolutional network: LeNet 5



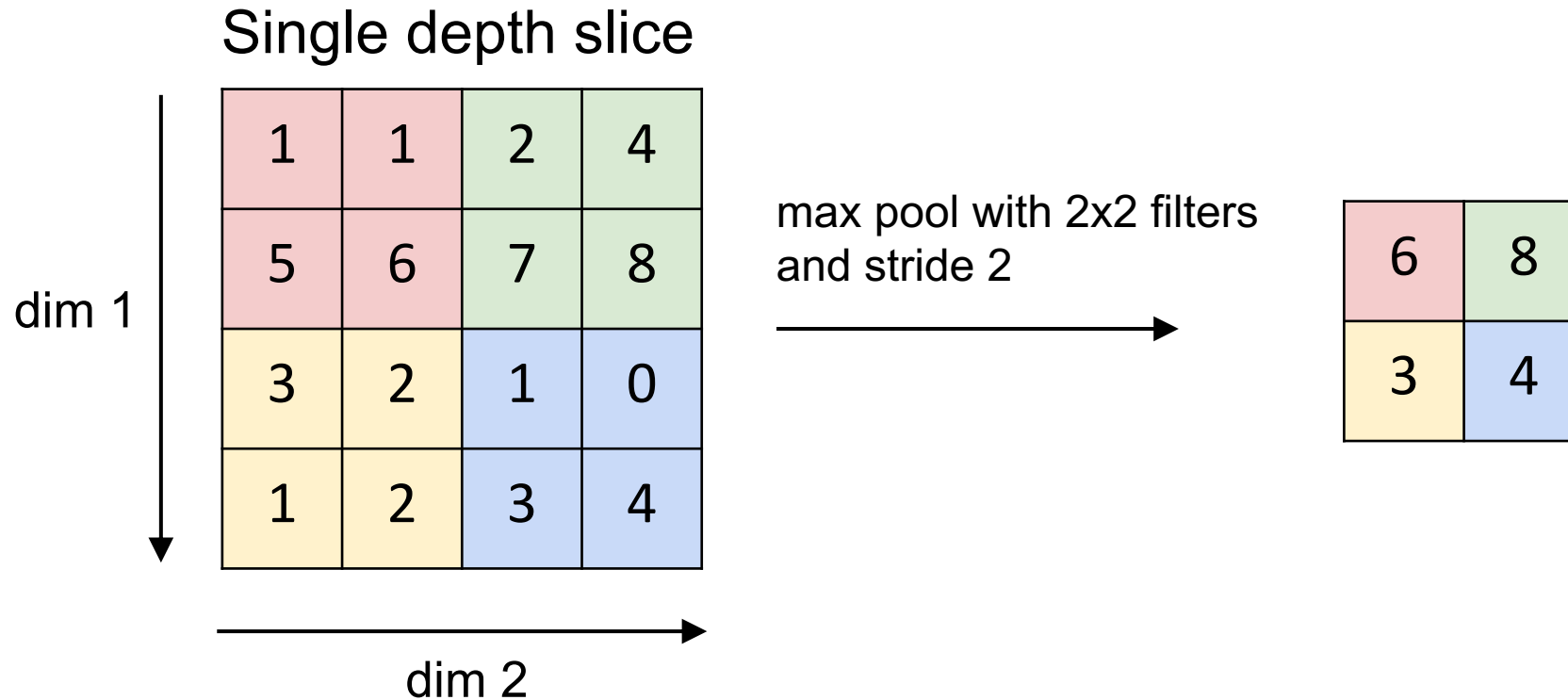
Pooling layer



- Operates over each activation map independently
- Either '**max**' or '**average**' pooling is used at the pooling layer. That is, the convolved features are divided into disjoint regions and pooled by taking either maximum or averaging.

Pooling layer

MAX Pooling



*Pooling is intended to subsample the convolution layer.
The default stride for pooling is equal to the filter width.*

Pooling layer

Consider pooling with non-overlapping windows $\{(l, m)\}_{l,m=-L/2,-M/2}^{L/2,M/2}$, of size $L \times M$

The **max pooling** output is the maximum of the activation inside the pooling window. Pooling of a feature map y at $p = (i, j)$ produce pooled feature

$$z(i, j) = \max_{l, m} \{y(i + l, j + m)\}$$

The **mean pooling** output is the mean of activations in the pooling window

$$z(i, j) = \frac{1}{L \times M} \sum_l \sum_m y(i + l, j + m)$$

Pooling layer

Why pooling?

A function f is **invariant** to g if $f(g(x)) = f(x)$.

- Pooling layers can be used for building inner activations that are (slightly) invariant to small translations of the input.
- Invariance to local translation is helpful if we care more about the presence of a pattern rather than its exact position.

Pooling layer - summary

A pooling layer

- Accepts a volume of size $D_1 \times H_1 \times W_1$
- Requires two hyperparameters
 - Their spatial extent F
 - The stride S
- Produces a volume of size $D_2 \times H_2 \times W_2$, where
 - $W_2 = (W_1 - F)/S + 1$
 - $H_2 = (H_1 - F)/S + 1$
 - $D_2 = D_1$
- Introduces zero parameters since it computes a fixed function of the input
- Note that it is not common to use zero-padding for pooling layers

Pooling layer in PyTorch

MaxPool2d

CLASS `torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False, ceil_mode=False)`

[SOURCE]

Applies a 2D max pooling over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C, H, W) , output (N, C, H_{out}, W_{out}) and `kernel_size` (kH, kW) can be precisely described as:

$$out(N_i, C_j, h, w) = \max_{m=0, \dots, kH-1} \max_{n=0, \dots, kW-1} input(N_i, C_j, stride[0] \times h + m, stride[1] \times w + n)$$

If `padding` is non-zero, then the input is implicitly zero-padded on both sides for `padding` number of points. `dilation` controls the spacing between the kernel points. It is harder to describe, but this [link](#) has a nice visualization of what `dilation` does.

The parameters `kernel_size`, `stride`, `padding`, `dilation` can either be:

- a single `int` – in which case the same value is used for the height and width dimension
- a `tuple` of two ints – in which case, the first `int` is used for the height dimension, and the second `int` for the width dimension

Examples:

```
>>> # pool of square window of size=3, stride=2
>>> m = nn.MaxPool2d(3, stride=2)
>>> # pool of non-square window
>>> m = nn.MaxPool2d((3, 2), stride=(2, 1))
>>> input = torch.randn(20, 16, 50, 32)
>>> output = m(input)
```

AvgPool2d

CLASS `torch.nn.AvgPool2d(kernel_size, stride=None, padding=0, ceil_mode=False, count_include_pad=True, divisor_override=None)`

[SOURCE]

Applies a 2D average pooling over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C, H, W) , output (N, C, H_{out}, W_{out}) and `kernel_size` (kH, kW) can be precisely described as:

$$out(N_i, C_j, h, w) = \frac{1}{kH * kW} \sum_{m=0}^{kH-1} \sum_{n=0}^{kW-1} input(N_i, C_j, stride[0] \times h + m, stride[1] \times w + n)$$

If `padding` is non-zero, then the input is implicitly zero-padded on both sides for `padding` number of points.

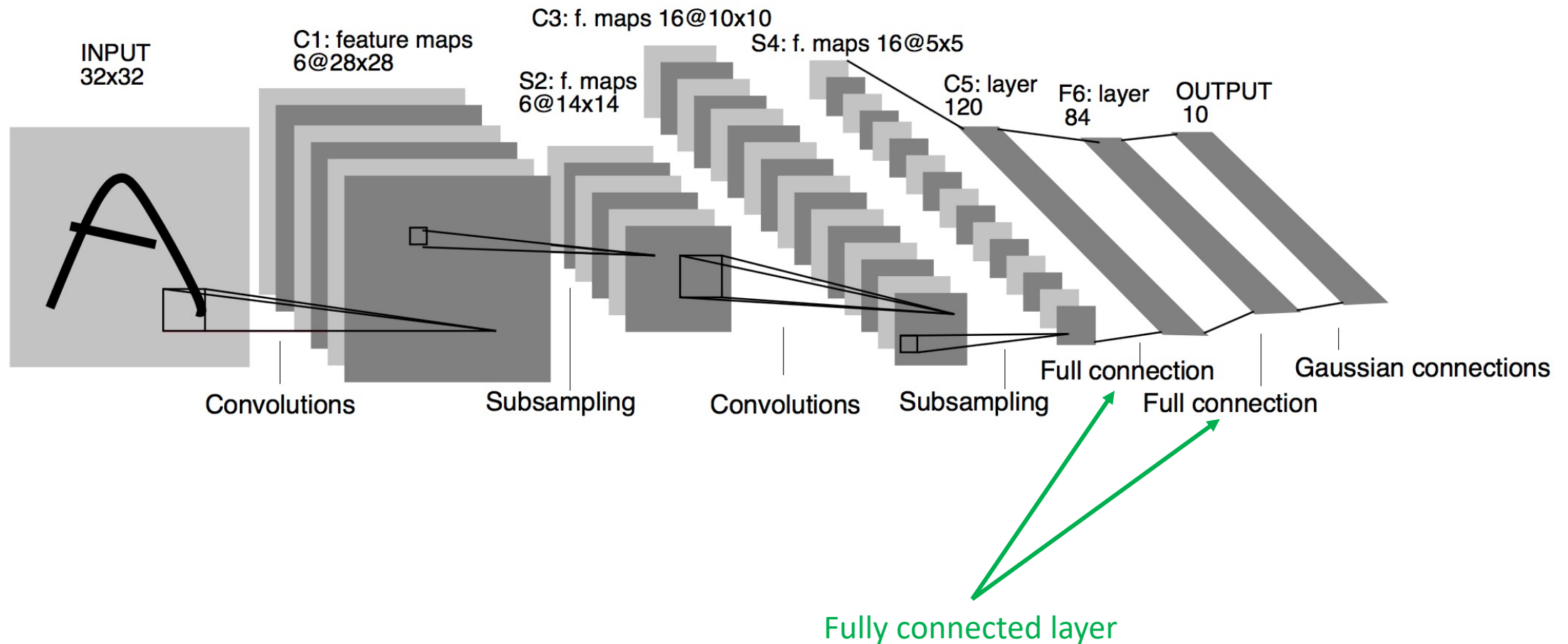
The parameters `kernel_size`, `stride`, `padding` can either be:

- a single `int` – in which case the same value is used for the height and width dimension
- a `tuple` of two ints – in which case, the first `int` is used for the height dimension, and the second `int` for the width dimension

Examples:

```
>>> # pool of square window of size=3, stride=2
>>> m = nn.AvgPool2d(3, stride=2)
>>> # pool of non-square window
>>> m = nn.AvgPool2d((3, 2), stride=(2, 1))
>>> input = torch.randn(20, 16, 50, 32)
>>> output = m(input)
```

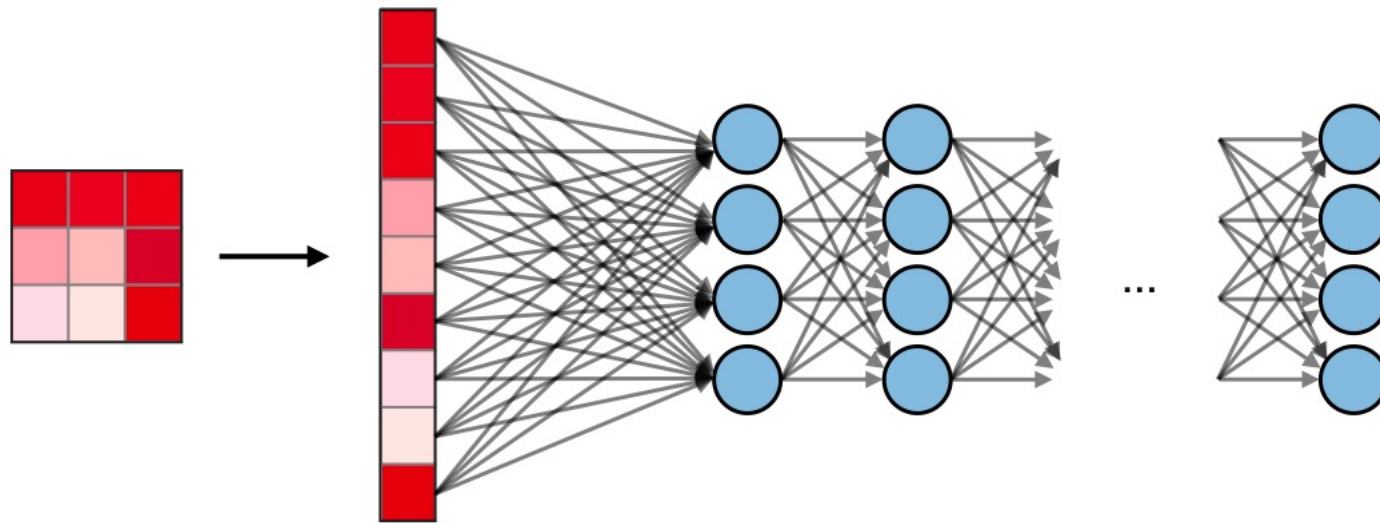
An example of convolutional network: LeNet 5



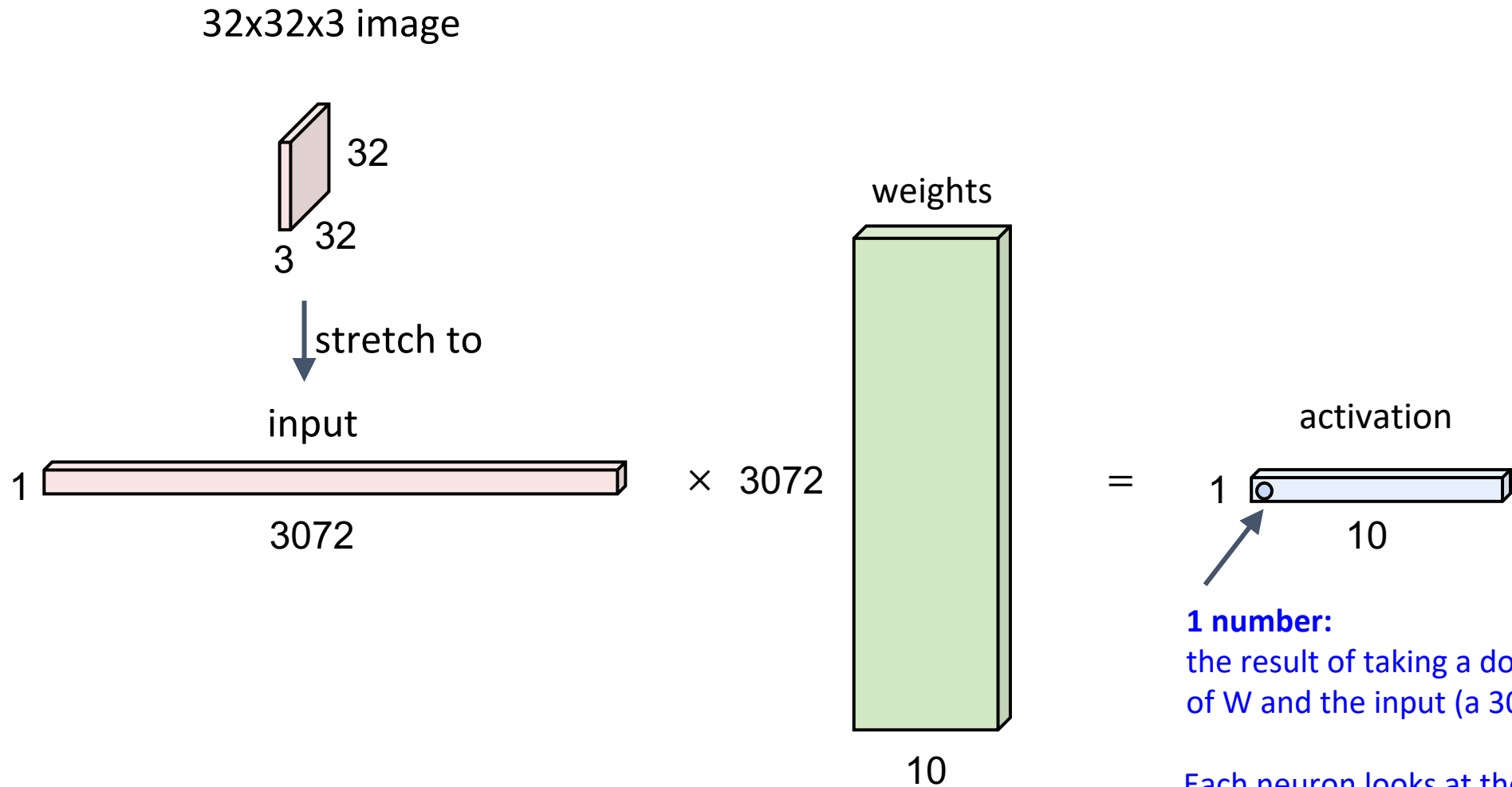
Fully connected layer

The fully connected layer (FC) operates on a **flattened input** where each input is connected to all neurons.

If present, FC layers are usually found towards the end of CNN architectures and can be used to optimize objectives such as class scores.



Fully connected layer



1 number:

the result of taking a dot product between a column of W and the input (a 3072-dimensional dot product)

Each neuron looks at the full input volume

Fully connected layer in PyTorch

Linear

CLASS `torch.nn.Linear(in_features, out_features, bias=True)`

[\[SOURCE\]](#)

Applies a linear transformation to the incoming data: $y = xA^T + b$

Parameters

- **in_features** – size of each input sample
- **out_features** – size of each output sample
- **bias** – If set to `False`, the layer will not learn an additive bias. Default: `True`

Shape:

- Input: $(N, *, H_{in})$ where $*$ means any number of additional dimensions and $H_{in} = \text{in_features}$
- Output: $(N, *, H_{out})$ where all but the last dimension are the same shape as the input and $H_{out} = \text{out_features}$.

Variables

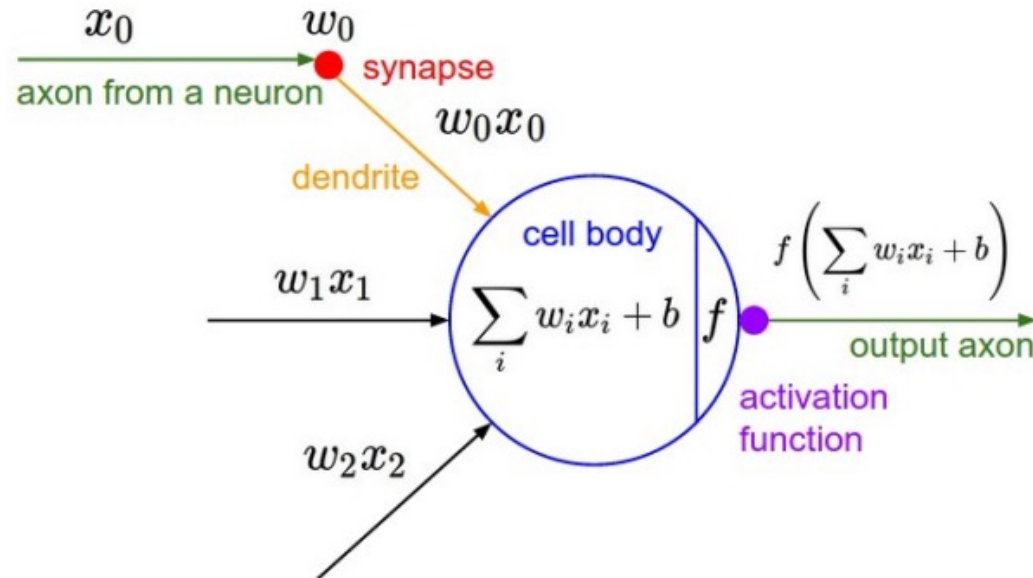
- **~Linear.weight** – the learnable weights of the module of shape $(\text{out_features}, \text{in_features})$. The values are initialized from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$, where $k = \frac{1}{\text{in_features}}$
- **~Linear.bias** – the learnable bias of the module of shape (out_features) . If `bias` is `True`, the values are initialized from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ where $k = \frac{1}{\text{in_features}}$

Activation function

Recall that the output of the neuron at (i, j) of the convolution layer

$$y(i, j) = f(u(i, j))$$

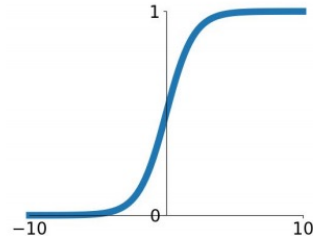
where u is the synaptic input and f is an activation function (It aims at introducing non-linearities to the network.).



Activation function

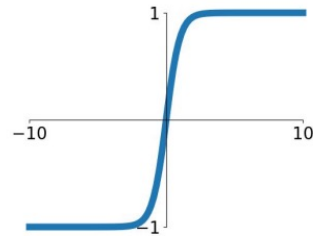
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



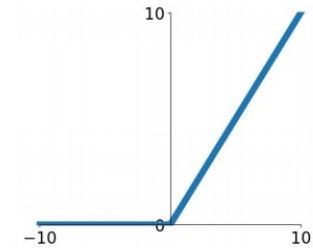
tanh

$$\tanh(x)$$



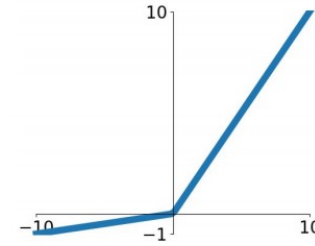
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

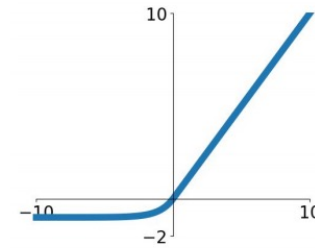


Maxout

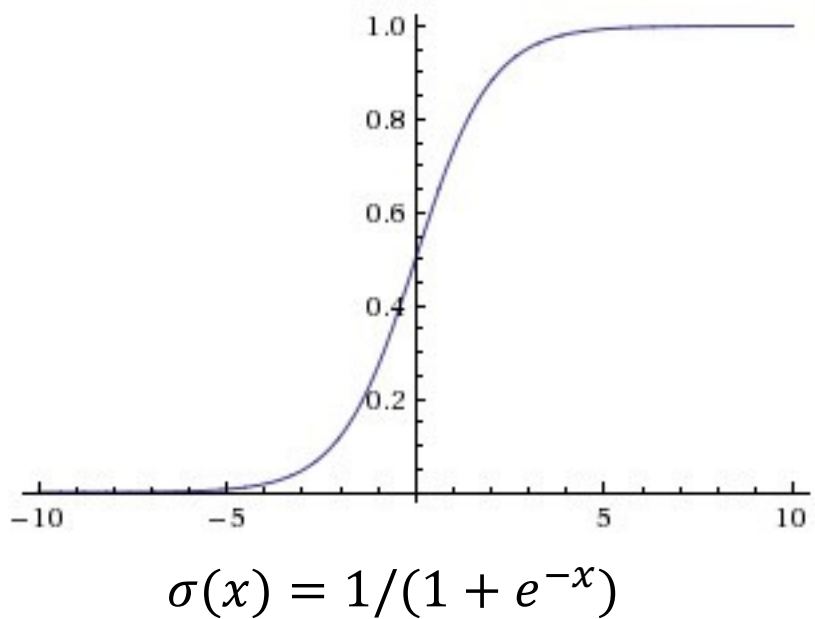
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

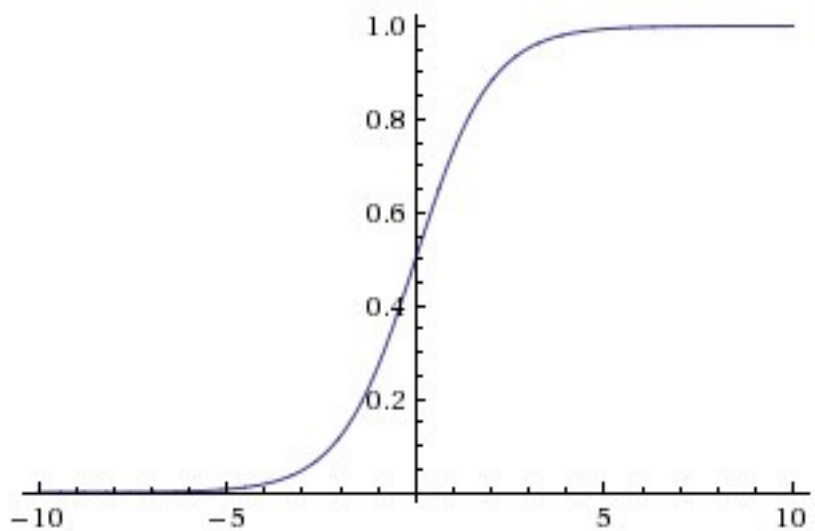


Activation function – Sigmoid

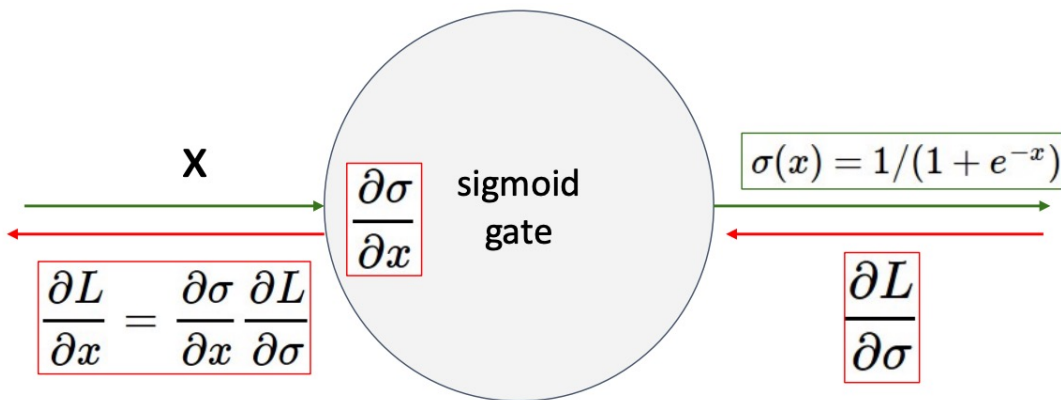


- Sigmoid non-linearity squashes real numbers to range between [0,1]
- Large negative numbers become 0 and large positive numbers become 1
- Frequent use historically since it has a nice interpretation as the firing rate of a neuron

Activation function – Sigmoid

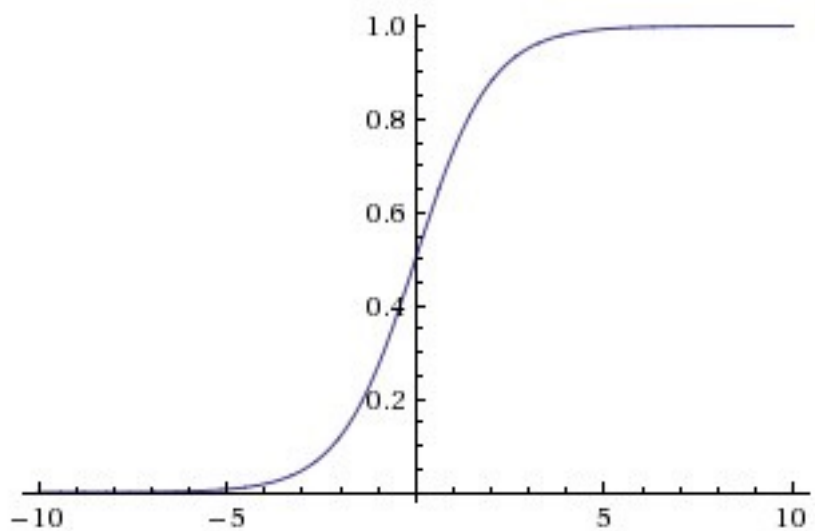


$$\sigma(x) = 1/(1 + e^{-x})$$



- The sigmoid non-linearity has fallen out of favor and it is rarely ever used
- Weaknesses:
 - *Sigmoids saturate and kill gradients*
 - A very undesirable property of the sigmoid neuron is that when the neuron's activation saturates at either tail of 0 or 1, the gradient at these regions is almost zero
 - During backpropagation, this (local) gradient will be multiplied to the gradient of this gate's output for the whole objective. Therefore, if the local gradient is very small, it will effectively "kill" the gradient and almost no signal will flow through the neuron to its weights and recursively to its data. This will make learning very slow.

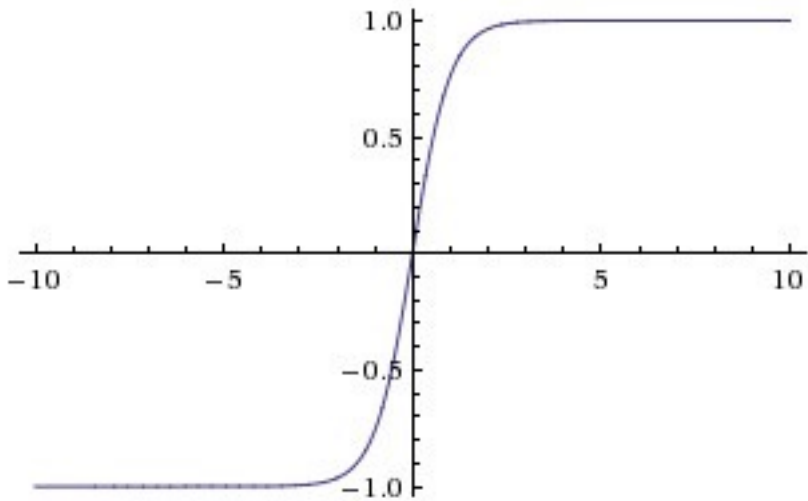
Activation function – Sigmoid



$$\sigma(x) = 1/(1 + e^{-x})$$

- Weaknesses:
 - *Sigmoid outputs are not zero-centered*
 - This is undesirable since neurons in later layers of processing in a neural network would be receiving data that is not zero-centered. This has implications on the dynamics during gradient descent (zig-zagging)

Activation function – Tanh

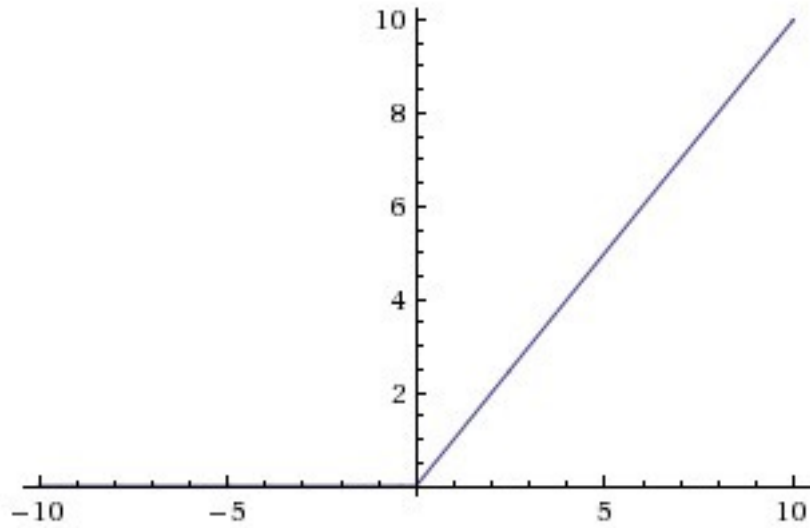


simply a scaled sigmoid neuron

$$\tanh(x) = 2\sigma(2x) - 1$$

- The tanh non-linearity squashes real numbers to range between $[-1,1]$
- Its activations saturate, but unlike the sigmoid neuron its output is zero-centered.
- Therefore, in practice the tanh non-linearity is always preferred to the sigmoid nonlinearity.

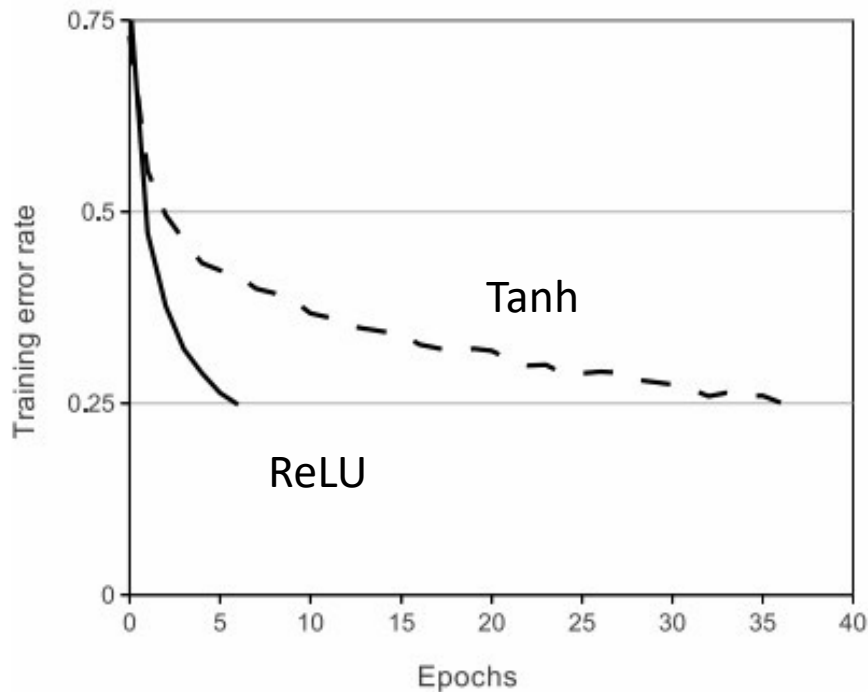
Activation function – ReLU



$$f(x) = \max(0, x)$$

- Rectified Linear Unit (ReLU) activation function, which is zero when $x < 0$ and then linear with slope 1 when $x > 0$

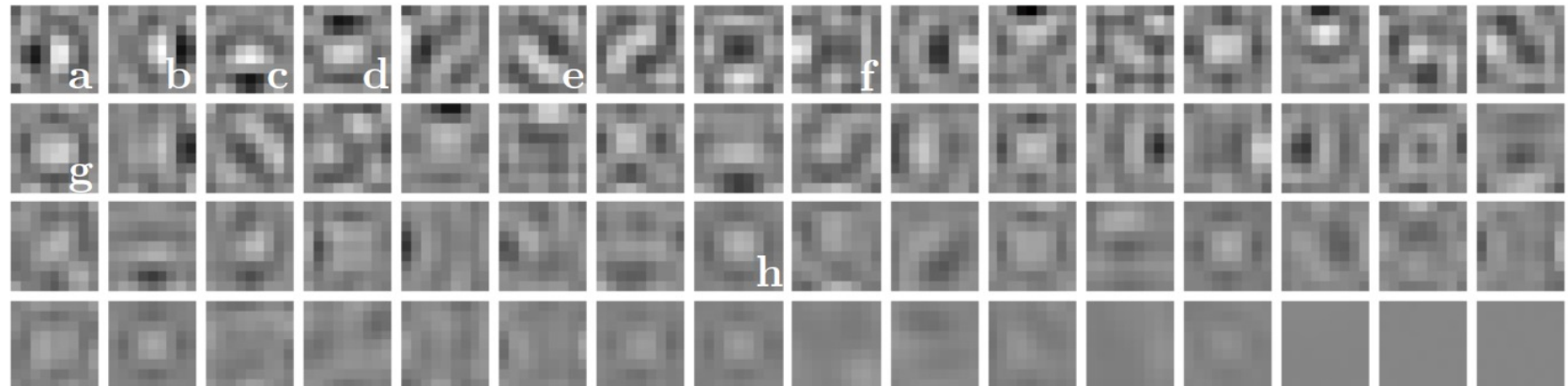
Activation function – ReLU



- (+) It was found to greatly accelerate (e.g. a factor of 6 in [Krizhevsky et al.](#)) the convergence of stochastic gradient descent compared to the sigmoid/tanh functions. It is argued that this is due to its linear, non-saturating form.
- (+) Compared to tanh/sigmoid neurons that involve expensive operations (exponentials, etc.), the ReLU can be implemented by simply thresholding a matrix of activations at zero.

Activation function – ReLU

- (-) Unfortunately, ReLU units can be fragile during training and can "die".
- For example, a large gradient flowing through a ReLU neuron could cause the weights to update in such a way that the neuron will never activate on any data point again (fall into the negative regime).
- If this happens, then the gradient flowing through the unit will forever be zero from that point on.
 - That is, the ReLU units can irreversibly die during training since they can get knocked off the data manifold.



Activation function – Leaky ReLU

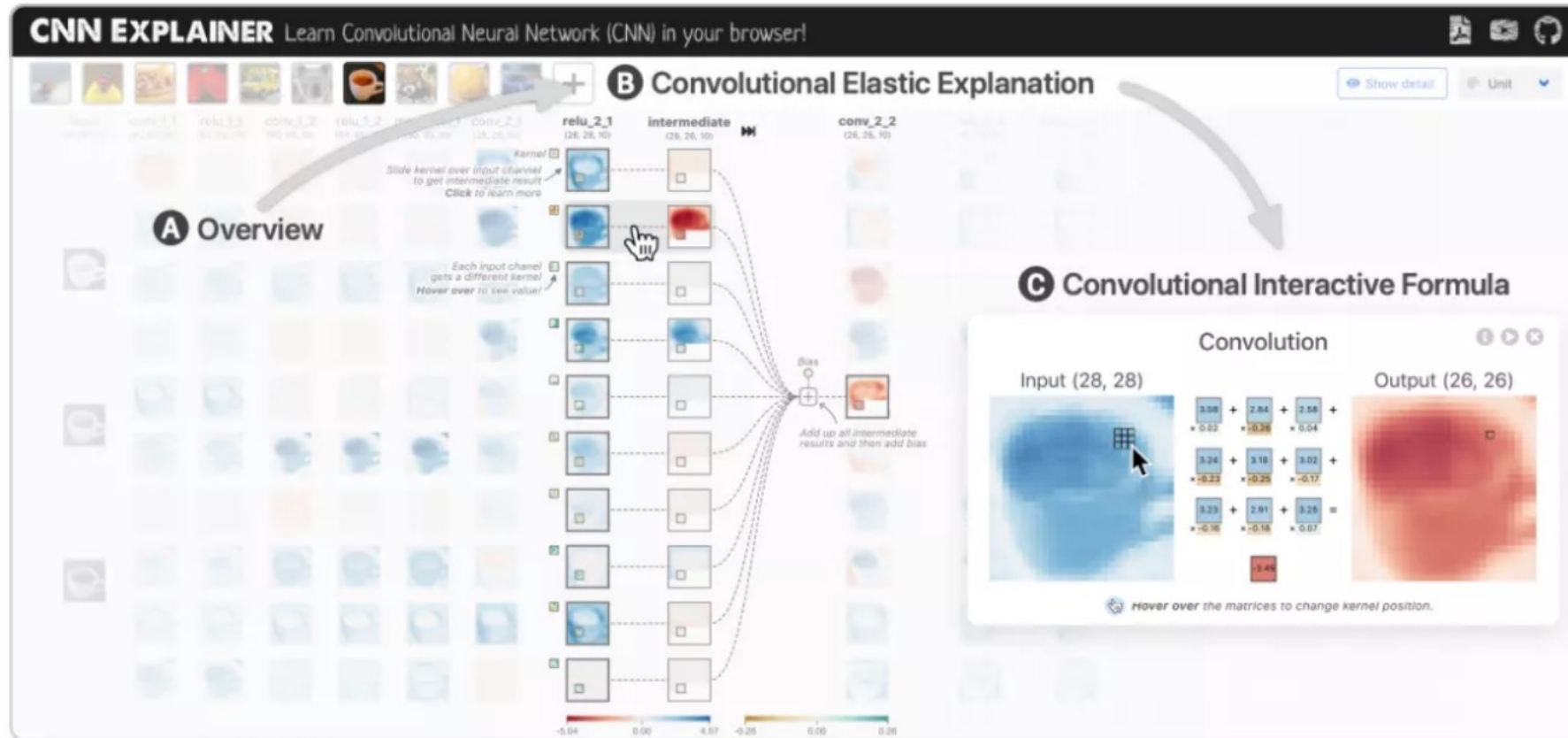
- Leaky ReLUs are one attempt to fix the "dying ReLU" problem. Instead of the function being zero when $x < 0$, a leaky ReLU will instead have a small negative slope (of 0.01, or so). That is, the function computes

$$f(x) = \mathbb{1}(x < 0)(\alpha x) + \mathbb{1}(x \geq 0)(x)$$

where α is a small constant.

- Some people report success with this form of activation function, but the results are not always consistent.

Visualization



Outline

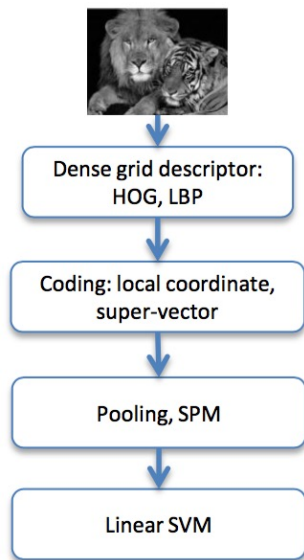
- Basic components in CNN
- CNN architectures
- Short introduction to Colab

CNN Architectures

Deep networks for ImageNet

Year 2010

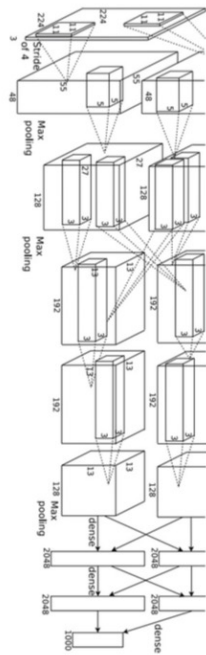
NEC-UIUC



[Lin CVPR 2011]

Year 2012

AlexNet



[Krizhevsky NIPS 2012]

Year 2014

GoogLeNet



[Szegedy arxiv 2014]

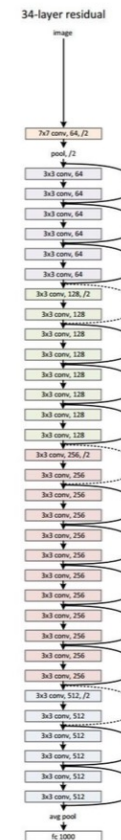
VGG



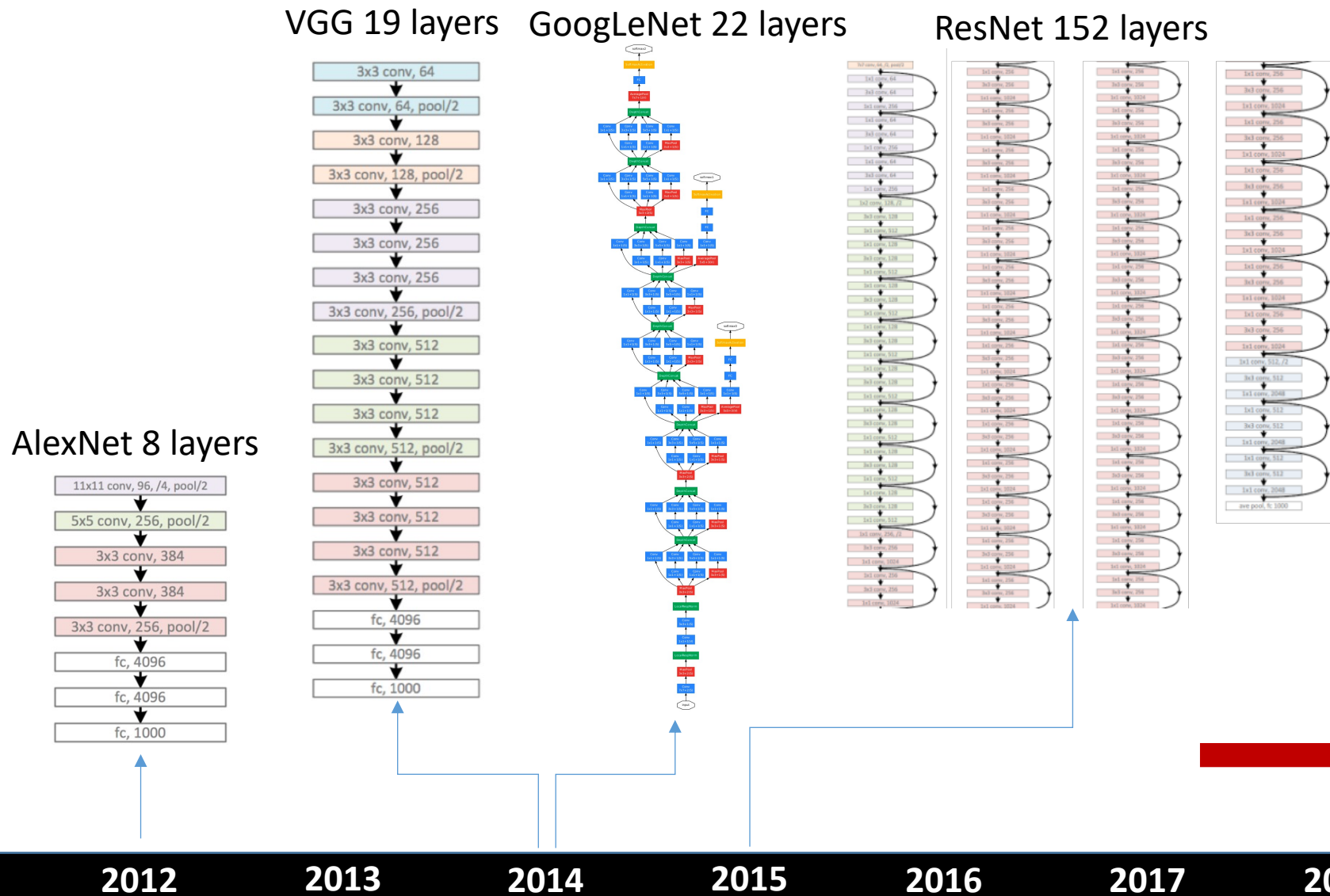
[Simonyan arxiv 2014]

Year 2015

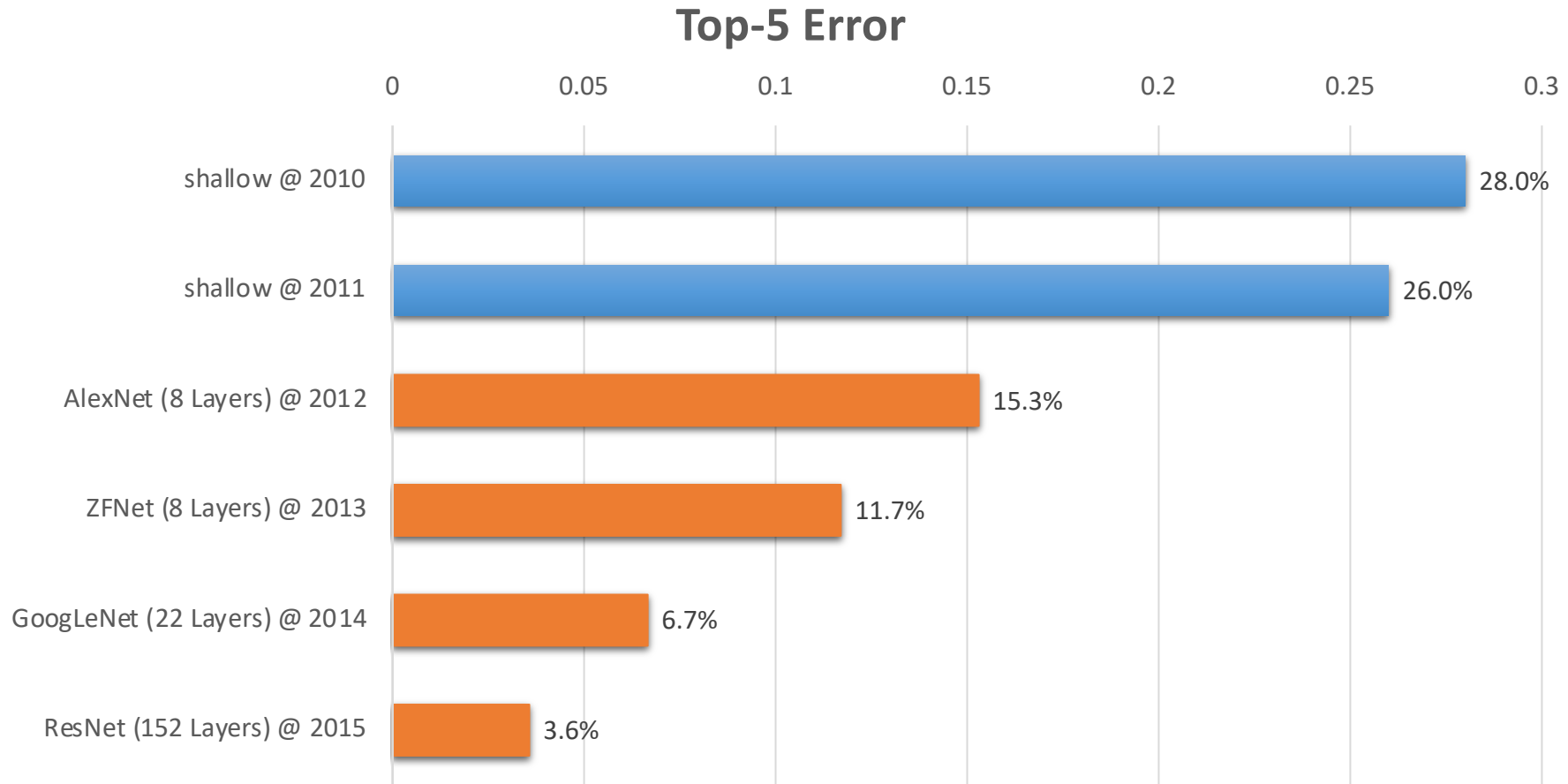
MSRA ResNet



Deep architectures

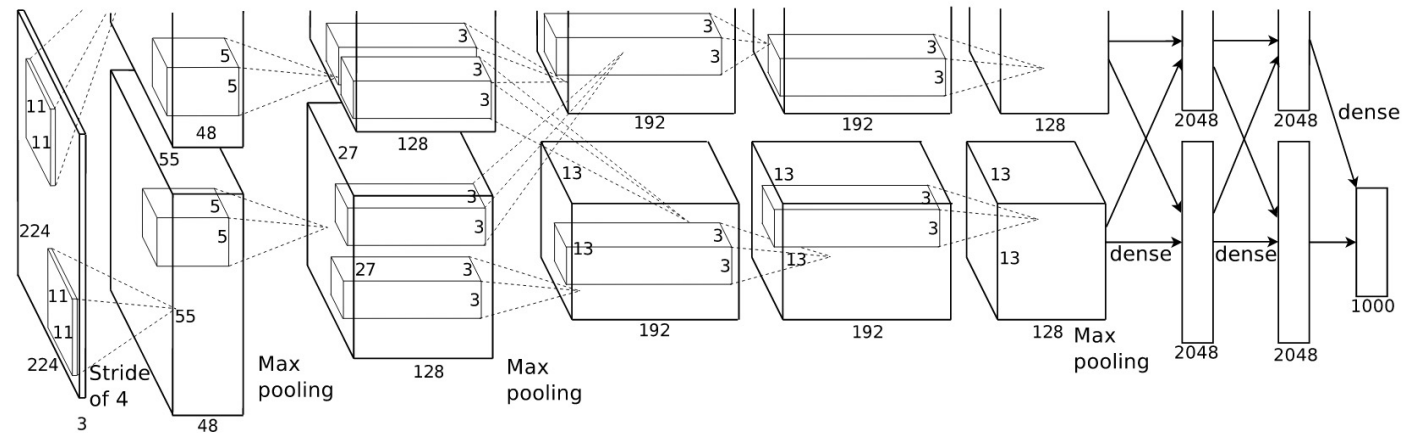
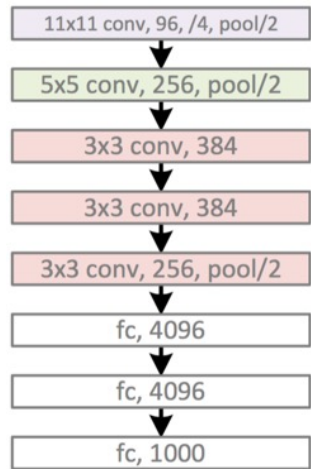


Performance of previous years on ImageNet



Depth is the key to high classification accuracy.

Deep architectures - AlexNet



- The split (i.e. two pathways) in the image above are the split between two GPUs.
- Trained for about a week on two NVIDIA GTX 580 3GB GPU
- 60 million parameters
- Input layer: size 227x227x3
- 8 layers deep: 5 convolution and pooling layers and 3 fully connected layers

2012

2013

2014

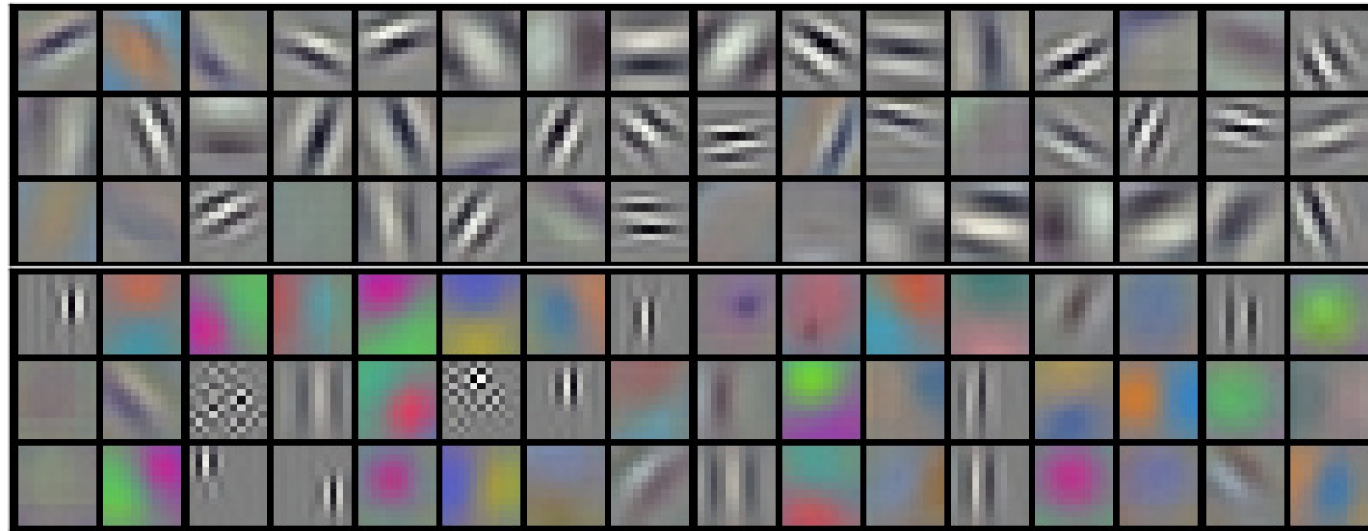
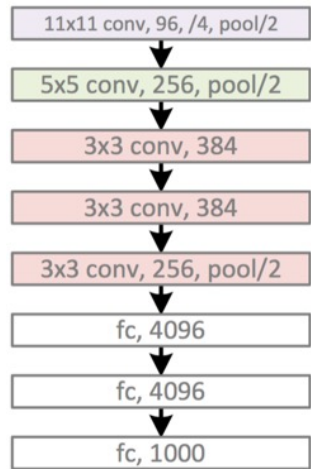
2015

2016

2017

2018 ...

Deep architectures - AlexNet



96 kernels learned by first convolution layer; 48 kernels were learned by each GPU

2012

2013

2014

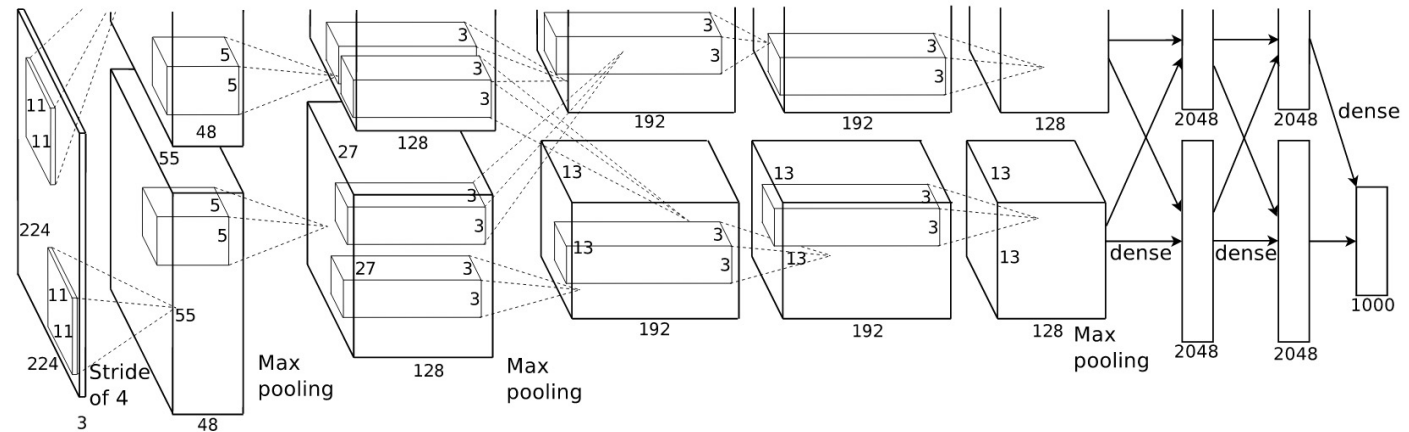
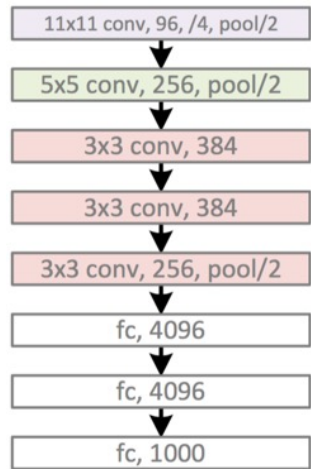
2015

2016

2017

2018 ...

Deep architectures - AlexNet



- Escape from a few layers
 - ReLU nonlinearity for solving gradient vanishing
 - Data augmentation
 - Dropout
 - Outperformed all previous models on ILSVRC by 10%

2012

2013

2014

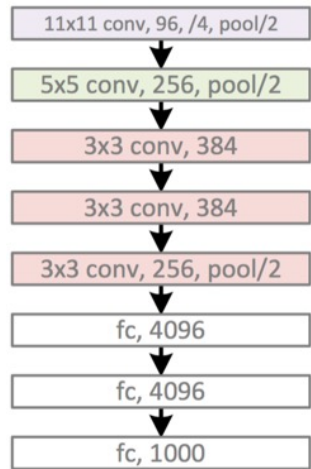
2015

2016

2017

2018 ...

Deep architectures - AlexNet



Layer	Weights
L1 (Conv)	34,944
L2 (Conv)	614,656
L3 (Conv)	885,120
L4 (Conv)	1,327,488
L5 (Conv)	884,992
L6 (FC)	37,752,832
L7 (FC)	16,781,312
L8 (FC)	4,097,000
Conv Subtotal	3,747,200
FC Subtotal	58,631,144
Total	62,378,344

← First convolution layer: 96 kernels of size 11x11x3, with a stride of 4 pixels

Number of parameters = $(11 \times 11 \times 3 + 1) \times 96 = 34,944$

Note: There are no parameters associated with a pooling layer. The pool size, stride, and padding are hyperparameters.

2012

2013

2014

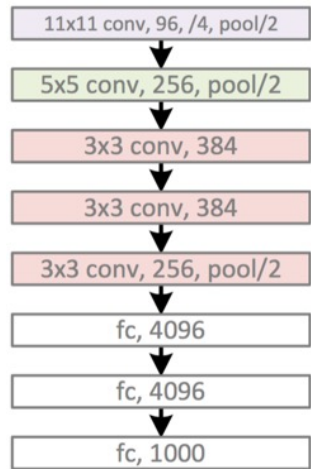
2015

2016

2017

2018 ...

Deep architectures - AlexNet



Layer	Weights
L1 (Conv)	34,944
L2 (Conv)	614,656
L3 (Conv)	885,120
L4 (Conv)	1,327,488
L5 (Conv)	884,992
L6 (FC)	37,752,832
L7 (FC)	16,781,312
L8 (FC)	4,097,000
Conv Subtotal	3,747,200
FC Subtotal	58,631,144
Total	62,378,344

← Second convolution layer: 256 kernels of size 5x5x96
Number of parameters = $(5 \times 5 \times 96 + 1) \times 256 = 614,656$

2012

2013

2014

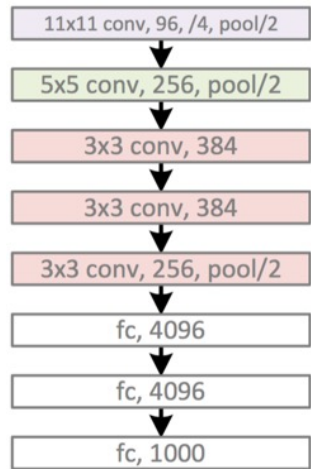
2015

2016

2017

2018 ...

Deep architectures - AlexNet



Layer	Weights
L1 (Conv)	34,944
L2 (Conv)	614,656
L3 (Conv)	885,120
L4 (Conv)	1,327,488
L5 (Conv)	884,992
L6 (FC)	37,752,832
L7 (FC)	16,781,312
L8 (FC)	4,097,000
Conv Subtotal	3,747,200
FC Subtotal	58,631,144
Total	62,378,344



First FC layer:

Number of neurons = 4096

Number of kernels in the previous Conv Layer = 256

Size (width) of the output image of the previous Conv Layer = 6

Number of parameters = $(6 \times 6 \times 256 \times 4096) + 4096 = 37,752,832$

2012

2013

2014

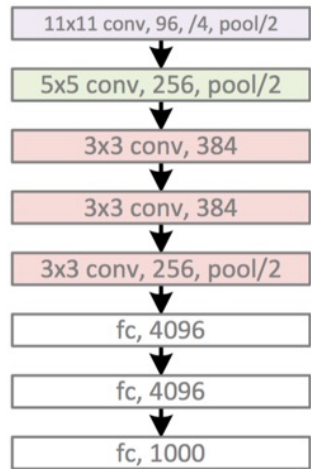
2015

2016

2017

2018 ...

Deep architectures - AlexNet



Layer	Weights
L1 (Conv)	34,944
L2 (Conv)	614,656
L3 (Conv)	885,120
L4 (Conv)	1,327,488
L5 (Conv)	884,992
L6 (FC)	37,752,832
L7 (FC)	16,781,312
L8 (FC)	4,097,000
Conv Subtotal	3,747,200
FC Subtotal	58,631,144
Total	62,378,344

The last FC layer:

Number of neurons = 1000

Number of neurons in the previous FC Layer = 4096

Number of parameters = $(1000 * 4096) + 1000 = 4,097,000$

2012

2013

2014

2015

2016

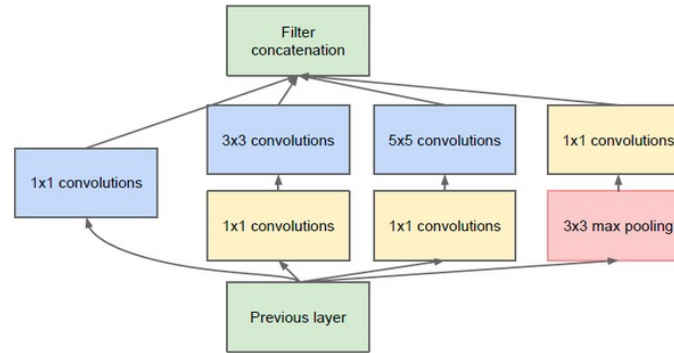
2017

2018 ...

Deep architectures - GoogLeNet



- An important lesson - go deeper
- Inception structures (v2, v3, v4)
 - Reduce parameters (4M vs 60M in AlexNet)



The 1x1 convolutions are performed to reduce the dimensions of input/output

- Batch normalization
 - Normalization the activation for each training mini-batch
 - Allows us to use much higher learning rates and be less careful about initialization

2012

2013

2014

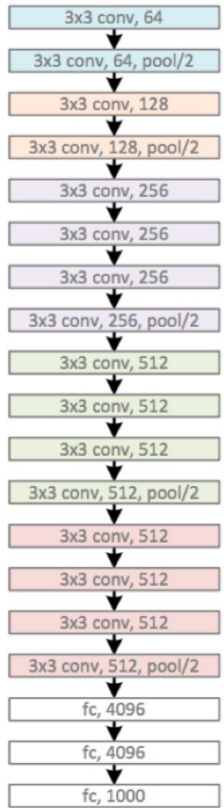
2015

2016

2017

2018 ...

Deep architectures - VGG



- An important lesson - go deeper
- 140M parameters
- Now commonly used for computing perceptual loss

2012

2013

2014

2015

2016

2017

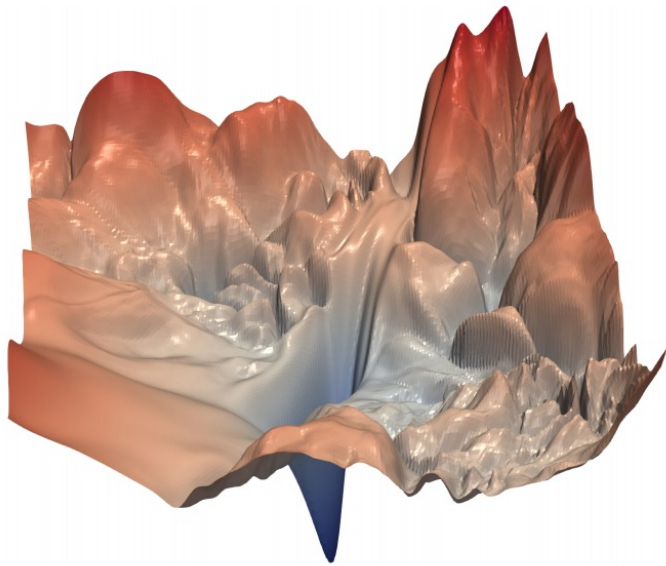
2018 ...

2012	2013	2014	2015	2016	2017	2018 ...
------	------	------	------	------	------	----------

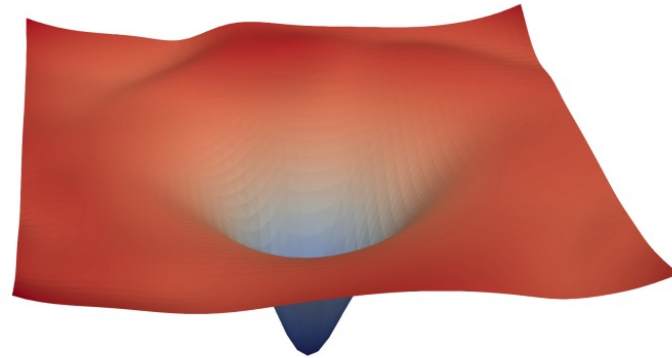


-
- Diagram illustrating a residual block structure:
- Input x splits into two paths.
 - The top path consists of two "weight layer" blocks followed by a "relu" activation.
 - The bottom path is a direct "identity" connection (labeled in red).
 - The outputs of both paths are added at a summation node ($\oplus$).
 - The result is then passed through a final "relu" activation.
 - The output is labeled $c) = F(x) + x$.
 - Text annotation: "drives the new layer to learn something different".

The importance of skip connections



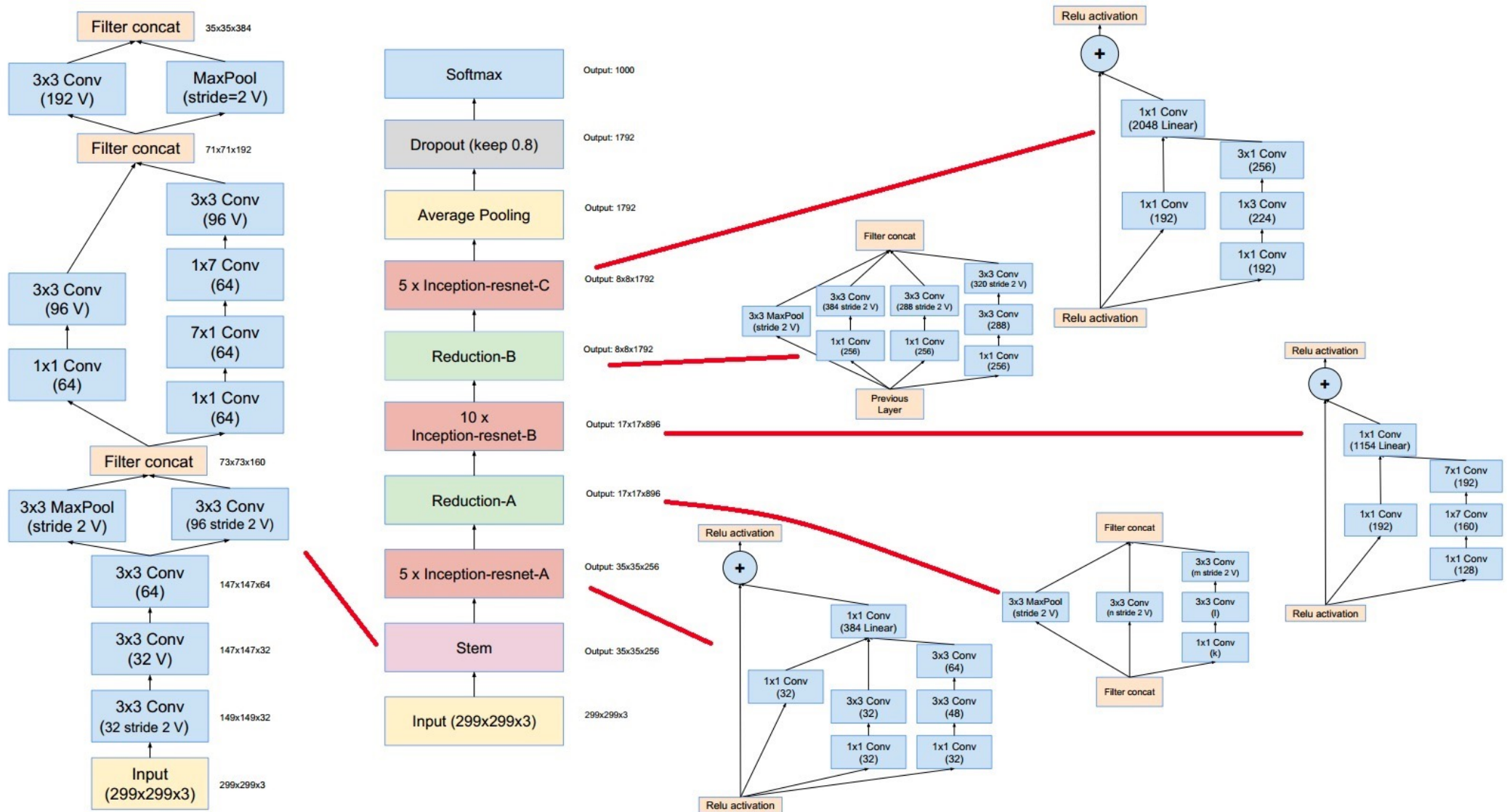
(a) without skip connections



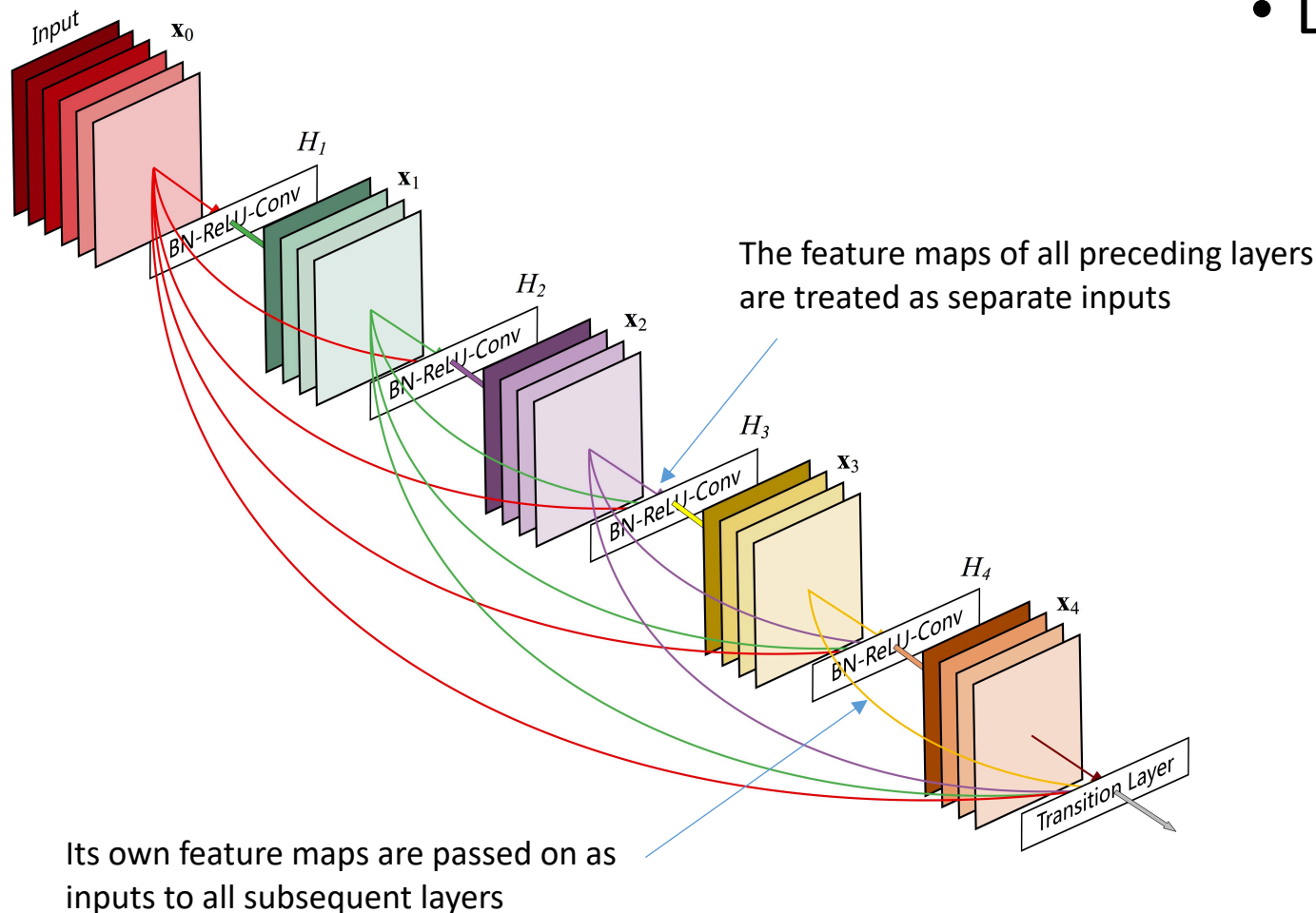
(b) with skip connections

- When networks become sufficiently deep, neural loss landscapes quickly transition from being nearly convex to being highly **chaotic**.
- This transition from convex to chaotic behavior coincides with a dramatic increase in generalization error, and ultimately to a lack of trainability.
- Skip connections **promote flat minimizers** and prevent the transition to chaotic behavior, which helps explain why skip connections are necessary for training extremely deep networks.

Deep architectures - Inception-ResNet



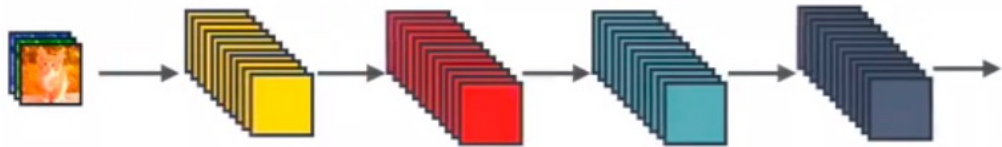
Deep architectures - DenseNet



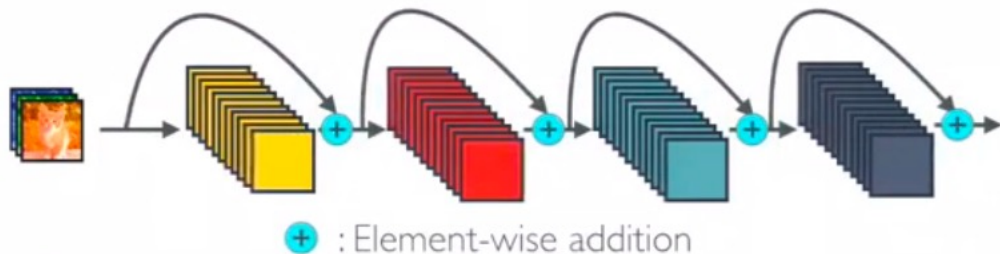
- Dense Block

- Each layer is directly connected to every other layer in a feed-forward fashion
- Each layer has direct access to the gradients from the loss function and the original input signal, leading to an implicit deep supervision
- Deep supervision enforces the intermediate layers to learn discriminative features

Deep architectures - DenseNet

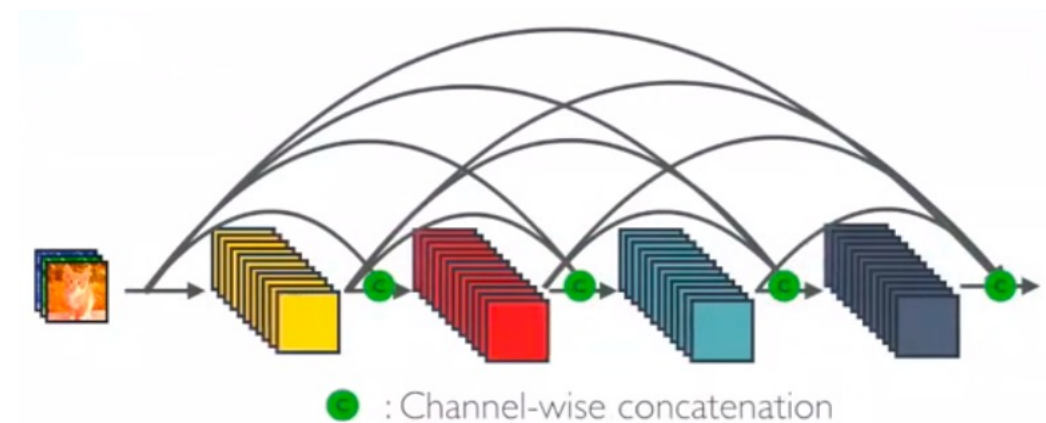


In **Standard ConvNet**, input image goes through multiple convolution and obtain high-level features.



In **ResNet**, identity mapping is proposed to promote the gradient propagation. Element-wise addition is used.

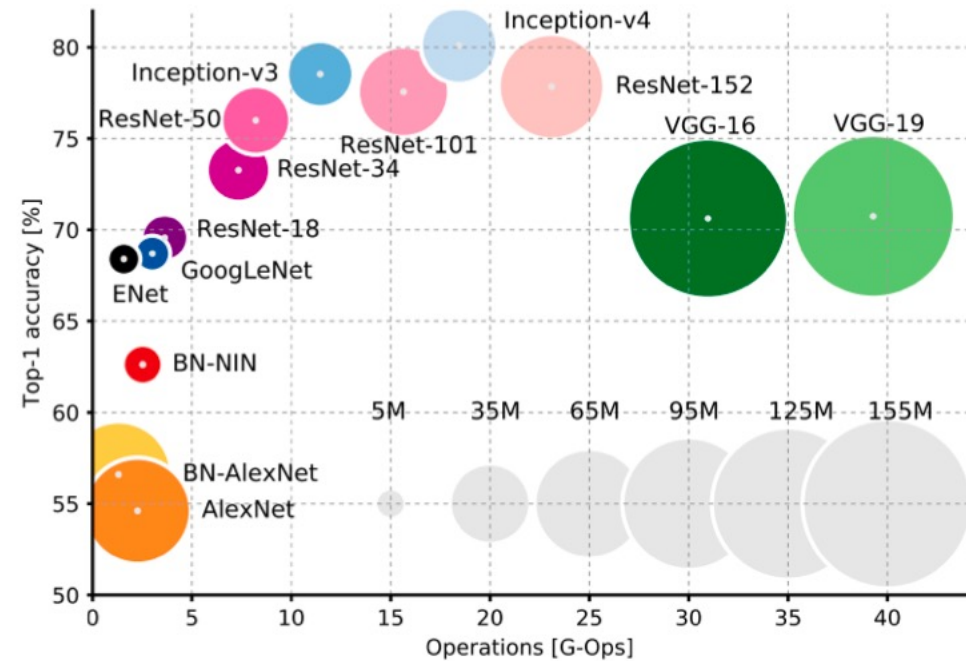
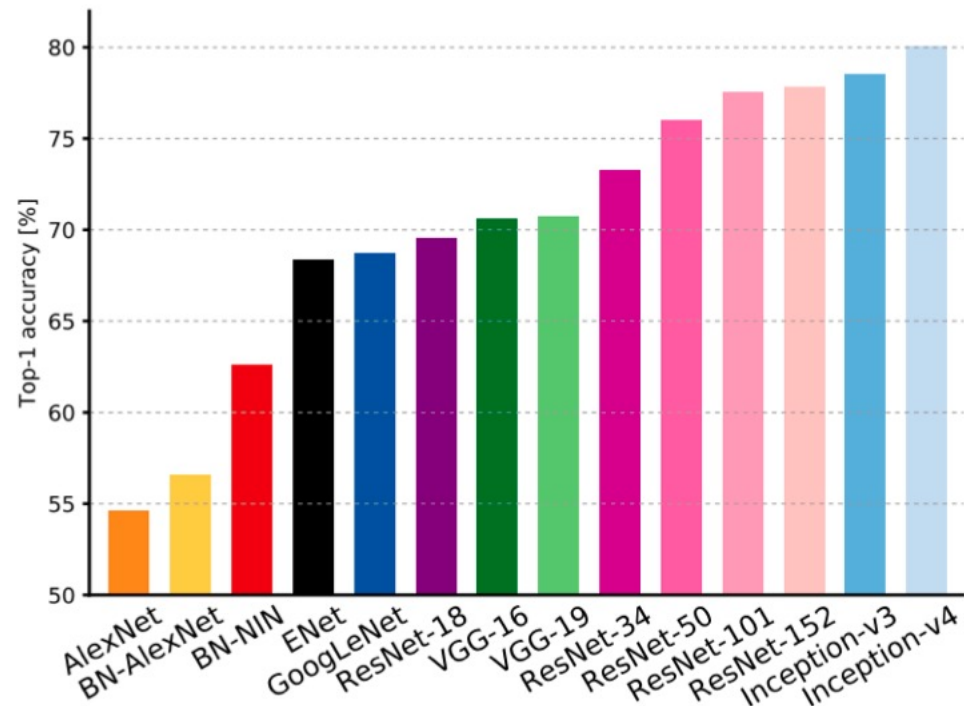
$$\mathbf{x}_\ell = H_\ell(\mathbf{x}_{\ell-1}) + \mathbf{x}_{\ell-1}$$



In **DenseNet**, each layer obtains additional inputs from all preceding layers and passes on its own feature-maps to all subsequent layers. **Concatenation** is used. **Each layer is receiving a “collective knowledge” from all preceding layers.**

$$\mathbf{x}_\ell = H_\ell([\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{\ell-1}])$$

The research is on-going



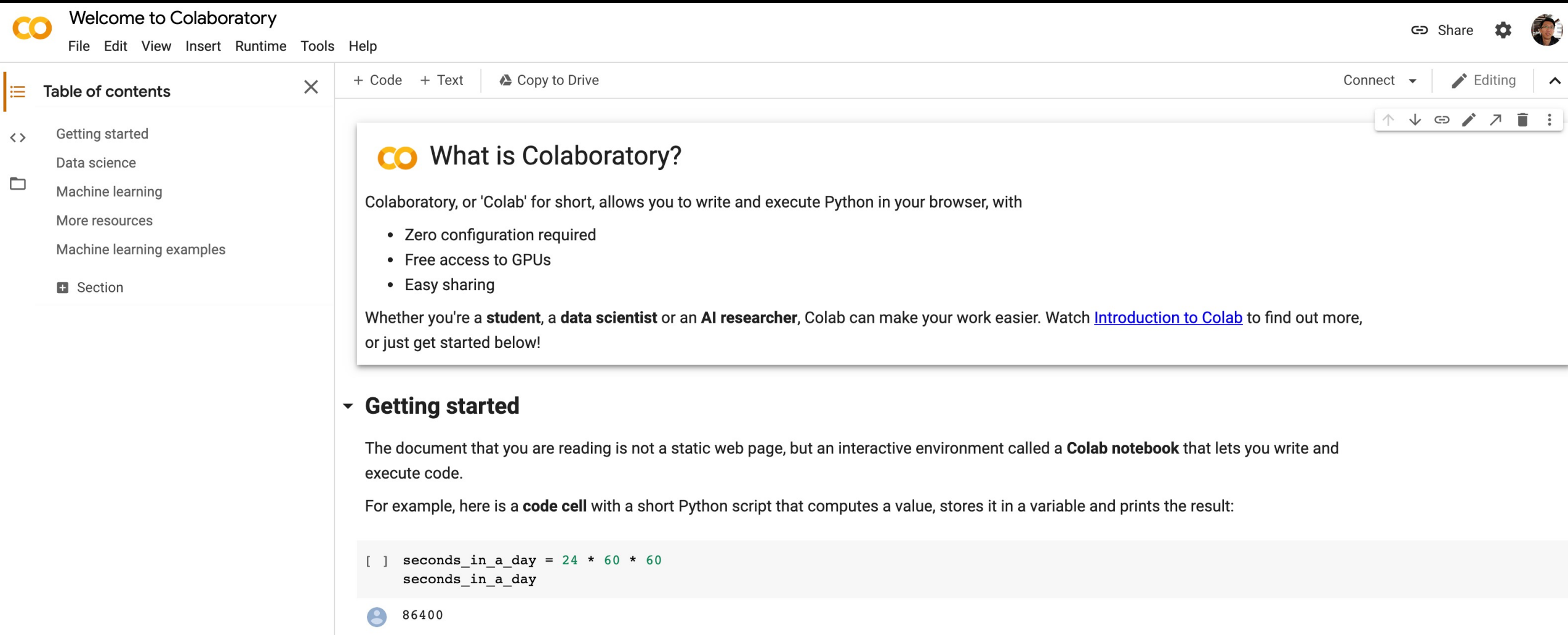
Finding the optimal neural network architecture remains
an [active area of research](#).

Outline

- Basic components in CNN
- CNN architectures
- Short introduction to Colab

Short Introduction to Colab

Get started



The screenshot shows the Google Colaboratory web interface. At the top, there's a header with the Colab logo, the text 'Welcome to Colaboratory', and a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. On the right of the header are 'Share', 'Settings', and a user profile icon. Below the header is a toolbar with '+ Code', '+ Text', and 'Copy to Drive'. A left sidebar contains a 'Table of contents' with links to 'Getting started', 'Data science', 'Machine learning', 'More resources', 'Machine learning examples', and a '+ Section' button. The main content area displays a 'What is Colaboratory?' section with a Colab logo, a title, a paragraph explaining Colab, a bulleted list of features, and a paragraph about its use for students, data scientists, and AI researchers. Below this is a 'Getting started' section with a paragraph about the interactive environment and a code cell containing a Python script to calculate seconds in a day. The code cell shows the output '86400'.

co Welcome to Colaboratory

File Edit View Insert Runtime Tools Help

Share Settings User Profile

+ Code + Text Copy to Drive

Connect Editing

Table of contents

- <> Getting started
- Data science
- Machine learning
- More resources
- Machine learning examples
- + Section

co What is Colaboratory?

Colaboratory, or 'Colab' for short, allows you to write and execute Python in your browser, with

- Zero configuration required
- Free access to GPUs
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) to find out more, or just get started below!

Getting started

The document that you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable and prints the result:

```
[ ] seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

86400

<https://colab.research.google.com/>

Mounting Google Drive locally

▼ Mounting Google Drive locally

The example below shows how to mount your Google Drive on your runtime using an authorisation code, and how to write and read files there.

Once executed, you will be able to see the new file (`foo.txt`) at <https://drive.google.com/>.

This only supports reading, writing and moving files; to programmatically modify sharing settings or other metadata, use one of the other options below.

Note: When using the 'Mount Drive' button in the file browser, no authentication codes are necessary for notebooks that have only been edited by the current user.

```
[ ] from google.colab import drive
    drive.mount('/content/drive')
```

 Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client_id=947318989803-6bn6qk8qdgf4n4g3pfee6491hc0brc4i.apps.googleusercontent.com&redirect_uri=https://colab.research.google.com/&response_type=code
Enter your authorization code:
.....
Mounted at /content/drive

```
[ ] with open('/content/drive/My Drive/foo.txt', 'w') as f:
    f.write('Hello Google Drive!')
    !cat /content/drive/My Drive/foo.txt
```

 Hello Google Drive!

```
[ ] drive.flush_and_unmount()
    print('All changes made in this colab session should now be visible in Drive.')
```

 All changes made in this colab session should now be visible in Drive.

<https://colab.research.google.com/notebooks/io.ipynb>

An example (padding)

The image shows a JupyterLab interface with a terminal window at the top and a code editor below. The terminal window displays the output of a pip install command for PyTorch and torchvision. The code editor contains Python code that imports torch and torchvision, creates a random tensor 'input' of size (1, 1, 3, 3), and then applies a zero-padding operation to create a new tensor 'm' of size (1, 1, 5, 5). The output of the zero-padding operation is displayed as a 5x5 tensor with the original 3x3 values in the center and zeros elsewhere.

padding.ipynb

File Edit View Insert Runtime Tools Help

+ Code + Text

RAM Disk Editing

```
!pip install -U torch==1.5.1+cu101 torchvision==0.6.1+cu101 -f https://download.pytorch.org/whl/torch_stable.html
```

Looking in links: https://download.pytorch.org/whl/torch_stable.html
Collecting torch==1.5.1+cu101
 Downloading https://download.pytorch.org/whl/cu101/torch-1.5.1%2Bcu101-cp36-cp36m-linux_x86_64.whl (704.4MB)
 704.4MB 24kB/s
Collecting torchvision==0.6.1+cu101
 Downloading https://download.pytorch.org/whl/cu101/torchvision-0.6.1%2Bcu101-cp36-cp36m-linux_x86_64.whl (6.6MB)
 6.6MB 7.6MB/s
Requirement already satisfied, skipping upgrade: numpy in /usr/local/lib/python3.6/dist-packages (from torch==1.5.1+cu101) (1.18.5)
Requirement already satisfied, skipping upgrade: future in /usr/local/lib/python3.6/dist-packages (from torch==1.5.1+cu101) (0.16.0)
Requirement already satisfied, skipping upgrade: pillow>4.1.1 in /usr/local/lib/python3.6/dist-packages (from torchvision==0.6.1+cu101) (7.0.0)
Installing collected packages: torch, torchvision
 Found existing installation: torch 1.6.0+cu101
 Uninstalling torch-1.6.0+cu101:
 Successfully uninstalled torch-1.6.0+cu101
 Found existing installation: torchvision 0.7.0+cu101
 Uninstalling torchvision-0.7.0+cu101:
 Successfully uninstalled torchvision-0.7.0+cu101
Successfully installed torch-1.5.1+cu101 torchvision-0.6.1+cu101

```
[2] import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
```

```
[3] m = nn.ZeroPad2d(2)
```

```
[4] input = torch.randn(1, 1, 3, 3)
```

```
[5] input
```

```
tensor([[[[ 0.3803,  0.0879,  0.9542],
           [ 2.6030, -1.6125,  0.7093],
           [-1.6644, -0.5175, -0.9109]]]])
```

```
[6] m(input)
```

```
tensor([[[[ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
           [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
           [ 0.0000,  0.0000,  0.3803,  0.0879,  0.9542,  0.0000,  0.0000],
           [ 0.0000,  0.0000,  2.6030, -1.6125,  0.7093,  0.0000,  0.0000],
           [ 0.0000,  0.0000, -1.6644, -0.5175, -0.9109,  0.0000,  0.0000],
           [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
           [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000]]]])
```

Install PyTorch and Torchvision

```
!pip install -U torch==1.5.1+cu101
torchvision==0.6.1+cu101 -f
https://download.pytorch.org/whl/torch\_stable.html
```

Import packages

Try more examples on

<https://pytorch.org/tutorials/>